

Bork in Action

Język, kompilator i mapa repozytorium

Na podstawie kodu Bork

2026

Spis treści

Przedmowa	7
O tej książce	8
Dla kogo	8
Jak czytać	8
Konwencje	9
Czego książka nie obiecuje	9
O kodzie źródłowym książki	10
O autorze języka i o tej książce	10
Rozdział 1. Pierwszy program i narzędzia	11
Ten rozdział obejmuje	11
Jedno zdanie	11
Dwa pierwsze programy	11
Sprawdzić i zbudować	12
Pierwszy błąd, który zobaczysz często	13
Co już wolno, a czego build nie obniży	13
Drzewo aren, zanim zrozumiesz regiony	14
Podsumowanie	14
Rozdział 2. Filozofia: regiony zamiast GC i borrow checkera	15
Ten rozdział obejmuje	15
Problem, który region ma rozwiązać	15
Pula, nie obiekt	15
Trzy przejścia	16
Pętla nie może przenieść rodzica	17
Czego model nie robi	17
Co jest modelem, a co długiem	18
Kompromis, który warto nazwać	18
Podsumowanie	19
Rozdział 3. Składnia, wiązania i układ pliku	20
Ten rozdział obejmuje	20
Plik	20
Jedna instrukcja na linię	21
Wiązania	21
Blok jest instrukcją i regionem	21
Wyrażenia, których używasz jako instrukcji	22

Łamanie linii w nawiasach	23
Czego parser nie ma	23
Podsumowanie	24
Rozdział 4. Typy	25
Ten rozdział obejmuje	25
Tabela powierzchni	25
Copy	26
Literały	26
Arytmetyka i porównania	27
Nullowalne	28
Typ funkcji	29
Nieznane i trucizna	29
Podsumowanie	29
Rozdział 5. Funkcje, wywołania i domknięcia	30
Ten rozdział obejmuje	30
Kształt funkcji	30
Własność na granicy wywołania	31
Wbudowane	32
Trailing closure	33
Trzy formy domknięcia z move	33
Wywołanie a region	34
Podsumowanie	34
Rozdział 6. Kontrola przepływu	35
Ten rozdział obejmuje	35
if	35
for	36
while, break, continue	37
Zwieranie	38
Wczesny return a regiony	38
Podsumowanie	39
Rozdział 7. Tablice	40
Ten rozdział obejmuje	40
Długość jest typem	40
Indeks	41
Wycinek	41
Napis jako element	41
Reprezentacja	42
Podsumowanie	42
Rozdział 8. Regiony, Copy, Shared i Move	43
Ten rozdział obejmuje	43
Dwa sprawdzania, jedna faza	43
Znaczniki dumpu	43
Wyrażeniowy move	44
promote	44

Assign-up	45
Regionalny move	45
Pętla	46
Shared kontra błąd	46
Podsumowanie	47
Rozdział 9. Gdzie żyją bajty String	48
Ten rozdział obejmuje	48
Deskryptor	48
Reguła 1. Alokujesz tam, gdzie stoisz	48
Reguła 2. Sink przypisania	48
Reguła 3. Hoist dwóch sąsiednich linii	49
Reguła 4. promote, gdy bajty już leżą źle	49
Reguła 5. Escape odrzuca resztę	49
concat i błąd w podręczniku	50
Co jeszcze nie działa	51
Podsumowanie	51
Rozdział 10. Gdy kompilator borks: diagnostyka i aborcja	52
Ten rozdział obejmuje	52
Nie ma obsługi błędów w języku	52
Format	53
Faza parse	53
Faza type	53
Faza ownership	54
Faza codegen	54
Panic kompilatora	55
Jak czytać kilka błędów naraz	55
Podsumowanie	55
Rozdział 11. Biblioteka, brak modułów i idiomy	57
Ten rozdział obejmuje	57
Modułów nie ma	57
Cała biblioteka	57
Idiomy, które warto kopiować	58
Idiomy tylko frontendu	58
Tak się tego nie pisze	59
Styl, który pasuje do regionów	59
Podsumowanie	59
Rozdział 12. Pipeline od źródła do binarki	60
Ten rozdział obejmuje	60
Jedno przejście, dwie bramy	60
CheckResult	60
Dlaczego typeck przed semą	62
Co która struktura pamięta	62
bork build po checku	62
Diagram zależności crate'ów	63
Czego w potoku nie ma	63

Podsumowanie	64
Rozdział 13. Lexer, parser i AST	65
Ten rozdział obejmuje	65
Narzędzie	65
Lekser	65
Piętra	65
AST	66
Layout	67
Z błędu parsowania w diagnostykę	67
Podsumowanie	67
Rozdział 14. Analiza regionów (sema)	68
Ten rozdział obejmuje	68
Sema nie alokuje płyt	68
Struktury	68
Wejście	69
Klasyfikacja użycia	69
Pętla i if	69
Czego sema nie robi	70
Testy, które definiują kontrakt	70
Podsumowanie	70
Rozdział 15. Typeck i typowany HIR	71
Ten rozdział obejmuje	71
Po co osobne drzewo	71
Ty	71
Zejście	72
Literały i oczekiwany typ	72
Wbudowane a zwykłe wołanie	72
HIR a backend	73
Testy	73
Podsumowanie	73
Rozdział 16. Hoist, escape i region_walk	74
Ten rozdział obejmuje	74
Tylko na czystym programie	74
Hoist	74
Escape	75
Jeden spacer	76
Kto dostaje bork_arena_push	76
Podsumowanie	76
Rozdział 17. Bramka, LLVM i runtime	78
Ten rozdział obejmuje	78
Bramka	78
Moduł	78
Wartości	79
Areny w LLVM	79

Wyrażenia, które emisja zna	80
Runtime	80
Linkowanie	81
Weryfikacja, którą ta książka zrobiła	81
Podsumowanie	81
Rozdział 18. Diagnostyki, CLI i serwer LSP	82
Ten rozdział obejmuje	82
Diagnostic	82
CLI	82
Serwer	83
Hover i dump	83
Rozszerzenie	84
Podsumowanie	84
Rozdział 19. Mapa repozytorium	85
Ten rozdział obejmuje	85
Drzewo	85
Jedno zdanie na moduł	86
Jak repo rośło	86
Kolejność czytania	87
Pałapki nawigacji	87
Podsumowanie	89
Rozdział 20. Jeden program przez cały kompilator	90
Ten rozdział obejmuje	90
Program	90
Krok 1. Layout i parse	91
Krok 2. Typeck	91
Krok 3. Sema	91
Krok 4. Stempel, hoist, escape	92
Krok 5. Bramka i moduł	92
Krok 6. Obiekt, link, proces	93
Czego ten ślad nie pokrywa	93
Podsumowanie	93
Rozdział 21. Testy, debugowanie i nowa funkcja języka	95
Ten rozdział obejmuje	95
Testy	95
Debugowanie	96
Ścieżka nowej konstrukcji	96
Dług, który już jest zapisany	97
Pierwszy patch	98
Podsumowanie	98
Rozdział 22. Ćwiczenia	99
Ten rozdział obejmuje	99
Część A. Checker	99
Część B. Binarka	100

Część C. Kod kompilatora	100
Odpowiedzi A	100
Odpowiedzi B	101
Odpowiedzi C	101
Podsumowanie	102
Dodatek A. Ściągawka składni	103
Program	103
Typy	103
Instrukcje	103
Wyrażenia	104
Własność w jednej tabeli	104
Wbudowane	105
Polecenia	105
Fazy diagnostyki	105
Dodatek B. Słowniczek	106
Dodatek C. Niedokończone, rozjazdy i paniki kompilatora	108
Świadomie niezaimplementowane	108
Rozjazdy dokumentu z kodem	108
Paniki i ostre krawędzie, sprawdzone	109
Co jest stabilne	110
Jak czytać niejasność	110

Przedmowa

Bork powstał jako odpowiedź na konkretne tarcie. Rust daje natywny kod i bezpieczeństwo pamięci, ale lifetime'y i borrow checker są osobnym językiem, którego uczy się dłużej niż samej składni. Kotlin daje czytelne bloki, trailing closures i składnię, w której DSL prawie nie wygląda jak DSL. C daje deterministyczny czas życia i zero garbage collector, ale każdy free jest osobną decyzją. Bork stawia te trzy obserwacje obok siebie i wybiera czwartą drogę: **nawiasy klamrowe są regionami pamięci**.

Region nie jest lifetime'em dopiętym do jednej zmiennej. Jest pulą. Wszystko, co w bloku alokuje bajty String albo bufor tablicy, żyje w tej puli. Wyjście z bloku zeruje offset puli jednym ruchem. Nie ma GC, nie ma ogólnego & i &mut, nie ma ręcznego free. Przepływ danych między pulami jest jawny: wartość Copy się kopiuje, val typu nie-Copy dziecko może obserwować w miejscu (Shared), a var trzeba przenieść (move) albo świadomie podnieść (promote).

Ta książka jest przewodnikiem po języku **takim, jaki akceptuje dzisiejszy frontend**, i po kompilatorze **takim, jaki leży w tym repozytorium**. To rozróżnienie jest ważne. Parser i typeck przyjmują nullable, ?:, !!, Some, None i trailing closures. bork build obniża do kodu maszynowego węższy podzbiór i przy kilku konstrukcjach, które frontend uważa za poprawne, albo odmawia diagnostyką fazy codegen, albo — co gorsza — panikuje w procesie kompilatora. Dodatki C wymienia te miejsca bez owijania.

Nazwa diagnostyk jest częścią projektu. Gdy program jest niepoprawny, komunikat mówi, że kompilator borks. Fazy nazywają się parse, ownership, type i codegen. Nie ma osobnego rozdziału o wyjątkach, bo język nie ma wyjątków, Result ani try. Błąd programisty jest albo błędem kompilacji, albo abort w czasie wykonania (dzielenie przez zero, indeks poza tablicą, przepełnienie areny 4096 bajtów).

Książka jest po polsku. Terminy, które w kodzie i w komunikatach kompilatora zostają po angielsku — borrow checker, lexer, arena, IR, HIR, move, Copy, Shared — zostają po angielsku. Tłumaczenie move na „przenieś” w tekście objaśniającym nie zmienia tego, co trzeba napisać w pliku .bork.

Jeśli czytasz to jako autor kompilatora albo jako ktoś, kto chce dopisać kolejną konstrukcję, część III jest właściwym wejściem. Jeśli chcesz najpierw napisać program, zacznij od części I i trzymaj się programów, które bork build naprawdę linkuje. Bramka codegen jest węższa niż gramatyką, i książka powtarza to za każdym razem, gdy różnica ma znaczenie.

O tej książce

Ta książka uczy języka Bork i jednocześnie jest mapą kodu kompilatora napisanego w Rustie. Nie jest specyfikacją życzeń. Jeśli funkcja nie występuje w parserze, typecku, semantyce albo codegenie, nie jest opisana jako część języka. Jeśli jest tylko w specyfikacji z docs/superpowers/specs/ albo w TODO.md, jest oznaczona jako plan albo dług.

Dla kogo

Książka zakłada, że piszesz już w jakimś języku programowania. Najwygodniej, jeśli znasz choć jeden z trzech punktów odniesienia, których Bork używa w dokumentacji projektu:

- Kotlin albo inny język z blokami, i f jako wyrażeniem i trailing lambda,
- Rust albo C++, na tyle żeby słowa „własność”, „przeniesienie” i „alias” nie były puste,
- odrobinę kompilatorów: wiesz, że lexer, parser, AST i backend to kolejne fazy, nawet jeśli nigdy nie pisałeś LALRPOP ani LLVM IR.

Nie musisz znać Inkwell ani tower-lsp. Rozdziały o backendzie tłumaczą te biblioteki w zakresie, w jakim repo ich używa.

Część I da się czytać bez zaglądania do .rs. Część II cytuje nazwy funkcji i plików. Część III zakłada, że masz repo otwarte obok książki.

Jak czytać

Są trzy sensowne przejścia.

Ścieżka języka. Przedmowa, rozdziały 1-11, dodatki A i C. Zatrzymaj się na każdym listingu oznaczonym „uruchomione” i odpal go sam. Listingi oznaczone „tylko check” przechodzą bork plik.bork, ale bork build ich nie obniża albo obniżenie kończy się paniką.

Ścieżka kompilatora. Rozdział 2 (po co regiony), potem 12-18, potem 20. Rozdział 2 przeprowadza jeden krótki program przez parse, typeck, semę, hoist, escape, bramkę i LLVM. To jest najszybszy sposób zobaczyć, które struktury danych naprawdę niosą informację.

Ścieżka kontrybutora. Rozdziały 19–22 i dodatek C. Rozdział 21 opisuje, które testy łapią którą fazę i jak dodać nową konstrukcję, idąc śladem `while` / `break` / `continue`, które w kodzie już są.

TIP

TIP. Komunikaty w książce pochodzą z uruchomienia `target/debug/bork`. Format linii to `plik:linia:kolumna: error: faza: treść`. Kolumna liczy znaki Unicode od początku linii, nie bajty. Gdy diagnostyka nie ma spanu, CLI pomija numer linii. Tak jest na przykład przy `returning the result of concat`.

Konwencje

Listing 4.2. tak podpisany jest numerowany w obrębie rozdziału. Pod listingiem numer w nawiasie, (1), odnosi się do komentarza w kodzie.

Ramki:

NOTE

NOTE. Fakt o modelu albo o kodzie, który łatwo przegapić.

TIP

TIP. Co zrobić w praktyce.

WARNING

WARNING. Pułapka: program, który `typeck` przyjmie, a `bork build` odrzuci albo na którym kompilator `spanikuje`. Osobno: zachowanie `runtime`, które kończy proces.

Kod Bork jest w blokach `bork`. Kod Rusta, który jest częścią kompilatora, jest w blokach `rust` i ma ścieżkę pliku w podpisie.

Diagramy są w Mermaid. W PDF są wyrenderowane do obrazków.

Czego książka nie obiecuje

Bork w tej rewizji nie ma:

- modułów, `import`, pakietów ani przestrzeni nazw,
- struktur, enumów użytkownika, `trait`ów, generyków,
- wyjątków, `panic` jako konstrukcji języka, typu `Result`,
- operatora jednoargumentowego minusa (literał ujemny nie parsuje się),
- średników jako separatorów instrukcji,
- `garbage collector`,
- referencji `&` / `&mut` i `borrow checker`a w stylu Rusta,
- JIT, interpretera `bork run`, `debugger`a,
- `codegen`u dla trailing closures, `Some`, `None`, `?:` i `!!`.

String przekazywany do funkcji użytkownika jest akceptowany przez frontend, ale bork build na tej ścieżce panikuje w `value_as_int`. To błąd kompilatora, nie reguła języka. Szczegóły są w rozdziale 17 i dodatku C.

O kodzie źródłowym książki

Rozdziały leżą w `docs/book/`. Przykłady, które uruchamialiśmy, są w `docs/book/przyklady/` razem z plikiem `WYNIKI.md`. Historia commitów projektu (parser, sema, typowany HIR, LLVM, sink/hoist/escape, tablice, LLVM 23) jest tłem rozdziału 19; książka nie streszcza każdego pull requestu.

O autorze języka i o tej książce

Język i kompilator są projektem Andrzeja Witkowskiego (historia git na main). Ta książka jest opisem tego kodu dla innych programistów. Nie zmienia semantyki. Gdy dokumentacja w `docs/language.md` rozjeżdża się z kompilatorem, książka staje po stronie kompilatora i mówi o rozjeździe wprost. Najważniejszy przykład: rozdział o konkatencji w `docs/language.md` woła `concat` na dwóch `var String` bez `move`. Frontend tego nie przyjmuje.

Rozdział 1. Pierwszy program i narzędzia

Ten rozdział obejmuje

- Co Bork jest, a czego w tej rewizji nie jest
- Program, który drukuje wiersz, i program, którego kodem wyjścia jest wynik
- Różnicę między `bork plik.bork` a `bork build`
- Jak czytać pierwszą diagnostykę
- Gdzie kończy się „język”, a zaczyna „podzbiór codegen”

Jedno zdanie

Bork jest małym językiem natywnym. Składnia jest zbliżona do Kotlin: `fun`, `val`, `var`, bloki w `{}`, `if` jako wyrażenie, trailing closure. Pamięcią nie zarządza garbage collector. Bajty wartości, które nie mieszczą się w rejestrze — dziś przede wszystkim `String` i tablice `[T; N]` — leżą w arenach przypiętych do bloków. Kompilator, gdy program jest zły, wypisuje diagnostykę i kończy się kodem 1. Stąd hasło z README: `when your code is invalid, the compiler borks`.

Repozytorium jest kompilatorem napisanym w Rustcie. Nie ma tu interpretera. Ścieżka natywna idzie przez LLVM (biblioteka Inkwell, pin na LLVM 23) i małą bibliotekę runtime `bork_runtime`, linkowaną statycznie do programu użytkownika. Sprawdzenie programu — `parse`, typy, własność — nie potrzebuje LLVM. Zbudowanie binarki potrzebuje.

Dwa pierwsze programy

Listing 1.1. Drukuję `hi` i nową linię. Uruchomione: kod wyjścia 0, `stdout hi\n`.

```
fun main() {  
    println("hi")  
}
```

`fun main` bez `:` Typ zwraca `unit`. Taki `main` kompiluje się do funkcji `main` w C ABI, która zwraca `i32` równe 0. `println` jest wbudowany. Nie jest funkcją, którą wolno zdefiniować ponownie.

Listing 1.2. Wynik funkcji jest kodem wyjścia procesu. Uruchomione: kod 42.

```
fun add(a: i32, b: i32): i32 {
    return a + b
}
```

```
fun main(): i32 {
    return add(40, 2)
}
```

- (1) Parametr bez val/var jest val.
- (2) main z wynikiem i32 przekazuje tę liczbę jako kod wyjścia. Innych typów wyniku main codegen nie obniża: i64, f64 i bool dostają diagnostykę main returning ... is not supported by codegen yet.

NOTE

NOTE. Nie ma instrukcji na poziomie pliku. Program to lista funkcji. main jest zwykłą funkcją o zastrzeżonej nazwie tylko na etapie bork build. Samo sprawdzenie pliku bez main przechodzi, jeśli reszta jest poprawna.

Sprawdzić i zbudować

Z katalogu repozytorium, bez LLVM, wystarczy checker:

```
cargo run --bin bork -- path/to/file.bork
cargo run --bin bork -- --dump-arenas path/to/file.bork
```

Pierwsza komenda uruchamia frontend::check: parse, typeck, analizę własności, a przy czystym programie także hoist i escape. Druga dopisuje na stdout drzewo aren. Diagnostyki idą na stderr. Kod wyjścia 0 oznacza brak diagnostyk, 1 oznacza błąd programu, 2 oznacza złą linię poleceń albo brak pliku.

Zbudowanie binarki wymaga feature codegen, LLVM 23, clang i bibliotek, które ciągnięte są z libLLVM (libz3, libedit, libxml2, libzstd, libffi). Na Ubuntu 24.04 pakietu LLVM 23 nie ma w domyślnym archiwum; README wskazuje apt.llvm.org. Po złożeniu:

```
cargo run --features codegen -- build file.bork
cargo run --features codegen -- build -o out file.bork
```

Domyślna nazwa wyjścia to ścieżka wejścia bez rozszerzenia. Kompilator odmawia, gdy wyjście nadpisałoby plik źródłowy (error: output path would overwrite input file, kod 2). Bez feature codegen podkomenda build kończy się komunikatem, że bork build wymaga złożenia z tą flagą, też kodem 2.

Listing 1.3. Diagnostyka braku main. Uruchomione, bork build, kod 1, binarka nie powstaje.

```
nomain.bork: error: codegen: `fun main` is required to build an executable
```

Źródło to jedna funkcja helper. Faza to codegen, nie type. Frontend uważa plik za poprawny.

Pierwszy błąd, który zobaczysz często

Listing 1.4. Dwie instrukcje w jednej linii. Uruchomione.

```
fun main() { val a = 1 val b = 2 }
```

```
err_two_stmt.bork:1:24: error: parse: unexpected token `val`; expected "!!",  
  ↪  "!=" , ... "}", "NL"
```

Instrukcje rozdziela się znakiem nowej linii, nie średnikiem. Średnik jest nieoczekiwanym tokenem. Lista expected jest surowa: LALRPOP wypisuje terminale, których mógłby użyć w tym stanie parsera. Nie jest to przyjazny tekst i książka nie będzie udawać, że jest.

Listing 1.5. Przypisanie do val. Uruchomione.

```
fun main() {  
    val n = 1  
    n = 2  
}
```

```
err_val.bork:3:5: error: type: cannot assign to immutable `val` binding `n`
```

Faza type przychodzi z typecku. Faza ownership przychodzi z semy i z analizy escape. Obie mogą pojawić się w jednym uruchomieniu. Kolejność w wyjściu jest taka, że komunikaty własności idą pierwsze, nawet gdy typeck policzył się wcześniej. Rozdział 12 tłumaczy, dlaczego pipeline jest w tej kolejności.

Co już wolno, a czego build nie obniży

Następujący program przechodzi bork plik.bork (sprawdzone, kod 0) i jest próbką z src/lib.rs (MVP_SAMPLE). bork build kończy się kodem 1 i komunikatem zawierającym codegen oraz not supported. Test build_rejects_mvp_sample_at_codegen_gate pilnuje dokładnie tego.

Listing 1.6. Trailing closure. Tylko check. Codegen odrzuca.

```
fun action(a: Int, b: Int, block: (Int, Int) -> Int): Int {  
    return block(a, b)  
}
```

```
fun main(): i32 {  
    val threshold = 5  
    var accumulator = 0  
    for (i in 0..10) {  
        accumulator = action(accumulator, i) { acc, current ->  
            if (current > threshold) {  
                acc + current  
            } else {  
                acc  
            }  
        }  
    }  
}
```

```

    }
    return accumulator
}

```

To jest język, nie martwa gramatyka. Typeck sprawdza arność domknięcia i typ wyniku. Sema buduje węzeł Closure w drzewie aren. Bramka w src/codegen/gate.rs odrzuca has_trailing_closure tekstem trailing closures are not supported by codegen.

WARNING

WARNING. Część programów, które check puszcza, nie dostaje grzecznej diagnostyki na bork build, tylko panic procesu kompilatora (kod 101). Najważniejszy przypadek: argument String do funkcji użytkownika. println i concat działają. Rozdział 17 pokazuje miejsce w coerce_value_to_ty.

Drzewo aren, zanim zrozumiesz regiony

Dla listingu 1.2 --dump-arenas wypisuje:

```

Arenas
├── fun add
│   ├── a [Local]
│   └── b [Local]
└── fun main

```

Local znaczy: nazwa powstała w tym regionie. main nie ma lokalnych nazw, bo jedyną instrukcją jest return. W rozdziale 8 zobaczysz Copy, Shared ← ... i Moved ← Dump jest tym samym tekstem, który komenda edytora **Bork: Dump Arenas** wkłada do kanału wyjścia.

Podsumowanie

- Program Bork to lista funkcji. Instrukcje rozdziela nowa linia.
- bork plik.bork sprawdza. bork build obniża podzbiór do binarki przez LLVM.
- main(): i32 zwraca kod wyjścia. main() bez typu kończy się zerem.
- Diagnostyka ma fazę: parse, ownership, type albo codegen.
- Trailing closures, nullable i ?: są w języku frontendu. Codegen ich nie obniża.
- Przekazanie String do funkcji użytkownika jest dziś dziurą codegenu: panic, nie komunikat.

Rozdział 2. Filozofia: regiony zamiast GC i borrow checkera

Ten rozdział obejmuje

- Dlaczego Bork nie ma GC i nie ma ogólnych referencji
- Czym region różni się od lifetime'u jednej zmiennej
- Trzy sposoby przekroczenia granicy regionu: Copy, Shared, Move
- Co jest modelem docelowym, a co jest jeszcze długiem
- Czego język świadomie nie planuje

Problem, który region ma rozwiązać

W programie systemowym są dwa rodzaje danych. Liczby, boole i unit mieszczą się w rejestrze albo na stosie. Tekst i bufor tablicy nie: to adres plus długość, a bajty leżą gdzie indziej. Pytanie brzmi, kto te bajty zwalnia i skąd kompilator wie, że wskaźnik jeszcze żyje.

GC odpowiada: runtime kiedyś to zbierze, a programista nie wskazuje miejsca. Borrow checker Rusta odpowiada: każda referencja ma lifetime dopięty do konkretnego wypożyczenia, a checker dowodzi, że wypożyczenie nie przeżywa właściciela. Oba rozwiązania są dobre w swoich niszach. Oba mają koszt, który Bork nie chce płacić. GC psuje determinizm czasu życia. Ogólny borrow checker wymaga składni i diagnostyk, których ten projekt celowo nie buduje.

Bork odpowiada trzecią regułą, zapisaną w docs/memory-model.md i zrealizowaną w semie oraz w bork_runtime:

Nawiasy klamrowe otwierają pulę. Alokacja w puli to bump wskaźnika. Wyjście z bloku ustawia offset na zero. Cała pula wraca na listę wolnych płyt. Nie ma zwalniania pojedynczego obiektu w środku puli.

Pula, nie obiekt

Listing 2.1. Dwa czasy życia. Uruchomione: `stdout b\n`, kod 0.

```
fun main() {  
    var outer = "a"  
    {
```



```

        outer = "b"
    }
    println(outer)
}

```

outer powstał w regionie funkcji. Przypisanie w środku bloku nie „używa zmiennej rodzica w sposób zabroniony”. To mutacja wiązania, które żyje dłużej. Bajty nowego napisu lądują w puli outer, nie w puli wewnętrznego bloku, który za chwilę zniknie. To jest reguła sink allocation z rozdziału 9. Na tym listingu widać skutek bez słownictwa kompilatora: po } napis "b" nadal wolno wydrukować.

Gdybyś napisał `var inner = "b"` i próbował użyć `inner` za zamykającym nawiasem, nazwa w ogóle nie jest w zasięgu. To osobna sprawa od `aren`. Region nie przedłuża zasięgu leksykalnego. Zasięg i pula są złączone, ale nie są tym samym: nazwa umiera według bloków, a bajty umierają według areny, w której zostały położone.

Trzy przejścia

Przejście	Kiedy	Co zostaje u rodzica
Copy	Nie-null typ prymitywny (i32, bool, f64, ...)	Wartość żyje dalej, dziecko ma kopię bitów
Shared	val typu nie-Copy, czytany w dziecku	Wiązanie żyje, dziecko patrzy na te same bajty
Move	var typu nie-Copy albo jawne move	Wiązanie źródłowe jest martwe

Listing 2.2. Shared. Uruchomione: `stdout x\n`.

```

fun main() {
    val s = "x"
    {
        println(s)
    }
}

```

Dump:

Arenas

```

└─ fun main
   └─ s [Local]
      └─ Block
         └─ s [Shared ← fun main]

```

Dziecko nie kopiuje znaków i nie przejmuje własności. Stos wywołań (tu: stos regionów) gwarantuje, że rodzic żyje dłużej niż dziecko. Dlatego Shared jest bezpieczne bez borrow checkera: nie da się schować wskaźnika z dziecka do rodzica, bo język nie ma typu referencji, którą można zapisać w polu.

Listing 2.3. Move zabija nazwę. Uruchomione checker; diagnostyka cytowana dosłownie.

```
fun main() {
    var s: String = "hi"
    val t = move s
    val u = s
}
```

err_use_after.bork:4:13: error: ownership: use of `s` after move from fun main

Move nie wycina dziury w arenie rodzica. Stare bajty zostają martwym miejscem aż do reset puli. Kompilator unieważnia **wiązanie**. To jest tańsze niż kompaktor areny i wystarcza, dopóki nie ma wskaźników w pola struktur — a struktur użytkownika nie ma.

move na wartości Copy jest no-opem własności. Program `val n = 1 / val m = move n / return n + m` przechodzi check i bork build, kod wyjścia 2. Nazwa `n` zostaje żywa. Dump nie oznacza jej jako Moved.

Pętla nie może przenieść rodzica

Pętla wykonuje ciało wiele razy. Własność zewnętrznego `var` da się oddać tylko raz. Sema trzyma zbiór `loop_move_ban`: nazwy widoczne przy wejściu do `for` albo `while` nie mogą być źródłem `move` w ciele.

Listing 2.4. Uruchomione checker.

```
fun main() {
    var s = "Hello, World!"
    for (i in 1..10) {
        move {
            val t = s
        }
    }
}
```

err_loop.bork:5:21: error: ownership: cannot move `s` inside a loop: it would
 ↪ already be moved on later iterations

err_loop.bork:5:21: error: ownership: `s` is not Copy; move it into
 ↪ `MoveBlock (move)` with `move`

Dwa komunikaty na ten sam span są uczciwe: pierwszy z `move_source` (zakaz pętli), drugi z klasyfikacji użycia, bo wnioskowany `move { }` nie zdążył przechwycić `s`. Lokalny `var` utworzony **wewnątrz** ciała pętli wolno przenieść. Każda iteracja dostaje świeże wiązanie. Rozdział 8 ma działający przykład.

Ciało pętli resetuje swoją arenę co iterację (`bork_arena_reset`), więc pamięć pętli zostaje $O(1)$ względem liczby obrotów, o ile nie ucieka do zewnętrznego `var`. To jest osobna oś od zakazu `move`: nawet poprawna pętla, która alokuje napis tymczasowy, nie rośnie liniowo po arenach.

Czego model nie robi

`docs/memory-model.md` ma jawną listę „Not planned”:

- ogólne & / &mut i referencje w polach między regionami,
- niejawna głęboka kopia przy każdym odczycie spoza regionu,
- zapisanie wypożyczenia z dziecka w polu rodzica.

Nie ma też typu, który mówiłby „ta referencja żyje tyle co tamten obiekt”. Jedyna dozwolona obserwacja krótsza niż właściciel to Shared z val rodzica do dziecka, które i tak jest zagnieżdżone leksykalnie.

Co jest modelem, a co długiem

Model docelowy puli jest zaimplementowany:

- pojemność jednej płyty to **4096** bajtów (ARENA_CAPACITY w crates/bork_runtime/src/lib.rs i bliźniaczy model w src/arena.rs),
- bork_arena_push bierze płytę z puli, pop oddaje, reset zeruje offset bez oddawania płyty (pętla),
- przepełnienie płyty **panikuje** runtime’u Rusta (arena overflow: need N bytes, capacity 4096), nie zwraca błędu do programu Bork.

Dług, opisany w tym samym pliku jako „Still open” i widoczny w escape.rs:

- brak areny wyniku po stronie wołającego, więc return concat(...) i return move ... są odrzucane,
- hoist działa tylko dla pary sąsiednich instrukcji var i zaraz potem outer = move var,
- promote nie działa na return,
- codegen nie eliduje memcpy, gdy bajty już leżą w docelowej puli.

WARNING

WARNING. src/arena.rs to model bump-allocator używany jako opis i testy koncepcji. Sema go nie woła. Sema buduje drzewo ArenaNode. Prawdziwe płytki 4 KiB powstają w bork_runtime, gdy bork build wyemituje bork_arena_push. Nazwa „arena” w dwóch crate’ach znaczy dwie różne struktury. Rozdział 16 trzyma je osobno.

Kompromis, który warto nazwać

Region jest gruboziarnisty. W bloku, który buduje jeden krótki napis i jeden duży bufor, oba dzielą 4096 bajtów i umierają razem. Nie da się zwolnić napisu wcześniej. To jest cena braku free i braku GC. Dla MVP jest przyjęta świadomość: stały rozmiar płyty jest przyjazny cache’owi i upraszcza codegen (jeden wskaźnik offsetu, jeden reset). Gdy program potrzebuje danych dłuższych niż blok, jedyna droga to położyć je w var zewnętrznego regionu przez przypisanie, move albo promote. Nie ma sterty „na zawsze” poza regionem funkcji i poza literałami w stałych programu.

Literał "hi" może w ogóle nie leżeć w arenie. Codegen kładzie bajty w prywatnym globalu LLVM, a w wartości zostaje deskryptor { ptr, len }. Wtedy koniec bloku nie unieważnia znaków. Kompilator i tak śledzi własność wiązania. Stały napis nie jest pretekstem do używania nazwy po move.

Podsumowanie

- Blok { } jest regionem: bump alokacji, zbiorcze zwolnienie przy wyjściu.
- Copy, Shared i Move to trzy legalne przejścia granicy. Innych nie ma.
- Move unieważnia wiązanie, nie kompaktuje areny.
- Pętla nie przenosi wiązania spoza ciała i resetuje swoją pulę co iterację.
- Pojemność płyty to 4096 bajtów. Przepełnienie panikuje runtime.
- Ogólne referencje nie są w planie. Świeżo zaalokowany String zwracany z funkcji też jeszcze nie: brakuje areny wyniku po stronie wołającego.

Rozdział 3. Składnia, wiązania i układ pliku

Ten rozdział obejmuje

- Kształt pliku: funkcje, komentarze, nowe linie
- `val` i `var`, cień nazw, adnotacje
- Które słowa są słowami kluczowymi, a które tylko terminalami gramatyki
- Wyrażenia, które wyglądają jak instrukcje
- Layout nawiasów okrągłych, czyli dlaczego wywołanie może złamać linię

Plik

Plik `.bork` to zero lub więcej funkcji. Pusta treść i sam znak nowej linii parsują się do `Program { functions: vec![] }`. Komentarz `//` biegnie do końca linii i jest zjadany przez lexer, zanim parser zobaczy tokeny.

Listing 3.1. Komentarz nie psuje instrukcji. Uruchomione: kod wyjścia 1.

```
fun main(): i32 {  
    // komentarz  
    val n = 1 // tez  
    return n  
}
```

Nie ma komentarza blokowego. Nie ma atrybutów, `pragm` ani `import`.

Słowa rozpoznawane w lexerze jako osobne tokeny to: `if`, `fun`, `val`, `var`, `for`, `in`, `return`, `Some`, `None`, `move`, `promote`. Słowa `while`, `break`, `continue`, `true`, `false` są zwykłymi terminalami gramatyki: lexer oddaje je jako identyfikatory-słowa, a parser oczekuje konkretnego napisu. Skutek praktyczny jest taki, że nie użyjesz ich jako nazwy, bo produkcja identyfikatora nie pokrywa się z tymi literałami tam, gdzie gramatyka oczekuje słowa. `else` ma osobną regułę: przed `else` wolno złamanie linii i komentarz, i to nadal jest token `else`, nie `NL` plus `else`. Dzięki temu `if` w następnej linii może mieć `else`.

Identyfikator to `[A-Za-z_][A-Za-z0-9_]*`. Nie ma unicode w nazwach.

Jedna instrukcja na linię

Listing 3.2. Średnik. Uruchomione.

```
semi.bork:2:14: error: parse: unexpected token `;`; expected ... "}" , "NL"  
dla źródła val n = 1;.
```

Parser traktuje NL jako separator w ciele bloku. Pusta linia jest zlepiana przez regułę „jednego lub więcej” złamań, więc kilka pustych linii między instrukcjami jest w porządku. Dwie instrukcje bez NL nie są.

TIP

TIP. Gdy komunikat mówi expected "NL", prawie zawsze dwie rzeczy stoją w jednej linii albo został średnik przywieziony z C lub Rusta.

Wiązania

Listing 3.3. val jest stałe, var można przypisać. Fragment uruchomiony w większych programach.

```
val n = 1  
var count: i32 = 0  
var s: String = "hi"  
count = count + 1
```

Reguły, które typeck naprawdę egzekwuje:

- : Typ jest opcjonalne, gdy inicjalizator wyznacza typ.
- Przypisanie do val to cannot assign to immutable val binding.
- Nazwa w bloku przesłania zewnętrzną do końca bloku.
- Cień może czytać zewnętrzną nazwę w swoim inicjalizatorze. Program val n = 1 / w bloku val n = n + 1 / return n daje kod wyjścia 2. Dump pokazuje w bloku lokalne n oraz obserwację n [Copy] — odczyt zewnętrznej wartości Copy.

Przypisanie po lewej stronie może być tylko nazwą albo nazwa[indeks]. Pole, wywołanie i dowolne inne wyrażenie dostają błąd użytkownika gramatyki: assignment target must be a name or name[index]. Ten błąd, jako ParseError::User, dostaje span **na końcu pliku**, nie na lewym składniku. To jest ograniczenie diag::from_parse, nie decyzji językowej.

var i val przy parametrze omawia rozdział 5. Przy lokalnej zmiennej słowo stoi na początku instrukcji i jest obowiązkowe: nie ma gołego n := 1 ani n = 1 jako deklaracji.

Blok jest instrukcją i regionem

```
{  
    val m = n  
}
```

Blok zagnieżdżony otwiera region semy z etykietą Block. Kilka warstw nawiasów, które nie zawierają nic poza jednym wewnętrznym blokiem, sema skleja. `peel_blocks` zdejmuję takie opakowania.

Listing 3.4. Trzy warstwy, jeden region. Uruchomione checker.

```
fun main() {
  val n = 1
  {
    {
      {
        val m = n
      }
    }
  }
}

└─ fun main
   └─ n [Local]
      └─ Block (compacted 2 braces)
         └─ m [Local]
            └─ n [Copy]
```

Zewnętrzny `{` zostaje regionem. Dwa kolejne, które tylko owijają, znikają z drzewa i zostają licznikiem `compacted_braces`. To nie jest optymalizacja „na oko”: codegen i sema muszą zdejmować tak samo, inaczej rozjadą się dzieci `ArenaNode` z miejscami w HIR. Funkcja `hir::peel_blocks` jest bliźniacza i komentarz w sema każe je trzymać w zgodzie.

`if`, `for`, `while`, `fun` i domknięcie nie sklejają się z otoczeniem. Każde ma własną etykietę (`IfThen`, `IfElse`, `ForLoop (i)`, `WhileLoop`, `fun nazwa`, `Closure`).

Wyrażenia, których używasz jako instrukcji

Każde wyrażenie może być instrukcją (`Stmt::Expr`). Wartość jest wtedy wyrzucana. `if` bez `else` jest typowym przypadkiem: służy do sterowania, a jego typem wyniku przy braku oczekiwanego typu jest `unit`.

Listing 3.5. `if` sterujący. Uruchomione: kod 1.

```
fun main(): i32 {
  var n = 0
  if (true) {
    n = 1
  }
  return n
}
```

Gdy `if` ma produkować wartość, obie gałęzie muszą być blokami i musi być `else`. Szczegóły typów są w rozdziale 6. Tu ważna jest składnia: warunek jest w nawiasach okrągłych, gałęzie są blokami, nie pojedynczymi wyrażeniami. Nie napiszesz `if n > 0 n else 0`.

Łamanie linii w nawiasach

Wywołanie i każda para (...) może złamać linię. Przed parserem `layout::normalize_parenthesized` zamienia `\n` na `\r` wewnątrz nawiasów okrągłych na tej samej głębokości klamer, pomijając stringi i komentarze. Lexer traktuje `\r` jak zwykły biały znak, więc NL nie rozdziela instrukcji w środku listy argumentów.

Listing 3.6. Uruchomione: `add(1, 2)` zwraca 3.

```
fun add(a: i32, b: i32): i32 {  
    return a + b  
}  
  
fun main(): i32 {  
    return add(  
        1,  
        2  
    )  
}
```

NOTE

NOTE. Normalizacja nie przesuw offsetów diagnostyk w sposób, który gubiłby miejsce błędu: `\n` i `\r` mają po jednym bajcie, więc spany zostają. Testy w `tests/parser.rs` pilnują wieloliniowego wywołania i stabilnego miejsca błędu.

Poza nawiasami złamanie linii jest znaczące. Nie ma kontynuacji wyrażenia „bo linia skończyła się operatorem”, w stylu niektórych języków. Operator na końcu linii kończy wyrażenie albo jest błędem parsowania.

Czego parser nie ma

Sprawdzone, kod 1, faza parse:

Źródło	Komunikat
<code>struct Point { x: i32 }</code>	unexpected token struct; expected "NL", "fun"
<code>module foo</code>	unexpected token module; expected "NL", "fun"
<code>return -1</code>	unexpected token - wśród atomów

Minus jest tylko operatorem binarnym (+ i - na tym samym piętrze). Liczbę ujemną da się dostać odejmowaniem: `0 - 1`. Nie ma literału ujemnego. To ogranicza też testy dzielenia `i32::MIN / -1`, które codegen umie obsłużyć w `guard_int_div`, ale których nie da się zapisać jako literału.

`fun` zagnieżdżone w funkcji nie istnieje. Funkcje są tylko na poziomie programu.

Podsumowanie

- Plik to lista fun. Komentarz to tylko //.
- Instrukcję kończy nowa linia. Średnik jest błędem parsowania.
- `val` nie przyjmuje przypisania. `var` przyjmuje. Cień nazw działa i może czytać przesłaniane imię w inicjalizatorze.
- Gole wielokrotne `{ { { } } }` sklejają się do jednego regionu.
- Wewnątrz `(...)` nowe linie są białym znakiem. Poza nimi są separatorami.
- Nie ma `struct`, modułów ani jednoargumentowego minusa.

Rozdział 4. Typy

Ten rozdział obejmuje

- Typy prymitywne i aliasy Int, Long, Byte, Float, Double
- Które typy są Copy
- Literały i wnioskowanie
- String, funkcje, nullowalność
- Błędy, które typeck wypisuje, gdy typy się nie zgadzają

Tabela powierzchni

Piszesz	Po obniżeniu
i8 i16 i32 i64	ten sam typ całkowity ze znakiem
u8 u16 u32 u64	ten sam typ całkowity bez znaku
Int	i32
Long	i64
Byte	u8
Float	f32
Double	f64
f32 f64	floaty
bool	bool, literały true i false
unit	brak wartości użytecznej; pominięty typ wyniku funkcji
String	jedyny dozwolony typ nazwany
[T; N]	tablica o długości stałej
T?	wariant nullowalny
(A, B) -> R	typ funkcji

Alias jest rozwiązywany w `Type::from_ident` w parserze, zanim typeck zobaczy drzewo. W HIR nie ma osobnego konstruktora Int. Jest `Prim::I32`.

Listing 4.1. Alias. Uruchomione: `bork build`, kod wyjścia 1 (zwracane jest n).

```
fun main(): i32 {  
    val n: Int = 1  
    val wide: Long = 2  
    val byte: Byte = 3  
    val f: Float = 1.5
```

```

    val d: Double = 2.25
    val flag: bool = true
    return n
}

```

Sam zapis tych lokalnych wartości codegen obniża. Użycie floatu w arytmetyce albo porównaniu jest osobną, niedokończoną ścieżką (rozdział 17). Ten listing ich nie używa, dlatego binarka powstaje.

Każda inna nazwa typu, na przykład Point albo Unit z dużej litery, daje unknown named type. Słowo unit małymi literami jest prymitywem. Wielka litera Unit nie jest aliasem.

Copy

Ty::is_copy jest prawdziwe tylko dla prymitywu, który nie jest nullowalny. Spoza Copy wypadają:

- String,
- każda tablica [T; N],
- każdy typ funkcji,
- cokolwiek z ?, także i32?.

To jest ta sama odpowiedź w HIR i w semie (sema mapuje typ HIR z powrotem na ast::Type albo zostawia Unknown, a Unknown **nie** jest Copy). Dlatego błąd typu, który zostawia typ nieznany, potrafi pociągnąć dodatkowy błąd własności. Nie ignoruj drugiej diagnostyki tylko dlatego, że „to już był błąd typu”.

Literały

Literał całkowity jest w AST liczbą i64, a typ przyjmuje z kontekstu, jeśli kontekst jest całkowity. Bez kontekstu staje się i32.

Listing 4.2. Literał dopasowany do i64. Uruchomione checker. bork build odrzuca main zwracające i64.

```

fun main(): i64 {
    val n: i64 = 1
    return n
}

```

```

int_widen.bork: error: codegen: `main` returning `i64` is not supported by
↳ codegen yet

```

Checker jest zadowolony. Bramka wyniku main nie.

Literał 1.5 bez adnotacji staje się f64. Z adnotacją Float albo f32 zostaje f32. Nie ma sufiksów 1.5f32.

Literał napisowy używa cudzysłowów i escape'ów \n, \r, \t, \\, \". **Listing 4.3.** Uruchomione, stdout to znak a, nowa linia, b, tabulacja, "c", nowa linia.

```
fun main() {
    println("a\\nb\\t\\c\\")
}
```

+ nie skleja napisów.

```
str_plus.bork: error: type: arithmetic operands must have the same numeric
↳ type, got String and String
```

Do sklejania jest concat (rozdział 9 i 11).

Arytmetyka i porównania

Operatory + - * / wymagają dwóch operandów tego samego typu liczbowego. Nie ma promocji i32 do i64.

Listing 4.4. Uruchomione checker. Dwa błędy na tym samym miejscu.

```
fun main(): i32 {
    val a: i64 = 1
    val b: i32 = 2
    return a + b
}
```

```
err_div_types.bork:4:12: error: type: arithmetic operands must have the same
↳ numeric type, got i64 and i32
```

```
err_div_types.bork:4:12: error: type: return value has type i64, expected i32
```

Drugi komunikat jest skutkiem ubocznym: przy niezgodnych operandach typeck zostawia typ lewej strony jako typ wyrażenia, a ten nie pasuje do i32.

Porównania > < >= <= wymagają tego samego nienulowalnego typu liczbowego. bool odpada:

```
cmp_bool.bork: error: type: ordered comparison operands must have the same
↳ non-nullable numeric type, got bool and bool
```

== i != wymagają identycznego typu, ale typ może być nulowalny. val n: i32? = None oraz n == None przechodzi checker. Codegen odrzuca None.

&&, || i ! działają na bool. Koniunkcja i alternatywa są zwierające. To widać w codegenie: prawa strona false && side() nie jest wywoływana. Test builds_logical_short_circuit i listing 6.4 to uruchamiają.

Dzielenie całkowite przez zero oraz, w kodzie backendu, dzielenie minimum typu ze znakiem przez -1 woła abort. Uruchomione: return 1 / 0 buduje się, a proces kończy sygnałem SIGABRT (kod powłoki 134). Komunikatu Bork na stderr nie ma. To abort z libc, nie panic Rusta z treścią.

NOTE

NOTE. Floaty w typecku wolno dodawać i porównywać. W codegenie dodawanie f32 daje diagnostykę internal arena schedule mismatch: float binary after walk is not supported by codegen yet. Porównanie f64 na ścieżce, którą

trafililiśmy, panikuje w `into_int_value`, bo wartość LLVM jest `double`, a `coerce` oczekuje `inta`. Traktuj arytmetykę `float` jako sprawdzaną, nie jako obniżaną.

Nullowalne

`T?` istnieje dla prymitywów, dla `String` i dla typu funkcji. Tablica nullowalna parsuje się i jest odrzucana: `nullable array types [T]? are not supported`.

Konstruktory to `Some(x)` i `None`. `None` bez oczekiwanego typu nie przechodzi:

```
err_none.bork:2:13: error: type: cannot infer type of `None`
```

`?:` wymaga lewej strony nullowalnej. Prawa strona musi pasować do typu po zdjęciu `?`.

```
elvis_bad.bork:3:13: error: type: left operand of `?:` must be nullable, got  
↪ i32
```

`!!` wymaga operandu nullowalnego i daje typ bez `?`. Na `i32` dostaniesz `operand of !! must be nullable, got i32`.

Listing 4.5. Nullowalny `String` i `i32`. Tylko `check`. Codegen odrzuca `?:`, `Some` i `None`.

```
fun processUser(name: String?, score: i32): i32 {  
    val fallbackName: String = name ?: "Guest"  
    val verifiedUser: String? = Some(fallbackName)  
    val emptyMiddle: String? = None  
    val verifiedLength: i32 = verifiedUser?.length ?: 0  
    if (verifiedLength > 0) {  
        return score  
    }  
    return 0  
}
```

`?.` jest bezpiecznym dostępem do pola. Na typie nienullowalnym `typeck` każe użyć `..`. Jedyne znane pole `String` i `[T; N]` to `length`, typu `i32`. Dla tablicy `length` jest zawsze `N` z typu, nie osobnym licznikiem.

```
unknown_field.bork:3:13: error: type: unknown field `foo` on type String
```

WARNING

WARNING. `bork build` na listingu 4.5 wypisuje osobny błąd codegen dla każdego `?:`, `Some` i `None`. To nie jest „częściowy runtime nullable”. Bramka odcina całe konstrukcje, zanim LLVM je zobaczy.

Typ funkcji też bywa nullowalny: `val f: ((i32) -> i32)? = None` przechodzi checker. Wywołanie takiego `f` bez `!!` nie jest tym, co codegen umie, i nawet po `!!` bramka odrzuca asercję.

Typ funkcji

$(A, B) \rightarrow R$ zapisuje się w nawiasach. Nullable funkcja to $((A, B) \rightarrow R)?$. Gramatyka odrzuca nullowanie nawiasów, które nie są typem funkcji: nullable parentheses require one function type oraz only function types may be parenthesized for nullability.

Nie ma krotek jako wartości. Nawias przy typie jest albo grupowaniem typu funkcji, albo — w wyrażeniu — zwykłym nawiasem arytmetycznym. Nie ma $(1, 2)$ jako pary.

Nieznane i trucizna

Gdy wyrażenie jest już błędne, typeck podstawia `TyKind::Unknown`. Kolejne niezgodności z `Unknown` są tłumione, żeby jeden błąd nie produkował kaskady. To nie dotyczy semy: typ, którego nie da się zmapować, jest tam nie-Copy. Własność nadal może krzyczeć.

Podsumowanie

- Aliasy `Int/Long/Byte/Float/Double` znikają już w AST.
- Copy to nienulłowny prymityw. `String`, tablice, funkcje i `T?` nie są Copy.
- Nie ma promocji liczbowych ani `+` dla napisów.
- `None` potrzebuje oczekiwanego typu. `?:` i `!!` wymagają `?`.
- `length` jest jedynym polem `String` i tablicy.
- Nullable i floaty są w typecku. Codegen nullable odcina bramką, a floaty albo diagnostyką „not supported yet”, albo paniką.

Rozdział 5. Funkcje, wywołania i domknięcia

Ten rozdział obejmuje

- Deklarację fun, parametry val/var, typ wyniku
- Wywołania, trailing closure i trzy formy move przy domknięciu
- Wbudowane print, println, concat — i zakaz ich przesłaniania
- Co codegen naprawdę umie wywołać
- Panicę przy przekazaniu String do funkcji użytkownika

Kształt funkcji

```
fun nazwa(parametry): TypWyniku { ciało }
```

Pominięty typ wyniku to unit, nie nazwa Unit. Parametr to nazwa: Typ, val nazwa: Typ albo var nazwa: Typ. Goła forma jest val.

Listing 5.1. Oba rodzaje parametrów. Uruchomione checker; HIR ma Val i Var, wynik i32.

```
fun add(val a: Int, var b: Int): Int {  
    return a  
}
```

var na parametrze znaczy: wołający, który przekazuje wiązanie nie-Copy, musi je oddać (move). Dla i32 różnica nie boli, bo i32 jest Copy. Dla String boli i jest sprawdzana.

Ciało jest blokiem. Ostatnie wyrażenie **nie** jest niejawnym wynikiem funkcji. Wynik idzie przez return. Typeck wymaga, żeby każda ścieżka funkcji o wyniku innym niż unit wykonała return.

Listing 5.2. Uruchomione.

```
missing_ret.bork: error: type: function must return a value of type i32 on  
↳ all paths
```

dla funkcji, która ma tylko val n = 1.

return bez wartości ma typ unit. W funkcji i32 dostaniesz empty return has type unit, expected i32. return z wartością sprawdza zgodność z zadeklarowanym wynikiem.

Nie ma rekurencji wzajemnej opisanej osobno: funkcje są w jednej mapie `fun_sigs` po nazwie, więcwołanie wstecz działa o tyle, o ile typeck widzi sygnaturę zebraną z całego programu przed ciałami. Nie ma przeciążeń. Dwie funkcje o tej samej nazwie nie są modelem, który książka może obiecać; mapa jest `HashMap` po gołej nazwie i komentarz w `sema/env.rs` mówi wprost: „MVP: keyed by bare function name (no local shadowing of callees yet)”.

Własność na granicy wywołania

Macierz, którą `sema` egzekwuje dla typu `nie-Copy`:

Źródło	Parametr <code>val</code>	Parametr <code>var</code>
<code>val nazwa</code>	użycie w miejscu (<code>Shared</code> , jeśli region na to pozwala)	trzeba <code>move</code>
<code>var nazwa</code>	trzeba <code>move</code>	trzeba <code>move</code>
literal, <code>concat(...)</code> , inne świeże wyrażenie	bez <code>move</code>	bez <code>move</code>

Listing 5.3. Świeży literal do `var`. Checker: tak. `bork build`: panic kompilatora w `value_as_int`.

```
fun f(var a: String) {
    println(a)
}

fun main() {
    f("hello")
}
```

To jest zgodne z regułą frontendu (świeże wyrażenie nie wymaga `move`) i niezgodne z tym, co backend umie obniżyć. Panic, sprawdzony:

```
thread 'main' panicked at src/codegen/llvm/expr.rs:901:14:
Found StructValue(...) but expected the IntValue variant
```

Ta sama panica jest na `f(move s)` i na `show(s)` gdy `s` jest `val String`. `println(s)` i `concat` działają, bo są intrinsicami, nie wywołaniami `bork.f`. Dopóki ta dziura istnieje, funkcja użytkownika w programie `bork build` powinna przyjmować i zwracać prymitywy. Napisy zostaw w `main` i przekazuj je do `print` / `println` / `concat`.

Zwracanie napisu, który już żyje na głębokości funkcji, checker i codegen akceptują.

Listing 5.4. Uruchomione: `stdout hi\n`.

```
fun shout(): String {
    var s = "hi"
    return s
}

fun main() {
```



```
println(shout())
}
```

return "hi" też działa (stdout hi\n). return name dla parametru przechodzi checker (fun greet(name: String): String { return name }). Wywołanie println(greet("Ada")) znowu panikuje na argumentcie String do greet, nie na return.

return concat("a", "b") checker odrzuca bez numeru linii, bo diagnostyka escape nie ma spanu:

```
err_ret_concat.bork: error: ownership: returning the result of `concat` is
↳ not supported yet: returned `String` bytes must outlive the callee arena
```

return move s jest w tej samej rodzinie: returning a moved or promoted String is not supported yet: use return s for parameters and locals, or return a string literal.

Wbudowane

Trzy nazwy są intrinsicami (src/builtins.rs). Ponowna deklaracja jest błędem typu, zanim ciało w ogóle ma znaczenie.

Listing 5.5. Uruchomione.

```
err_builtin.bork:1:13: error: type: cannot redefine builtin function `println`
```

Nazwa	Efekt	Sygnatura nominalna
print	pisze na stdout i robi flush	w tabeli (i32) -> unit, ale przyjmuje też i64 i String
println	to samo plus \n	j.w.
concat	jeden nowy String	(String, String) -> String

Listing 5.6. Kolejność i flush. Uruchomione, stdout dokładnie 1\ntail7 (bez końcowej nowej linii).

```
fun main() {
    println(1)
    print("tail")
    print(7)
}
```

print i println biorą jeden argument. concat bierze dwa. Wynik concat ląduje w arenie aktualnego sinku, a gdy sinku nie ma — w arenie bieżącego bloku. Rozdział 9 rozbiera to na reguły.

Trailing closure

Ostatni argument może być blokiem po liście nawiasów. Parametry domknięcia stoją przed `->`.

Listing 5.7. Tylko check. Ostatni parametr action ma typ `(Int, Int) -> Int`.

```
fun action(a: Int, b: Int, block: (Int, Int) -> Int): Int {
    return block(a, b)
}

fun main(): i32 {
    val threshold = 5
    return action(1, 2) { acc, current ->
        if (current > threshold) { acc + current } else { acc }
    }
}
```

Błędy typeck, które tu pilnują kontraktu:

- function ... requires a function type as its last parameter when called with a trailing closure,
- trailing closure expects N parameters, got M,
- trailing closure body has type ..., expected ...,
- only named functions can be called — nie wywołasz wartości typu funkcji, nawet jeśli typ jest w HIR; callee musi być nazwą funkcji albo, w typecku, wiązaniem typu funkcyjnego, ale codegen i tak odrzuca wywołanie pośrednie tekstem `indirect calls`.

Sema dla parametrów domknięcia wpisuje `Ty::Unknown`. Nie dostaje typów z sygnatury. Skutek: wewnątrz domknięcia polityka Copy/Move dla tych parametrów jest zachowawcza. Parametry z listingu 5.7 są w dumpie `Local`, a odczyt `acc` w gałęzi `if` jest `Shared` ← `Closure`, nie `Copy`, mimo że `Int` jest `Copy`. To rozjazd semy z typeckiem, widoczny w drzewie `aren`, a nie błąd użytkownika. Program check przechodzi.

Domknięcie nie jest wartością pierwszej kategorii, którą można zapisać w `val f = { ... }` poza wywołaniem. Gramatyka ma closure tylko jako trailing albo jako blok `move`. Typ funkcyjny istnieje, ale nie ma literału funkcji poza tym miejscem.

Trzy formy domknięcia z move

Składnia	Co przenosi
<code>f(...) { ... }</code>	nic; odczyt zewnętrznego <code>var String</code> jest błędem
<code>f(...) move () { ... }</code>	jawnie nic
<code>f(...) move (a, b) { ... }</code>	dokładnie wymienione nazwy
<code>f(...) move { ... }</code>	wnioskuję: każde żywe, nie-Copy imię z rodzica, którego ciało używa

To samo da się napisać jako instrukcję, bez wywołania: `move (a) { ... }, move () { ... }, move { ... }`. Rozdział 8 ma uruchomione binarki dla form instrukcji. Formy trailing są w języku frontendu i odpadają na bramce:

```
trail_move.bork:6:12: error: codegen: trailing closures are not supported by  
  ↪ codegen
```

Wnioskowanie nie patrzy na `move` nazwa i `promote` nazwa jako na swobodne zmienne (`free_vars` je pomija). Inaczej wyrażeniowy `move` byłby liczony drugi raz jako przechwycenie regionu.

Wywołanie a region

Ciało wołanej funkcji ma własne regiony i własne `bork_arena_push` na czas aktywacji. Argumenty są liczone w regionie wołającego. Wynik, jeśli jest prymitywem, wraca w rejestrze. Wynik `String` nie dostaje dziś bufora własności wołającego, dlatego `escape` zabrania `return` świeżych bajtów. Listing 5.4 działa, bo `"hi"` jest literałem w stałych albo lokalną wartością na głębokości 0, a `println` tylko czyta deskryptor.

TIP

TIP. W programie, który ma przejść `bork build`, trzymaj funkcje przy liczbach. Napisy drukuj z `main`. To nie jest styl na zawsze. To jest obwód dziury w `coerce_value_to_ty`, która każdą wartość nie-bool i nie-float przepycha przez `into_int_value`.

Podsumowanie

- Wynik funkcji jest jawny (`return`). Brak ścieżki to błąd typu.
- Goły parametr jest `val`. `var` wymaga `move` przy przekazaniu wiązania nie-Copy.
- Świeży literał nie wymaga `move`, ale `String` jako argument funkcji użytkownika wywraca `codegen`.
- `print`, `println` i `concat` są wbudowane i nie wolno ich zadeklarować.
- Trailing closure jest w typecku i semie. Bramka `codegen` ją odcina.
- Sema nie typuje parametrów domknięcia; w dumpie potrafią wyglądać jak nie-Copy.

Rozdział 6. Kontrola przepływu

Ten rozdział obejmuje

- if jako wyrażenie i jako sterowanie
- for po zakresie lo..hi
- while, break, continue
- Zwieranie && i ||
- Jak sema scala flagi moved po gałęziach

if

Warunek ma mieć typ bool. Obie gałęzie, jeśli else jest, są blokami. Typ wyniku zależy od tego, czy typeck **oczekuje** typu, i od tego, czy else istnieje. To jest w typeck/expr/control.rs, funkcja check_if.

else	Oczekiwany typ	Typ wyniku
brak	brak	unit
brak	jest (np. adnotacja val x: i32)	błąd: value-producing if expression requires an else branch
jest, typy gałęzi równe	dowolny	ten typ
jest, typy różne, oczekiwany typ jest		błąd: else branch has type ..., expected ...
jest, ale wywołanie nie oczekuje typu		unit

Ostatni wiersz zaskakuje. if użyty tam, gdzie nikt nie prosi o wartość, dostaje typ unit nawet z else. Wartość gałęzi jest sprawdzana, ale nie wpływa. Dlatego nie ma „niejawnego wyniku bloku” w stylu Rusta poza ostatnim wyrażeniem gałęzi, i tylko gdy ktoś tego typu oczekuje.

Listing 6.1. Wyrażenie if. Uruchomione: kod 42.

```
fun pick(flag: i32): i32 {  
    val v = if (flag == 1) { 40 } else { 7 }  
    return v + 2  
}
```

```
fun main(): i32 {
    return pick(1)
}
```

Tu `val v` nie ma adnotacji, ale prawa strona jest w kontekście, w którym obie gałęzie mają ten sam typ `i32`, więc wynik `if` to `i32`. Dump ma osobno `IfThen` i `IfElse`.

Listing 6.2. Brak `else` przy oczekiwaniu `i32`. Uruchomione.

```
fun main(): i32 {
    val x: i32 = if (1 > 0) { 1 }
    return x
}
```

```
if_ann.bork: error: type: value-producing if expression requires an else
↳ branch
```

Bez adnotacji, `val x = if (n > 0) { n }`, błąd jest inny: `if` staje się `unit` (nikt nie oczekiwał typu), a dopiero `return x` mówi `return value has type unit, expected i32`. Komunikat o brakującym `else` pojawia się wtedy, gdy oczekiwany typ jest `Some`.

Gałąź `if` jest regionem. Napis zbudowany tylko w jednej gałęzi nie może być wartością tego `if`, bo arena gałęzi ginie przy jej końcu. `Escape` mówi:

```
a `String` moved inside an `if` branch cannot be its value: the branch's arena
↳ is freed when the branch ends
```

Tekst mówi `String`, choć warunek w kodzie patrzy na `uses_arena_storage` (także tablica). To drobny rozjazd komunikatu z warunkiem.

`Sema scala` stan `moved` po `if`: nazwa jest przeniesiona po całym `if` wtedy, gdy była przeniesiona przed nim, albo gdy przeniosły ją **obie** gałęzie. Samo `then` bez `else` nie zostawia nazwy martwej po `if`. Test `if_then_only_move_without_else_does_not_stick` to zamyka. Inaczej gałąź, która może nie pobiec, zabijałaby zmienną na stałe.

for

`for (nazwa in lo..hi) { ciało }`. Zakres jest prawostronnie otwarty. Granice są typu `i32` i są liczone raz. Nazwa pętli jest `i32` w typecku. W semie wiązanie indeksu jest zakładane jako `Int`, czyli też `i32`, ale wpisane przez `Type::from_ident("Int")`, nie przez typ elementu zakresu z HIR. Przy poprawnym programie to ten sam typ.

Listing 6.3. Suma `0+1+2+3+4`. Uruchomione: kod 10.

```
fun main(): i32 {
    var total = 0
    for (i in 0..5) {
        total = total + i
    }
    return total
}
```

Dump:

```

└─ fun main
    └─ total [Local]
        └─ ForLoop (i)
            └─ i [Local]
                └─ total [Copy]

```

total w ciele jest Copy, bo i32. Przypisanie `total = ...` jest mutacją zewnętrznego var, nie przeniesieniem. Codegen emituje jeden `region_enter` dla ciała, a na zatrzasku pętli `bork_arena_reset`, nie `pop` i ponowny `push`. Płyta zostaje. To jest kontrakt opisany w `TOD0.md`: jeden `enter`, wiele `resetów`, jeden `exit`.

Zakres `i64..i64` odpada:

```

range_bad.bork:4:15: error: type: range bounds must have type i32, got i64
↪ and i64

```

Iterator, który nie jest zakresem (`for (i in n)`), daje `for-loop iterator must be a range`. Operator `..` poza `for` tworzy wartość typu zakresu; typeck i tak wymaga `i32`. Nie ma `for` po tablicy.

while, break, continue

Listing 6.4. `break`. Uruchomione: kod 3.

```

fun main(): i32 {
    var i: i32 = 0
    while (i < 10) {
        if (i == 3) {
            break
        }
        i = i + 1
    }
    return i
}

```

Warunek `while` musi być `bool` (`while condition has type ..., expected bool`). `break` i `continue` poza pętlą:

```

err_break.bork:2:5: error: type: `break` outside of a loop

```

To błąd fazy `type`, nie osobnej fazy sterowania. Typeck trzyma `loop_depth` i podbija go na `for` oraz `while`.

Listing 6.5. `continue` pomija `i == 2`. Suma `0+1+3+4 = 8`. Uruchomione: kod 8.

```

fun main(): i32 {
    var s = 0
    for (i in 0..5) {
        if (i == 2) {
            continue
        }
        s = s + i
    }
}

```

```
    return s
}
```

continue w while też jest. Sema oznacza odczyt i oraz s w ciele jako Copy. Nie ma for z klauzulą else. Nie ma pętli nieskończonej innej niż while (true).

Zwieranie

Listing 6.6. Prawa strona nie powinna podbić n. Uruchomione: kod 0.

```
fun side(): i32 {
    return 1
}

fun main(): i32 {
    var n: i32 = 0
    if (false && side() == 1) {
        n = 1
    }
    if (true || side() == 1) {
        return n
    }
    return 99
}
```

side nie ma efektu poza wynikiem, więc listing nie udowadnia zwierania obserwowalnym printem. Udawadnia je zgodność z testem builds_logical_short_circuit oraz kształt LLVM: && i || dostają osobne bloki, a region_walk nie schodzi w prawy operand, gdy flaga short_circuit_logical_operands jest włączona. Dzięki temu stempel codegen_push nie rozjeżdża się z emisją. To była osobna poprawka (fix: walk logical RHS for stamp/schedule only).

! jest jednoargumentowe i wymaga bool. Program val flag = !false oraz if (flag) { return 1 } else { return 0 } daje kod 1.

Wczesny return a regiony

return w zagnieżdżonym bloku musi zdjąć areny, które funkcja otworzyła. Codegen robi unwind uchwytów i nie wykonuje region_exit drugi raz po ścieżce, która już wróciła (seria poprawek „skip region exit after return”, „sync region stack”, „clear arena handles”). Test builds_nested_loops_with_regions_and_early_return zwraca 63: zagnieżdżone for liczą pary $j > i$, potem pętla do 100 zwraca $\text{total} * 10 + k$ przy $k == 3$.

To jest szczegół backendu, ale ma skutek językowy: return w środku bloku jest legalny dla wartości Copy i dla napisu na głębokości 0. Nie jest legalny dla napisu, którego bajty żyją głębiej.

Listing 6.7. Uruchomione checker, kod 1. bork build też kończy się 1 i nie zostawia binarki.

```

fun mk(): String {
    var s = "esc"
    {
        val x = move s
        return x
    }
}

fun main() {
    println(mk())
}

```

err_ret_inner.bork:5:16: error: ownership: returning a `String` whose bytes
 ↪ live in an inner region is not supported: that arena is freed before the
 ↪ value is returned

Faza to ownership, bo escape dokłada diagnostykę do tej samej fazy co sema. HIR przy błędzie escape jest wyrzucany: `CheckResult.hir` jest `None`, raport aren zostaje.

Podsumowanie

- if bez else w kontekście bez oczekiwanego typu ma typ unit.
- Z adnotacją brak else jest osobnym błędem.
- Move w jednej gałęzi nie zabija nazwy po if, jeśli druga gałąź jej nie ruszyła.
- for idzie po `i32..i32`, zakres prawostronnie otwarty, arena ciała resetowana co obrót.
- break i continue są błędami typu poza pętlą.
- `&& i ||` zwierają. Backend i spacer regionów są co do tego zgodne.
- return napisu z wewnętrznego bloku jest odrzucany, zanim powstanie binarka.

Rozdział 7. Tablice

Ten rozdział obejmuje

- Typ `[T; N]` i literał `[1, 2, 3]`
- Indeks, przypisanie elementu, wycinek o stałych granicach
- `.length`
- Własność całej tablicy i współdzielenie elementu `String`
- Co robi runtime, gdy indeks nie mieści się w `N`

Długość jest typem

`N` w `[T; N]` jest literałem całkowitym w typie, nie wyrażeniem. Za duże `N` dla `u32` daje błąd gramatyki `array length is too large`. Pusta tablica nie ma skąd wziąć `N` ani `T`:

```
err_empty.bork:2:13: error: type: empty array literal requires an explicit
↳ type, e.g. `val a: [i32; 0] = []`
```

Z adnotacją działa. **Listing 7.1.** Uruchomione: kod 0, bo `length` zera zwrócone jako kod wyjścia.

```
fun main(): i32 {
    val a: [i32; 0] = []
    return a.length
}
```

Elementy literału muszą mieć jeden typ. Liczba elementów musi równać się `N`.

Listing 7.2. Uruchomione checker.

```
array_mismatch.bork:2:23: error: type: array literal has 3 elements, expected
↳ 2
```

```
array_mismatch.bork:2:9: error: type: initializer for `a` has type [i32; 3],
↳ expected [i32; 2]
```

```
dla val a: [i32; 2] = [1, 2, 3].
```

Dozwolony element to typ `Copy` albo nienullowalny `String`. `String?` jako element jest odrzucany. Nullowalna tablica jako całość też.

Tablica nie jest `Copy`, nawet gdy elementy są. Przypisanie całych tablic wymaga identycznego `T` i `N`. Nie ma `[i32; 3]` w miejsce `[i32; 2]`.

Indeks

Indeks jest `i32` i jest liczony w runtime. Nie jest częścią typu, więc typeck **nie** odrzuca `a[9]` na tablicy trzelementowej. Sprawdzono: checker mówi tak, bork build produkuje binarkę, uruchomienie kończy się SIGABRT (134). `guard_index` woła `abort`.

Listing 7.3. Uruchomione: kod 12. Po zapisie `a[1] = 9` wycinek widzi ten sam bufor, więc `b[1]` to 9, a `a.length` to 3.

```
fun main(): i32 {  
    var a: [i32; 3] = [1, 2, 3]  
    a[1] = 9  
    val b = a[0..2]  
    return b[1] + a.length  
}
```

`val` nie przyjmuje przypisania elementu:

```
val_index.bork:3:5: error: type: cannot assign to immutable `val` binding `a`
```

Indeks złego typu: array index must be `i32` albo, przy przypisaniu, array index has type ..., expected `i32`.

Wycinek

`a[lo..hi]` wymaga, żeby `lo` i `hi` były literałami `i32`. Typ wyniku to `[T; hi-lo]`. Wycinek nie kopiuje bufora. Deskryptor wskazuje w środek tablicy, a długość bierze z typu.

Poza zakresem:

```
err_slice.bork:3:13: error: type: slice [1..9] is out of bounds for `[i32; 3]`
```

Granice, które nie są literałami: slice bounds must be integer literals so the result type is `[T; N]`. Codegen **nie** wstawia drugiego sprawdzenia wycinka w runtime. Ufa typeckowi. Indeks w runtime sprawdzany jest, bo nie da się go włożyć do typu.

Listing 7.4. Uruchomione: `stdout 20\n20\n`.

```
fun main() {  
    val a = [10, 20, 30]  
    val b = a[0..2]  
    println(b[1])  
    println(b.length)  
}
```

Escape traktuje wycinek jak odbiorcę: nie może przeżyć areny, która trzyma bufor. Test `rejects_slice_escaping_inner_region` oczekuje fazy ownership i tekstu `inner region`.

Napis jako element

Odczyt `a[i]`, gdy element jest `String`, jest widokiem `Shared` na bufor tablicy, niezależnie od `val` czy `var` tablicy. Nie ma `move` jednego elementu. `move` i `promote` dotyczą całej tablicy.

Zapis elementu nie-Copy używa areny tablicy jako sinku przypisania i wymaga move albo promote po prawej.

Listing 7.5. Uruchomione: stdout z\n.

```
fun main() {  
    var a: [String; 2] = ["a", "b"]  
    var s = "z"  
    a[0] = move s  
    println(a[0])  
}
```

Bez move checker mówi, że trzeba move (test index_assign_string_requires_move). Po move nazwa s jest martwa. Dump listingu 7.5: s [Moved ← fun main].

Reprezentacja

W runtime tablica i napis mają ten sam kształt deskryptora LLVM: { ptr, i64 }. Dla tablicy len równa się N. Bajty elementów leżą w arenie domu tej wartości: arena aktywna w miejscu utworzenia albo arena sinku, gdy tablica jest od razu zapisywana do zewnętrznego var.

length w codegenie jest wycięciem pola i64 i obcięciem do i32. Dlatego typ pola w języku to i32, mimo że w deskrytorze długość jest 64-bitowa. Przy N, które mieści się w gramatyce jako u32, obcięcie do i32 jest prawdziwe tylko dopóki N mieści się w i32. Gramatyka przyjmuje u32. To jest krawędź, której testy nie spinają dużym N. Nie udawaj, że length tablicy o $N > 2^{31}-1$ jest dobrze określone.

Podsumowanie

- [T; N] niesie długość w typie. Literał musi mieć dokładnie N elementów.
- [] wymaga adnotacji.
- Indeks spoza zakresu aboruje proces. Zły wycinek jest błędem typu.
- Wycinek jest widokiem, nie kopią, i ma typ [T; hi-lo].
- Element String czyta się jako Shared. Zapis elementu wymaga move.
- Całą tablicę wolno move / promote tylko na identyczny [T; N].

Rozdział 8. Regiony, Copy, Shared i Move

Ten rozdział obejmuje

- Drzewo aren i znaczniki w `--dump-arenas`
- Wyrażeniowy `move` i `promote`
- Regionalny `move (...)`, `move ()` i wnioskowany `move { }`
- `Assign-up`: przypisanie do zewnętrznego `var`
- Co jest błędem własności, a co dopiero błędem `escape`

Dwa sprawdzania, jedna faza

Własność nazw sprawdza `sema (src/sema)`. To, czy **bajty** przeżyją użycie, sprawdza `escape::check_function`, ale tylko gdy `typeck` i `sema` nie zgłosiły nic wcześniej. Oba źródła drukują się jako `ownership`. Dlatego „przeszło semę” nie znaczy „przeszło książkowe reguły napisów”. Znaczą: nazwy nie są użyte po `move` i `nie-Copy` nie wycieka przez gołe imię. `Escape` jest drugim sitem.

Raport semy zostaje nawet przy błędach. HIR zostaje tylko przy pustej liście diagnostyk, i to po `escape`. Na błędzie parsowania nie ma nawet raportu.

Znaczniki dumpu

Znacznik	Znaczenie
[Local]	deklaracja w tym regionie albo parametr
[Copy]	odczyt prymitywu z regionu rodzica
[Shared ← etykieta]	odczyt <code>val</code> <code>nie-Copy</code> z rodzica
[Moved ← etykieta]	własność zabrana z tamtego regionu

Etykiety węzłów: `fun nazwa`, `Block`, `Block (compacted K braces)`, `ForLoop (i)`, `WhileLoop`, `IfThen`, `IfElse`, `MoveBlock (move)`, `Closure`, `Closure (move)`.

Wyrażeniowy move

move nazwa konsumuje wiązanie. Po nim użycie nazwy jest błędem use of nazwa after move from etykieta.

Gołe przekazanie var nie-Copy w przypisaniu albo w argumencie nie zgaduje move za ciebie.

Listing 8.1. Uruchomione checker.

```
err_move.bork:3:13: error: ownership: use `move s` to transfer ownership
```

dla `var s: String = "hi"` oraz `var x = s`.

W wywołaniu tekst jest inny: `use move s to pass ownership`. Gdy sema widzi przeniesienie między różnymi arenami w przypisaniu, potrafi powiedzieć ``s` is not Copy; move it into `Block` with `move``.

Listing 8.2. Move i druk. Uruchomione: `stdout ab\n42\n`.

```
fun main() {  
    var s = "ab"  
    val t = move s  
    println(t)  
    print(4)  
    println(2)  
}
```

Dump: `s [Moved ← fun main], t [Local]`.

promote

promote nazwa jest legalne tylko po prawej stronie przypisania do var w **ściśle zewnętrznym** regionie. Kopiuje bajty do areny celu i unieważnia źródło.

Listing 8.3. Uruchomione: `stdout temp\n`.

```
fun main() {  
    var outer = "a"  
    {  
        var held = "temp"  
        outer = promote held  
    }  
    println(outer)  
}
```

Dump pokazuje held dwa razy w bloku: jako Local (deklaracja) i jako Moved ← Block (obserwacja po promote).

Błędy, sprawdzone albo obecne w sema/walk.rs:

Sytuacja	Komunikat
promote poza assign-up	`promote` is only valid when assigning to a binding in an outer region
źródło Copy	`promote` is not needed for Copy binding ...
cel nie jest głębiej niż źródło	`promote held` cannot lift into `fun main`: value already lives in that arena or further out

Ostatni dostaniesz, gdy held i cel są w tym samym regionie (`val x = promote held` na głębokości `main`). Sprawdzono.

`promote` na `return` nie istnieje. Nie ma takiej produkcji w roli wyniku, która miałaby `sink` powrotny.

Assign-up

`outer = rhs`, gdy `outer` jest `var` w przodku i nie jest `moved`, **nie** jest „użyciem `var` rodzica w dziecku”. Sema pomija `note_use` na lewej stronie. Prawa strona dostaje `sink` transferu równy arenie `outer`, nie arenie bloku.

Listing 8.4. Uruchomione: `stdout b\n`. W bloku nie ma lokalnych nazw.

```
fun main() {
    var outer = "a"
    {
        outer = "b"
    }
    println(outer)
}
```

To jest mutacja. Nie potrzebujesz `inner`. `docs/language.md` mówi to samo i tutaj kompilator jest zgodny z dokumentem.

Regionalny move

Listing 8.5. Uruchomione: `stdout A\nB\n`.

```
fun main() {
    var a: String = "A"
    var b: String = "B"
    move (a) {
        val x = a
        println(x)
    }
    move {
        val y = b
        println(y)
    }
}
```

```
    }
}
```

Dump:

```
|— a [Local]
|— b [Local]
|— MoveBlock (move)
|   |— a [Moved ← fun main]
|   |— x [Local]
|— MoveBlock (move)
|   |— b [Moved ← fun main]
|   |— y [Local]
```

Nazwy z listy przechwycenia są już lokalne w bloku. Nie pisze się `val x = move` a dla nich drugi raz. Sema odrzuca ponowny `move` przechwycenia w tym samym regionie: `cannot move nazwa: it was already moved into this region as a capture`.

`move () { }` nie wnioskuje. Nic nie jest przeniesione. Odczyt zewnętrznego `var String` wewnątrz jest wtedy zwykłym błędem „not Copy”.

`move { }` bez nawiasów przenosi każdą żywą, nie-Copy nazwę rodzica, której ciało używa jako zmiennej wolnej. Nazwy nieczytane zostają. Copy się nie przenosi. W listingu 8.5 drugi blok wspomina tylko `b`, więc `a` już i tak było martwe po pierwszym bloku, a `b` pada ofiarą wnioskowania.

Jawna lista wygrywa z wnioskowaniem. Pusta jawna lista wyłącza wnioskowanie. To jest różnica między `None` a `Some([])` w AST (captures).

Pętla

Zakaz z rozdziału 2 dotyczy też regionalnego `move`. Lokal utworzony w ciele wolno przenieść:

```
for (i in 1..10) {
    var a: String = "A"
    move (a) {
        val t = a
    }
}
```

Checker takich programów pilnuje test `move_loop_local_binding_is_ok`. Każda iteracja ma świeże `a`. Zewnętrzne `a` byłoby błędem „inside a loop”.

Shared kontra błąd

Odczyt `val` nie-Copy w dziecku jest `Shared` i jest poprawny (listing 2.2). Odczyt `var` nie-Copy w dziecku bez `move` jest błędem. Nie ma trzeciej możliwości „pożycz na chwilę `var`”. Albo przenosisz, albo trzymasz `val`.

`Some(s)` i argumenty zagnieżdżone podlegają tej samej regule: nazwany `var` nie-Copy w środku konstruktora też chce `move`. Literał nie chce.

Podsumowanie

- Sema pilnuje nazw. Escape pilnuje bajtów. Obie diagnostyki nazywają się ownership.
- move nazwa zużywa wiązanie. Gołe var nie-Copy w przypisaniu i w wywołaniu jest błędem z podpowiedzią move.
- promote podnosi bajty do zewnętrznego var i też zużywa źródło.
- Assign-up nie jest odczytem lewej strony.
- move (a, b), move () i move { } to trzy różne listy przechwycenia.
- Wnioskowanie nie rusza wartości Copy i nie rusza nazw, których ciało nie wspomina.

Rozdział 9. Gdzie żyją bajty String

Ten rozdział obejmuje

- Deskryptor kontra bajty
- Pięć reguł z docs/language.md, skonfrontowanych z kompilatorem
- Hoist sąsiednich dwóch linii
- concat i to, czego przykładowy rozdział dokumentacji nie kompiluje
- Czego jeszcze nie ma: arena wyniku, promote na return

Deskryptor

Wartość String w LLVM to struktura { ptr, i64 }: adres bajtów i długość. Bajty są albo w globalu (literał), albo w płycie areny. Koniec regionu unieważnia płytę, nie deskryptor na stosie. Dlatego checker musi odrzucić deskryptor, którego ptr wskazywałby w martwą płytę, zanim codegen go wyemituje. To robi `escape::place`.

Głębokość 0 to region funkcji. Każdy zagnieżdżony region, który naprawdę otwiera scope w `escape`, zwiększa głębokość. `place` zwraca głębokość puli, w której leżą bajty wyrażenia.

Reguła 1. Alokujesz tam, gdzie stoisz

Bez sinku literał i concat biorą bieżącą pulę. Wyjście z bloku czyści pulę. Nazwa zadeklarowana w bloku i tak nie jest widoczna na zewnątrz. Problem zaczyna się, gdy deskryptor wynosisz przypisaniem albo return.

Reguła 2. Sink przypisania

Gdy prawa strona jest zapisywana do var, bajty **wyniku** (nie każdego podwyrażenia) idą do areny tego var. Argumenty wywołania są liczone przy wyczyszczonym `alloc_sink`, więc tymczasowe zostają w bloku.

Listing 9.1. Uruchomione: `stdout b\n`. Literał nie potrzebuje zmiennej pośredniej.

```
fun main() {  
    var outer = "a"  
    {  
        outer = "b"    }  
}
```

```

    }
    println(outer)
}

```

`outer = concat(left, right)` kładzie **wynik** w puli `outer`. Napisy `left` i `right` są czytane tam, gdzie już żyją.

Reguła 3. Hoist dwóch sąsiednich linii

Gdy w bloku jest

```

var piece = <literał albo wywołanie>
outer = move piece

```

i `outer` jest już w zasięgu, `hoist::annotate` ustawia na deklaracji `alloc_in_binding` = `Some(outer)`. Codegen buduje `piece` od razu w puli `outer`. Druga linia nie kopiuje znaków drugi raz.

Wzór jest celowo wąski (`src/hoist.rs`):

- tylko następna instrukcja, nic pomiędzy,
- inicjalizator to `Str` albo `Call`, nie tablica i nie inne wyrażenie,
- prawa strona przypisania to `move` tej samej nazwy,
- cel to nazwa, nie `a[i]`,
- `outer` jest parametrem albo wcześniejszym `var` w tym bloku lub w `outer` przekazanym przy zejściu.

Pętla, `if`, domknięcie i luka między liniami wyłączają `hoist`. Zostaje kopia przy `move` albo jawny `promote`.

NOTE

NOTE. Hoist nie dowodzi, że „`inner` istnieje tylko po to, żeby nakarmić `outer`”. Patrzy na dwie sąsiednie instrukcje. Rozdzieli je, a płacisz kopię.

Reguła 4. promote, gdy bajty już leżą źle

Listing 8.3. `held` powstał w wewnętrznej puli. `promote` kopiuje do puli `outer` i oznacza źródło jako `moved`. Nie omija kopii. Omija ją `hoist` albo przypisanie świeżego wyniku wprost do `outer` (reguła 2).

Reguła 5. Escape odrzuca resztę

Komunikaty z `escape.rs`, sprawdzone tam, gdzie dało się je wywołać:

Komunikat	Kiedy
returning a `String` whose bytes live in an inner region...	return wartości o głębokości > 0

Komunikat	Kiedy
returning the result of concat is not supported yet...	return concat(...)
returning a moved or promoted String is not supported yet...	return move / return promote
assigning a value whose bytes live in an inner region to nazwa...	przypisanie w górę, którego sink nie uratował
a String moved inside an if branch cannot be its value...	wartość if z bajtami gałęzi

return if (c > 0) { concat("a", "b") } else { "x" } wpada w komunikat o concat, nie w komunikat o gałęzi. Sprawdzono na err_if_string.bork. Span znowu bywa pusty.

concat i błąd w podręczniku

docs/language.md pokazuje:

```
var left = "L"
var right = "R"
{
    var piece = concat(left, right)
    out = move piece
}
```

Ten program **nie** przechodzi checku. left i right są var. W bloku trzeba je przenieść. Sprawdzono:

```
concat.bork:6:28: error: ownership: `left` is not Copy; move it into `Block`
  ↳ with `move`
concat.bork:6:34: error: ownership: `right` is not Copy; move it into `Block`
  ↳ with `move`
```

Dwie wersje, które przechodzą i drukują LR\n:

Listing 9.2. val jest Shared. Uruchomione.

```
fun main() {
    var out = "prefix"
    val left = "L"
    val right = "R"
    {
        var piece = concat(left, right)
        out = move piece
    }
    println(out)
}
```

Listing 9.3. var z jawnym move. Uruchomione. Po bloku left i right są martwe.

```

fun main() {
    var out = "prefix"
    var left = "L"
    var right = "R"
    {
        var piece = concat(move left, move right)
        out = move piece
    }
    println(out)
}

```

W obu wypadkach stdout to LR, nie prefixLR. concat nie dokleja do istniejącego bufora. Buduje nowy napis i przypisanie **zastępuje** deskryptor out. Stary "prefix" zostaje śmieciem w puli aż do końca regionu funkcji.

Dump listingu 9.2: w bloku piece [Moved ← Block], left [Shared ← fun main], right [Shared ← fun main]. Hoist może ustawić alloc_in_binding na piece, bo następna linia to out = move piece, a inicjalizator jest wywołaniem. Efekt widoczny dla programisty jest ten sam: out po bloku da się wydrukować.

Co jeszcze nie działa

Z docs/language.md i z kodu, bez dorabiania:

- hoist tylko dla wzorca dwóch linii, nie dla ogólnego „ta zmienna ucieka”,
- brak bufora zwracanego, który należałby do wołającego,
- promote nie występuje na return,
- return świeżego concat nawet w jednej gałęzi if jest odrzucany.

Dodatkowo codegen: nawet poprawny checkerowo przekaz String do funkcji użytkownika panikuje. concat i println są wyjątkami, bo mają własne emitory (emit_concat, emit_print), nie coerce_value_to_ty.

Podsumowanie

- String to deskryptor. Bajty leżą w globalu albo w płycie 4 KiB.
- Przypisanie do var alokuje wynik w puli tego var.
- Dwie sąsiednie linie var piece = ... / outer = move piece mogą zbudować bajty od razu u celu.
- promote kopiuje, gdy bajty już powstały za głęboko.
- Przykład concat(left, right) na dwóch var z dokumentacji języka nie kompiluje się. Trzeba val albo move.
- concat zastępuje deskryptor. Nie dokleja do starego bufora w miejscu.

Rozdział 10. Gdy kompilator borks: diagnostyka i aborcja

Ten rozdział obejmuje

- Cztery fazy i jeden format linii
- Co jest błędem kompilacji, a co śmiercią procesu
- Kolejność diagnostyk i to, że HIR znika
- Panikę kompilatora, która nie jest diagnostyką
- Czego język nie ma: wyjątków, Result, try

Nie ma obsługi błędów w języku

Nie ma throw, try, catch, typu Result, typu Option poza powierzchnią T?, ani makra panic. Program Bork albo przechodzi kompilację, albo nie. W runtime są dokładnie trzy awarie, które kod naprawdę wywołuje:

Zdarzenie	Mechanizm	Co zobaczysz
Dzielenie przez zero, albo (w backendzie) MIN / -1	abort z libc	proces SIGABRT, kod powłoki 134, pusty stderr
Indeks tablicy poza N	ten sam abort	j.w.
Alokacja ponad 4096 bajtów płyty	panic! w bork_runtime	komunikat Rust arena overflow: need ... bytes, capacity 4096

T? nie jest błędem. Jest wartością, którą frontend umie sprawdzić, a codegen umie odrzucić. Nie myl nullable z modelem błędów.

NOTE

NOTE. Składniowo nie zapiszesz literału ujemnego, więc ścieżki MIN / -1 nie odpalisz literałem -1. Strażnik w guard_int_div i tak jest: porównuje dzielnik z zerem i, dla typów ze znakiem, parę (minimum, minus jeden).

Format

src/main.rs, funkcja `print_diagnostics`:

ścieżka:linia:kolumna: error: faza: treść

albo, gdy span jest None:

ścieżka: error: faza: treść

Fazy to `parse`, `ownership`, `type`, `codegen`. `Severity` ma wariant `Warning`, ale ścieżki, które książka przeszła, produkują `Error`. CLI i tak drukuje słowo `error`.

Kolumna jest liczbą znaków (`chars`) od ostatniego `\n`, plus jeden. Nie jest indeksem bajtu. Span w kompilatorze jest bajtowy. Przy ASCII, czyli przy całym dzisiejszym lekserze, to jest to samo.

LSP dokłada prefiks fazy drugi raz w treści ("`parse: ...`") i przelicza bajty na pozycje UTF-16, których wymaga protokół. Rozdział 18.

Faza parse

Źródła komunikatów: `diag::from_parse`.

Wariant LALRPOP	Tekst
<code>InvalidToken</code>	<code>invalid token</code>
<code>UnrecognizedEof</code>	<code>unexpected end of file; expected ...</code>
<code>UnrecognizedToken</code>	<code>unexpected token `...`; expected ...</code>
<code>ExtraToken</code>	<code>unexpected extra token `...`</code>
<code>User</code>	tekst z gramatyki, span na końcu pliku

Listing 10.1. Uruchomione.

```
err_parse.bork:1:10: error: parse: unexpected end of file; expected
↪  r#[a-zA-Z_][a-zA-Z0-9_]*"#, ")", "val", "var"
```

dla `fun oops()`. Lista `expected` jest surowa, z regexami leksera. Przy błędzie parsowania `CheckResult.report` i `hir` są `None`. `--dump-arenas` nie drukuje drzewa.

Błąd użytkownika gramatyki, na przykład zły cel przypisania, nie wskazuje miejsca tokenu. Wskazuje EOF. To warto wiedzieć, zanim zaczniesz szukać błędu w ostatniej linii.

Faza type

Komunikaty z `src/typeck`. Rozdziały 4–7 cytowały te, które uruchomiliśmy. Pełniejszy katalog jest w dodatku A. Kilka, które warto znać z nazwy, bo psują całe funkcje, nie pojedyncze wyrażenia:

- `function must return a value of type ... on all paths`,
- `cannot redefine builtin function ...`,

- unknown binding ...,
- unknown function ...,
- function ... expects N arguments, got M.

Typeck nie zatrzymuje się na pierwszym błędzie. Zbiera wektor. Potem frontend i tak uruchamia semę. Dlatego jeden plik potrafi mieć i type, i ownership. Test `reports_ownership_and_type_together` używa `1 + "x"` razem z użyciem nazwy po `move`.

Faza ownership

Dwa autorzy: `diag::from_sema` oraz `escape::check_function`. Escape nie idzie, gdy wcześniejsze diagnostyki są niepuste. Skutek praktyczny: przy błędzie typu możesz nie zobaczyć błędu escape, który w czystym programie by się pojawił. Najpierw napraw typy i gołe `move`, potem czytaj `escape`.

Sema przy błędzie nadal oddaje `ArenaReport`. Dump przy `err_use_after` pokazuje `s [Moved ← fun main]` oraz lokalne `u`, mimo że program jest zły. Drzewo jest przybliżeniem tego, co spacer zdążył zbudować, nie certyfikatem poprawności.

Faza codegen

Pojawia się tylko na bork `build`. Checker jej nie produkuje. Bramka (gate) odrzuca konstrukcje, zanim powstanie moduł LLVM:

- trailing closures are not supported by codegen,
- ``None`` is not supported by codegen,
- ``Some`` is not supported by codegen,
- ``!!`` is not supported by codegen,
- ``?:`` is not supported by codegen,
- field access is not supported by codegen (wszystko poza `.length` na napisie lub tablicy),
- this binary operator is not supported by codegen.

Potem emisja dokłada między innymi:

- ``fun main`` is required to build an executable,
- ``main`` returning ``f64`` is not supported by codegen yet (analogicznie `i64`, `bool`),
- indirect calls is not supported by codegen yet,
- float binary after walk is not supported by codegen yet.

Ostatni tekst potrafi przyjść owinięty: `internal arena schedule mismatch: ...`. `resolve_walk_failure` dokleja ten prefiks, gdy błąd spaceru nie niesie osobnej diagnostyki albo gdy niesie ją w treści `WalkError`. To jest szorstkie. Treść po dwukropku jest tą, która mówi, czego brakuje.

Błąd codegen ma kod wyjścia 1 i **nie** zostawia binarki (testy `gate_rejection_exits_one_without_bin` i `brak main`).

Panic kompilatora

To nie jest diagnostyka. Proces bork umiera z kodem 101 i śladem Rusta. Sprawdzone przypadki:

Program	Miejsce
argument String do funkcji użytkownika porównanie f64 na ścieżce walk	expr.rs value_as_int / into_int_value ten sam into_int_value, wartość jest FloatValue

Przyczyna w kodzie: `coerce_value_to_ty` dla wszystkiego, co nie jest bool ani floatem (a float i tak zwraca błąd „not supported” tylko na części ścieżek), woła `value.into_int_value()`. Inkwell na złym wariancie **panikuje**, zamiast zwrócić `Err`. Nie ma match na `BasicValueEnum`.

WARNING

WARNING. Panic kompilatora na poprawnym według frontendu programie jest błędem projektu backendu. Nie obchodź jej „bo język zabrania”. Język, w typeck i semie, tego nie zabrania. Dodatek C trzyma to przy innych długach.

Błąd narzędzi, nie programu, kończy się kodem 2: zły flag, brak pliku, nakładające się ścieżki, brak clang albo brak feature codegen. Tekst zaczyna się od `error:` bez fazy.

Jak czytać kilka błędów naraz

Frontend nie implementuje severity „pierwszy błąd wygrywa”. Dostajesz wszystkie. Heurystyka, która oszczędza czas:

1. Najpierw parse. Przy błędzie składni reszty nie ma.
2. Potem type o niezgodności podstawowej (expected, unknown).
3. Potem ownership o move.
4. codegen tylko gdy check jest czysty albo gdy build i tak doszedł do bramki. Build woła `frontend::check` i przy błędach frontendu w ogóle nie wchodzi w LLVM; wypisuje te diagnostyki i kończy się 1.

Sprawdzone na bork build programu z None: faza codegen, kod 1, pliku wyjściowego nie ma. Program z błędem ownership na return z wewnętrznego regionu też kończy build kodem 1, z tekstem `inner region`, bez binarki. Escape jest częścią checku, więc build nawet nie udaje, że to problem LLVM.

Podsumowanie

- Język nie ma wyjątków. Ma diagnostykę kompilacji i trzy awarie runtime.
- Linia diagnostyki niesie fazę. Pusty span pomija numer linii.
- ownership to sema albo escape. Escape milczy, gdy wcześniej są inne błędy.
- Bramka codegen mówi wprost `not supported`. Część emisji mówi `not supported yet`.

- Przekazanie `String` do funkcji użytkownika nie daje diagnostyki. Zabija kompilator.
- Kod 1 to zły program. Kod 2 to złe wywołanie albo brak narzędzia. Kod 101 to panic Rusta.

Rozdział 11. Biblioteka, brak modułów i idiomy

Ten rozdział obejmuje

- Dlaczego nie ma rozdziału o module w sensie językowym
- Całą bibliotekę standardową, która naprawdę istnieje
- Idiomy, które `check` i `bork build` udźwigną
- Idiomy tylko frontendu
- Krótki katalog „tak się tego nie pisze”

Modułów nie ma

Parser na poziomie programu oczekuje `NL` albo `fun`. Token `module` i `token struct` są błędami `expected "NL", "fun"`. Nie ma `import`, ścieżek, widoczności `pub` ani `crate`ów użytkownika. Jeden plik, jedna lista funkcji, jedna mapa nazw.

Słowo „moduł” w tej książce dalej występuje, ale o `crate`ach Rusta: `bork` i `bork_runtime`. To nie jest mechanizm języka Bork.

LSP i rozszerzenie edytora (`tools/bork-lsp-extension`) są narzędziami obok języka. Nie dodają składni.

Cała biblioteka

Trzy `intrinsic`s. Nic więcej nie jest zarejestrowane w `builtins::resolve`.

Piszesz	Dostajesz
<code>print(x)</code>	<code>x</code> na <code>stdout</code> , <code>flush</code> , <code>x</code> to <code>i32</code> , <code>i64</code> albo <code>String</code>
<code>println(x)</code>	to samo i <code>\n</code>
<code>concat(a, b)</code>	nowy <code>String</code> w <code>sinku</code> albo w bieżącej arenie

Nie ma `assert`, `format`, wejścia z `stdin`, plików, alokacji poza areną, zegara ani argumentów wiersza poleceń. `main` nie dostaje `argv`. `Codegen` odrzuca `main` z parametrami przy deklaracji funkcji LLVM (`emit_fn`).

Runtime C ABI, którego nie wołasz z Borka wprost, to `bork_arena_push`, `bork_arena_pop`, `bork_arena_reset`, `bork_arena_alloc`, `bork_print_i64`, `bork_println_i64`, `bork_print_str`, `bork_println_str`. Rozdział 17.

Idiomy, które warto kopiować

Wynik liczbowy jako kod wyjścia, gdy ćwiczysz codegen bez drukowania.

```
fun main(): i32 {  
    return 7  
}
```

Uruchomione: kod 7. Tak robi test `builds_return_constant`.

Pętla i akumulator Copy. Listing 6.3. Indeks i akumulator są i32, więc ciało nie walczy z własnością.

Napis w main, druk na końcu. Listing 9.1. Nie zwracaj świeżego napisu z pomocnika, dopóki nie ma areny wyniku.

val dla napisu, który dziecko tylko czyta. Listing 2.2. `Shared` jest tańsze pojęciowo niż `move wte` i `wewte`.

Dwie linie, gdy hoist ma pomóc. `var piece = concat(...)` i od razu `outer = move piece`. Nie wstawiaj między nie `println`.

Tablica i wycinek o stałych granicach, gdy długość jest znana. Listing 7.4. Nie używaj wycinka jako „`vec slice`” z granicami z zmiennych: `typeck` tego nie przyjmie.

if z dwoma blokami, gdy potrzebujesz wartości. Listing 6.1.

Idiomy tylko frontendu

Pisz je, gdy ćwiczysz checker albo LSP. Nie wkładaj ich do programu, który ma być binarką.

- Trailing closure i `move { }` po wywołaniu.
- `Some`, `None`, `?:`, `!!`, `?:..`
- Wartość typu funkcji w `val` i próba wywołania pośredniego.
- `String` jako parametr funkcji użytkownika. To nawet nie jest „tylko frontend”: frontend jest za, codegen panikuje. Trzymaj się z daleka od tej kombinacji w `bork build`.

Listing 11.1. Fragment próbki `PROCESS_USER_SAMPLE` z `src/lib.rs`. Cała próbka przechodzi check (uruchomione). `bork build` odrzuca `?:`, `Some` i `None`.

```
fun processUser(name: String?, score: i32): i32 {  
    val fallbackName: String = name ?: "Guest"  
    val verifiedUser: String? = Some(fallbackName)  
    val emptyMiddle: String? = None  
    val verifiedLength: i32 = verifiedUser?.length ?: 0  
    if (verifiedLength > 0) {  
        return score  
    }
```

```

    }
    return 0
}

```

Tak się tego nie pisze

Pokusa	Co się stanie
<pre> val s = "a" + "b" return -1 </pre>	błąd typu, + nie jest konkatenacją błąd parsowania, minus nie jest jednoargumentowy
<pre> val n = 1; var x = s dla var s: String concat(left, right) gdy oba są var w zagnieżdżonym bloku return concat(a, b) a[i] z indeksem spoza N 1 / 0 drugie fun println struct albo module </pre>	błąd parsowania, średnik use move s to transfer ownership not Copy; move it into Block escape, „not supported yet” kompiluje się, proces aboruje kompiluje się, proces aboruje cannot redefine builtin błąd parsowania na poziomie programu

Styl, który pasuje do regionów

Krótki blok robi jedną pulę tymczasową. Długi blok funkcji trzyma dane, które mają przeżyć pętlę. Pętla nie powinna przenosić stanu z zewnątrz; powinna czytać Copy albo Shared val i zapisywać wynik do zewnętrznego var przez przypisanie.

Nazwy val dla progów i napisów tylko do odczytu, var dla akumulatorów. To nie jest konwencja formatowania. To jest różnica, którą sema sprawdza.

Nie ma rustowego clone(). Świadoma kopia napisu to concat(s, "") tylko wtedy, gdy oba argumenty są legalne w tym regionie, albo przypisanie literału. Nie ma ogólnego głębokiego kopiowania tablicy poza zbudowaniem nowego literału. Wycinek nie jest kopią.

Podsumowanie

- Biblioteka standardowa to print, println i concat.
- Modułów, struktur i import nie ma. Jeden plik.
- Idiom binarki: liczby w funkcjach, napisy w main, val do Shared, var do akumulatora.
- Nullable i trailing closures są prawdziwym językiem frontendu i nie są językiem bork build.
- + na napisach, średnik, minus jednoargumentowy i gołe użycie var String są błędami, nie lukami stylu.

Rozdział 12. Pipeline od źródła do binarki

Ten rozdział obejmuje

- Kolejność faz w `frontend::check` i w `codegen::build`
- Co niesie każda struktura danych
- Dlaczego `typeck` jest przed `sema`, wbrew starszej specyfikacji
- Kiedy HIR znika, a raport `aren` zostaje
- Diagram crate'ów i przepływu

Jedno przejście, dwie bramy

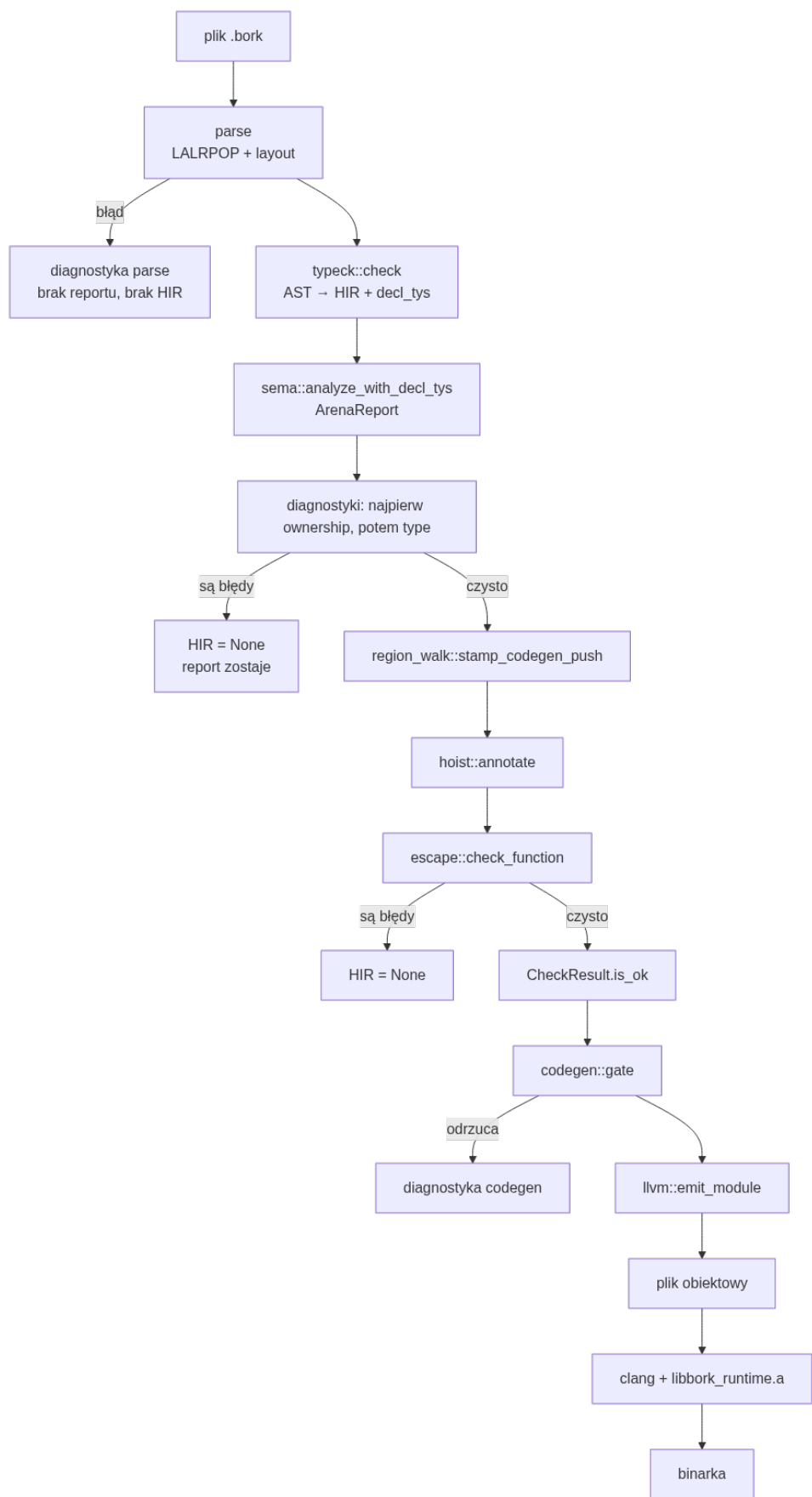
Kompilator nie ma interpretera i nie ma MIR. Są dwie bramy wejścia: bork `plik.bork` kończy na `frontend::check`, bork `build` woła ten sam `check` i potem, tylko przy pustych diagnostykach, bramkę HIR oraz LLVM.

Komentarz na górze `src/frontend.rs` mówi „`parse -> typeck -> sema -> HIR`”. To jest skrót. Po czystym `typeck` i `semie` dochodzą `stempel`, `hoist` i `escape`. HIR w `CheckResult` jest obecny **wtedy i tylko wtedy**, gdy lista diagnostyk jest pusta po `escape`.

CheckResult

```
pub struct CheckResult {  
    pub report: Option<ArenaReport>,  
    pub hir: Option<HirProgram>,  
    pub diagnostics: Vec<Diagnostic>,  
}
```

Pole	Kiedy Some / niepuste
report	parse się udał, także przy błędach typu, własności i escape
hir	zero diagnostyk
diagnostics	zbiór ze wszystkich faz, które zdążyły pobiec



Rysunek 1: Diagram 1

`is_ok()` to pusty wektor, nie osobna flaga.

Kolejność w wektorze: najpierw błędy semy (`from_sema`), potem doklejone błędy typeck. Typeck **liczył się pierwszy**, bo `sema` potrzebuje `decl_tys`. Drukuje się odwrotnie. To nie jest przypadek testu, tylko `frontend::check` linie 38-42.

Dlaczego typeck przed sema

Starsza specyfikacja typed HIR ([docs/superpowers/specs/2026-09-23-typed-hir-llvm-design.md](https://docs.superpowers/specs/2026-09-23-typed-hir-llvm-design.md)) opisuje kolejność „najpierw własność, potem typy”. Kod robi odwrotnie. `Sema` czyta `Vec<hir::Ty>` w kolejności deklaracji `val/var` i zdejmuje ją `pop_front` przy każdym `VarDecl`. Bez typecku nie wie, czy nazwa jest `Copy`. Funkcja `sema::analize` (używana w części testów) woła `typeck` wewnętrznie jeszcze raz. Frontend używa `analize_with_decl_tys`, żeby nie płacić dwa razy.

`Sema` nie dostaje całego HIR. Dostaje AST i cienki wektor typów deklaracji. Sygnatury wołań (`fun_sigs: HashMap<String, Vec<BindingKind>>`) buduje sobie z AST, nie z HIR. `Kind` parametru (`val/var`) jest informacją własności, nie tylko typem.

Co która struktura pamięta

Struktura	Pamięta	Nie pamięta
<code>ast::Program</code>	składnię, spany, rodzaje wiązań	typów wywnioskowanych, <code>aren</code>
<code>hir::HirProgram</code>	typy na wyrażeniach, <code>UseKind</code> , <code>alloc_in_binding</code> po hoiście	identyfikatorów regionów
<code>sema::ArenaReport</code>	drzewo regionów, własność nazw, później <code>codegen_push</code>	typów HIR w pełni (część typów spłyca się do <code>Unknown</code>)
moduł LLVM	instrukcje, globalne literały, deklaracje runtime	źródła; diagnostyka jest wcześniej

Komentarz w `src/hir/mod.rs` jest normą dla reszty kompilatora: „HIR carries no region identity. Arenas and nesting live in `sema::ArenaReport`”. `Codegen` idzie po HIR i po raporcie **równocześnie**, spacerem `region_walk`. Rozjazd liczby dzieci areny z liczbą miejsc w HIR jest błędem wewnętrznym (`internal arena schedule mismatch`), nie błędem użytkownika.

bork build po checku

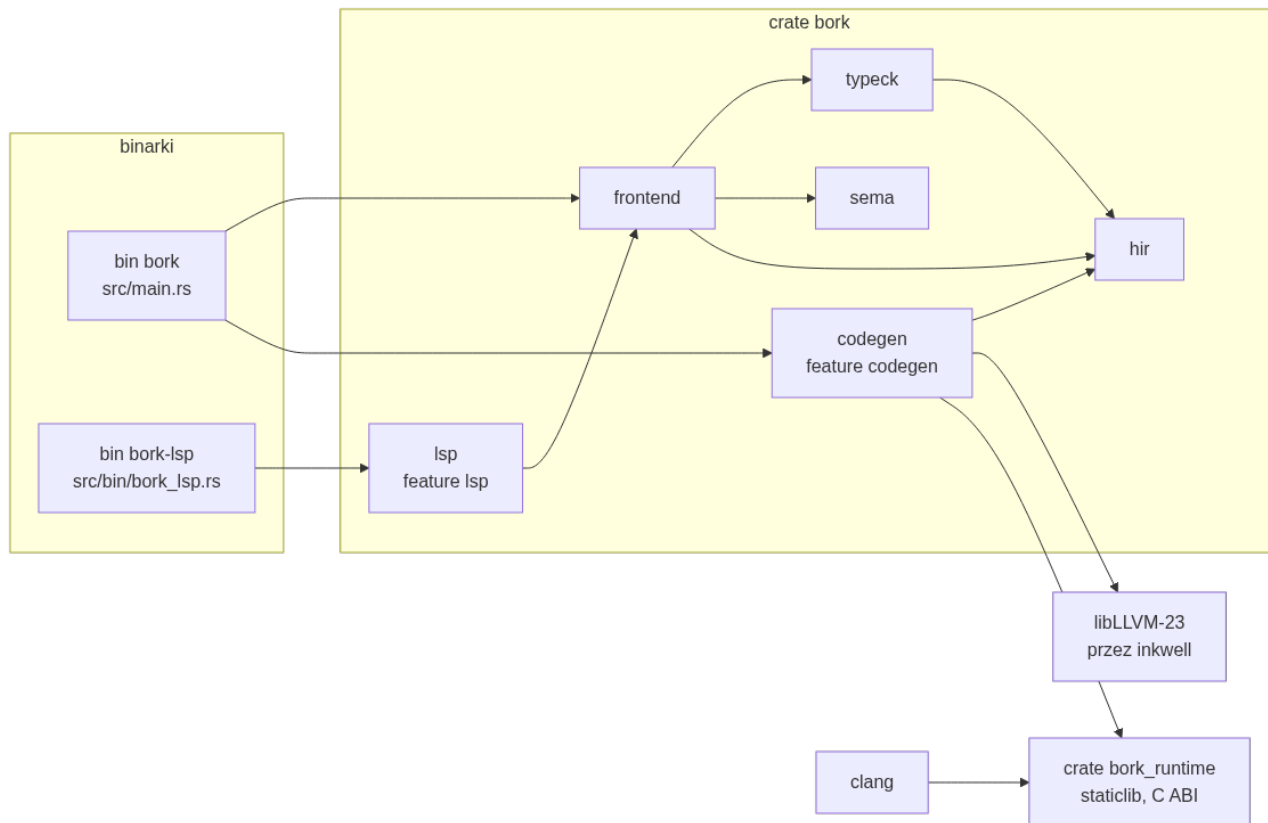
`codegen::build` (`src/codegen/mod.rs`):

1. Wymaga czystego `CheckResult`. Inaczej zwraca diagnostyki frontendu.
2. `gate(hir)` — czysto składniowy filtr na HIR.
3. `Context::create()`, `emit_module`, `write_object`.

4. `link::link_executable` woła `clang` z plikiem obiekowym i `libbork_runtime.a`.

Błąd linkera i brak `clang` to `BuildError::Toolchain`, kod wyjścia 2, tekst error: ... bez fazy. Błąd bramki to `BuildError::Diagnostics`, kod 1.

Diagram zależności crate'ów



Rysunek 2: Diagram 2

`bork_runtime` nie jest zależnością Cargo crate'a `bork` w sensie `use`. `build.rs` przy `feature codegen` kompiluje `go rustc --crate-type staticlib` do `libbork_runtime.a` i przekazuje ścieżkę przez `BORK_RUNTIME_LIB`. Program użytkownika linkuje ten archiwum. Sam kompilator linkuje `libLLVM-23` dynamicznie (`llvm23-1-force-dynamic`).

Feature `lsp` (domyślny) dokłada `tower-lsp`, `tokio`, `serde_json` i binarkę `bork-lsp`. Feature `codegen` dokłada `inkwell` i `tempfile`. Da się złożyć checker bez żadnego z nich (`--no-default-features`). Wtedy `bork build` odmawia kodem 2.

Czego w potoku nie ma

Nie ma osobnej reprezentacji MIR, nie ma optymalizatora poza tym, co LLVM zrobi na module, nie ma passu inliningu domknięć (domknięcia nie dochodzą do LLVM), nie ma monomorfizacji (nie ma generyków). Hoist jest jedyną adnotacją middle-endu na HIR i jest lokalny.

`src/arena.rs` nie stoi na tym wykresie jako faza. To bliźniaczy model płyty 4 KiB. Sema go nie woła. Runtime ma własną kopię stałej.

Podsumowanie

- Check to parse, typeck, sema, potem przy braku błędów stempel, hoist i escape.
- HIR istnieje w wyniku tylko dla programu bez diagnostyk.
- Raport aren przeżywa błędy semantyczne i ginie tylko przy błędzie parsowania.
- Typeck jest przed semą, bo sema konsumuje typy deklaracji. Starsza specyfikacja opisuje odwrotność.
- HIR nie ma identyfikatorów regionów. Codegen synchronizuje HIR z raportem w `region_walk`.
- Binarka to obiekt LLVM zlinkowany z `libbork_runtime.a` przez clang.

Rozdział 13. Lexer, parser i AST

Ten rozdział obejmuje

- LALRPOP i wbudowany lekser
- Piętra pierwszeństwa
- AST: które enumy naprawdę istnieją
- Normalizację nawiasów
- Jak błąd parsowania staje się Diagnostic

Narzędzie

Gramatyka jest w `src/parser.lalrpop`. `build.rs` woła `lalrpop::process_src()`. Moduł `parser` jest publiczny przez `lalrpop_mod!`. Wejście do reszty kompilatora to `bork::parse` w `src/lib.rs`: najpierw `normalize_parenthesized_newlines`, potem `ProgramParser::new().parse`. Typ błędu to `ParseError<usize, String, &'static str>`.

Nie ma osobnego pliku tokenów. Lekser jest blokiem `match` w gramatyce.

Lekser

Pomijane: spacje, tabulatory, `\r`, `\f`, oraz `//` do końca linii. Złamania linii, które nie stoją tuż przed `else`, stają się tokenem `NL`. `else` wchłania poprzedzające puste linie i komentarze, żeby `if` mógł złamać linię przed `else`.

Słowa kluczowe w `match`: `if`, `fun`, `val`, `var`, `for`, `in`, `return`, `Some`, `None`, `move`, `promote`. Reszta, w tym `while` i `true`, idzie ścieżką domyślną i jest porównywana jako literal terminala w produkcji.

Liczba całkowita to `[0-9]+` i mieści się w `i64`, inaczej błąd użytkownika `integer literal out of range for i64`. Float to cyfry.cyfry (`invalid float literal` przy przepełnieniu parsowania). Napis przechodzi przez `unescape_string_literal` w `ast.rs`.

Piętra

Od najciaśniejszego do najluźniejszego, tak jak produkcje wołają siebie:

Produkcja	Operatory
ExprSuffix	wywołanie, [i], [lo..hi], . / ?., przyrostkowe !!
ExprUnary	!
ExprMul	* /
ExprAdd	+ -
ExprCompare	> < >= <= == !=
ExprRange	..
ExprAnd	&&
ExprOr	\ \
ExprElvis	?:

?: jest prawostronnie łączny, bo prawa produkcja woła z powrotem ExprElvis. Reszta binarna jest lewostronna przez BinTier.

docs/language.md w jednym zdaniu o pierwszeństwie wymienia wywołania, * /, + -, porównania, .. i ?: . Pomija &&, || i !, które w gramatyce są. Książka trzyma się gramatyki. && wiąże ciaśniej niż ||, oba luźniej niż porównanie i ciaśniej niż ?: .

Indeks i granice wycinka używają ExprCompare, nie pełnego Expr. Bez dodatkowych nawiasów nie włożysz ?: , || , && ani .. w środek [...].

AST

Plik src/ast.rs. Skrót, który wystarcza, żeby czytać typeck.

Program ma functions. Function ma name, params, return_type, body. Param ma BindingKind (Val albo Var), SpannedName i Type.

Type to Primitive, Named, Array { elem, len, nullable }, Func { params, ret, nullable }.

Stmt to Block, VarDecl, Assign, For, While, Break, Continue, MoveBlock, Return, Expr. AssignTarget to Name albo Index. Cel inny niż te dwa ginie już w gramatyce.

Expr to literały (Int, Float, Bool, Str, ArrayLit), Index, Slice, Ident, Move, Promote, None, Some, Binary, Unary, Field, Call, If. Call niesie trailing: Option<Closure>. Closure ma params, body, is_move, captures.

BinOp obejmuje arytmetykę, porównania, RangeTo, Elvis, And, Or. UnaryOp to Not i NotNullAssert (!!).

Spany są Span { start, end } w bajtach. Nazwy często są SpannedName { name, span }. Specyfikacja parsera MVP mówiła „bez spanów”. To jest nieaktualne.

MoveBlock.captures i Closure.captures są Option<Vec<...>>. None znaczy „wnioskuj”. Some puste znaczy „jawnie nic”. Tej różnicy nie wolno spłaszczyć do pustego wektora.

Layout

src/layout.rs. Wewnątrz (...) na głębokości klamer z momentu otwarcia nawiasu, \n staje się \r. Pomiń stringi i komentarze. Długość pliku w bajtach się nie zmienia, więc diagnostyka nie jedzie.

Testy: tests/parser.rs (pusta program, próbka MVP, przypisanie indeksu, val/var parametrów, move, komentarze, pierwszeństwo, while, błędy lokalizacji). To jest pierwszy plik testowy, który warto czytać, gdy ruszasz gramatykę.

Z błędu parsowania w diagnostykę

diag::from_parse mapuje warianty ParseError na Phase::Parse. User { error } dostaje span Span::new(source.len(), source.len()). Dlatego „assignment target must be a name or name[index]” wskazuje koniec pliku. Poprawka byłaby lokalna w from_parse albo w akcji =>, która dziś nie niesie pozycji. Nikt jej jeszcze nie zrobił.

frontend::check przy błędzie parse wraca natychmiast. Typeck i sema się nie wykonują.

Podsumowanie

- Gramatyka LALRPOP jest jedynym lekserem i parserem.
- Nowa linia jest tokenem NL, chyba że layout zamienił ją w \r wewnątrz nawiasów, albo stoi przed else.
- Option przy liście move odróżnia wnioskowanie od pustej listy.
- Pierwszeństwo obejmuje !, && i ||, nawet jeśli skrót w language.md je pomija.
- Błąd User z gramatyki ma span na EOF.
- AST nie niesie typów wywnioskowanych. To robota HIR.

Rozdział 14. Analiza regionów (sema)

Ten rozdział obejmuje

- ArenaNode, Ownership, Analyzer
- Wejście w region: `open_ordinary` i `open_move`
- `classify_use` i politykę transferu
- Scalanie `moved` po `if` i zakaz w pętli
- Czego sema świadomie nie wie

Sema nie alokuje płyt

`src/arena.rs` zaczyna się komentarzem, że sema go nie używa. Sema buduje drzewo opisujące, które nazwy w którym bloku żyją i jak przekroczyły granicę. Identyfikator to `ArenaNode.id`: `usize` z licznika `Analyzer::alloc_id`. Nie ma typu `ArenaId`. Nie ma też enumu `RegionKind`, choć plany o nim wspominały. Region jest albo zwykły (`open_ordinary`), albo move (`open_move`). Reszta to etykieta napisowa.

Struktury

`ArenaReport { roots }` — jeden korzeń na funkcję.

`ArenaNode` ma `id`, `label`, `compacted_braces`, `codegen_push`, `bindings`, `observations`, `children`. `bindings` to deklaracje i przechwycenia. `observations` to użycia spoza deklaracji (`Copy`, `Shared`, `Move`). Dump pomija obserwacje `Local`, żeby nie dublować deklaracji, z wyjątkiem miejsc, gdzie `Local` i tak wpada w listę `bindings`.

`Ownership` to `Local`, `Copy`, `Shared { from }`, `Moved { from }`.

`EnvBinding` pamięta `arena_id`, `arena_label`, ty (`Known(ast::Type)` albo `Unknown`), `kind`, `moved`, `origin` (`Declared` albo `Captured`).

`Analyzer` trzyma środowisko `HashMap<String, EnvBinding>`, sygnatury `fun_sigs`, kolejkę `decl_tys`, stos `loop_move_ban` i wektor błędów `SemaError { message, name, span }`.

Wejście

`analyze_with_decl_tys` rejestruje kind parametrów każdej funkcji, potem `analyze_function` woła `open_ordinary(az, "fun {name}", body, params)`.

`open_ordinary / open_move (src/sema/walk.rs)`:

1. `peel_blocks` zdejmuję opakowania z samych bloków i liczy je.
2. `open_frame` bierze nowe id, buduje `RegionFrame`, wiąże parametry.
3. Dla `move`: `bind_capture` każdego imienia z listy (jawnej albo wynioskowanej). Przechwycenie woła `move_source`, cieniuje nazwę jako `Captured`, dopisuje `Ownership::Moved` do dziecka i po `finish` ustawia `moved` u rodzica.
4. `walk_block` schodzi w instrukcje.
5. `finish` zdejmuję cienie.

`resolve_move_captures`: jawna lista wygrywa. Przy `None` zbierane są `free_vars_in_block`, odfiltrowane o parametry, zostają nazwy żywe i nie-Copy. `Expr::Move` i `Expr::Promote` nie są zmiennymi wolnymi.

Klasyfikacja użycia

`classify_use` w `src/sema/policy.rs` jest całym modelem Copy/Shared/Move w kilkunastu liniach:

1. `binding.moved` → błąd use of nazwa after move from etykieta.
2. Ta sama `arena_id` → `Local`.
3. `ty.is_copy()` → `Copy`.
4. `Val` → `Shared { from: etykieta rodzica }`.
5. Inaczej → nazwa is not Copy; move it into etykieta with move.

`Assign-up` obchodzi krok 1-5 dla **lewej** strony: jeśli cel jest `Var`, nie `moved`, i `arena_id < node.id`, `sema` nie woła `note_use`.

`bare_ident_move_message` daje krótsze odpowiedzi przy transferze: `use move nazwa to pass ownership` dla argumentu, `use move nazwa to transfer ownership` dla przypisania w tej samej okolicy, oraz dłuższy tekst „not Copy” gdy areny się różnią.

Pętla i if

Na `For` i `While` `sema` pcha na `loop_move_ban` zbiór wszystkich kluczy środowiska i zdejmuję go po ciele. `move_source` zwraca `InLoop`, a `error_move_in_loop` buduje zdanie o kolejnych iteracjach.

Na `If` `sema` zapamiętuje zbiór `moved`, schodzi w `then`, przywraca flagi, schodzi w `else` jeśli jest, przywraca znowu i woła `apply_moved_merge`: nazwa jest `moved` po `if`, gdy była `moved` wcześniej **albo** (`moved` w `then` i `moved` w `else`, gdy `else` istnieje). `Then-only` nie zostaje.

To jest analiza na flagach boolowskich, nie na siatce ścieżek. Nie ma „`moved` jeśli warunek jest prawdziwy”. Albo obie gałęzie, albo nic.

Czego sema nie robi

- Nie sprawdza, czy bajty concat przeżyją return. To escape.
- Nie typuje parametrów trailing closure (Ty::Unknown w walk.rs).
- Nie rozumie indeksu for inaczej niż jako Int.
- Nie ma ścieżek wyjątków, bo język ich nie ma.
- Nie kompaktuje aren bajtowo. compacted_braces dotyczy tylko gołych nawiasów w AST.

Ty::Unknown nie jest Copy. Test unknown_type_is_not_treated_as_copy trzyma ten konserwatyzm. Lepiej dostać zbędne „not Copy” przy już błędnym typie niż puścić ucieczkę.

promote jest w apply_expr_promote. W katalogu src/sema/tests/ nie ma osobnego pliku o promote. Zachowanie jest w walk.rs i w testach wyższego poziomu (check + codegen). Gdy czytasz semę, nie wnioskuj z milczenia testów jednostkowych, że promote jest niezaimplementowany. Listing 8.3 przechodzi bork build.

Testy, które definiują kontrakt

Plik	Co zamyka
sema/tests/smoke.rs	peel_blocks, próbka MVP
sema/tests/infer.rs	brak fałszywych move przy Copy
sema/tests/expr_move.rs	wyrażeniowy move, ponowny move, assign-up
sema/tests/regional.rs	bloki move, pętle, scalanie if, kompaktowanie
sema/tests/call.rs	val/var na granicy wywołania
sema/policy.rs testy	classify_use i treść odpowiedzi

Dump ASCII jest w src/dump.rs, nie w semie. Sema produkuje drzewo, dump je drukuje. LSP używa tego samego dump_arenas.

Podsumowanie

- Sema buduje drzewo ArenaNode i flagi moved. Nie dotyka płyt 4 KiB.
- classify_use jest definicją Copy, Shared i Move.
- None kontra Some([]) przy przechwyceniu jest różnicą językową i różnicą w AST.
- Pętla zabrania move nazw widocznych na wejściu. if wymaga zgodności obu gałęzi.
- Escape i hoist są później i nie żyją w tym katalogu.
- Parametry domknięcia mają w semie typ Unknown, więc dump potrafi pokazać Shared tam, gdzie typeck widzi Copy.

Rozdział 15. Typeck i typowany HIR

Ten rozdział obejmuje

- Ty, Prim, UseKind
- Jak checker schodzi po instrukcjach i wyrażeniach
- Różnicę AST i HIR na move / promote
- Wnioskowanie literałów i truciznę Unknown
- Dlaczego HIR nie wystarcza backendowi sam

Po co osobne drzewo

AST jest kształtem źródła. HIR jest tym samym kształtem po typach, z kilkoma rzeczami wyciętymi albo doklejonymi:

- każde wyrażenie to `HirExpr { ty, span, kind }`,
- `Expr::Move` i `Expr::Promote` stają się `Ident { use_kind: Move | Promote }`,
- `Call` traci ciało domknięcia jako `Option<Closure>` i zostawia flagę `has_trailing_closure` (ciało i tak jest osobno w miejscach, gdzie typeck schodził w blok; flaga służy bramce),
- `VarDecl` dostaje `ty` oraz `alloc_in_binding`, na starcie `None`,
- nie ma spanów na parametrach, są gołe `String`.

`UseKind` to `Local`, `Shared`, `Copy`, `Move`, `Promote`. `Typeck` wpisuje go w `check_ident`. **Nie** emituje błędów własności. Komentarz i podział odpowiedzialności są świadome: własność krzyczy sema, HIR tylko opisuje, jak typeck widział użycie. `Codegen` później ufa `UseKind` przy kopii deskryptora. Gdyby typeck i sema się rozjechały, backend kopiowałby co innego, niż sema pozwoliła. Na prymitywach i na napisach w main testy trzymają je razem. Na parametrach domknięcia sema i tak ma `Unknown`.

Ty

```
src/hir/ty.rs. Ty { kind, nullable }.
```

```
TyKind: Prim(Prim), Named(String), Array { elem, len }, Func { params, ret },  
Range(Box<Ty>), Unknown.
```


Range i Unknown nie mają odpowiednika w `ast::Type`. Zakres istnieje tylko po typecku, jako typ wyrażenia `lo..hi`.

`is_copy`: nienullowalny prymityw. `uses_arena_storage`: String albo nienullowalna tablica. To drugie jest predykatem „czy escape i codegen muszą myśleć o płycie”.

`lower_type` odrzuca `[T]?` i każdą nazwę poza String.

Zejsście

`typeck::check` zbiera sygnatury, odmawia redefinicji wbudowanych, potem sprawdza ciała. Env ma stos zakresów, mapę funkcji, diagnostyki, wektor `decl_tys` (dopisywany przy każdej deklaracji, w kolejności źródła) i `loop_depth`.

Instrukcje są w `typeck/stmt.rs`. Wyrażenia są podzielone:

Plik	Co
<code>expr/mod.rs</code>	dyspozytor, identyfikatory, <code>UseKind</code>
<code>expr/binary.rs</code>	arytmetyka, porównania, <code>&& \ \ </code> , zakres
<code>expr/call.rs</code>	wołania, trailing closure, wbudowane
<code>expr/control.rs</code>	<code>if</code> , bloki jako wartość
<code>expr/array.rs</code>	literał, indeks, wycinek
<code>expr/field.rs</code>	<code>. i ?</code> .

Ostatnia instrukcja bloku, który ma produkować wartość, musi być `Stmt::Expr`. W przeciwnym razie typ bloku to `unit` (`check_value_block_in_current_scope`). Dlatego `return` w gałęzi `if` nie jest „wartością gałęzi”. Jest instrukcją, a gałąź jako wyrażenie ma typ `unit`, jeśli ostatnią rzeczą nie jest gołe wyrażenie.

Literały i oczekiwany typ

Checker przekazuje `expected: Option<&Ty>` w dół. Literał całkowity bez oczekiwania to `i32`. Z oczekiwaniem całkowitym przyjmuje ten typ. Float bez oczekiwania to `f64`. None bez oczekiwania to błąd, nie „jakiś nullable”.

Unknown tłumy kaskadę: porównanie z Unknown nie dokłada kolejnego `expected`. Nie tłumy semy.

Wbudowane a zwykłe wołanie

`call.rs` rozpoznaje `print` / `println` / `concat` zanim sprawdzi arność użytkownika. Dlatego `print` przyjmuje String, choć tabela `builtins::signatures` mówi (`i32`) `-> unit`. To jest specjalny przypadek, nie ogólna podtypowość. Inna funkcja o parametrze `i32` nie przyjmie String.

Trailing closure dokleja się jako ostatni argument typu funkcyjnego. Ciało jest sprawdzane w nowym zakresie. Sema robi osobny spacer po AST i **nie** widzi typów

wyliczonych dla parametrów domknięcia. Dwa spacery, dwa środowiska. To jest powód rozjazdu Shared kontra Copy z rozdziału 5.

HIR a backend

Backend nie powinien odtwarzać zasięgu z HIR. Ma iść `region_walk`, który na każdym `For`, bloku, `if` i domknięciu bierze kolejne dziecko `ArenaNode`. Jeśli `typeck` wyrzuci albo wstawi blok inaczej niż `sema`, spacer padnie. `peel_blocks` musi być wspólny. Jest skopiowany w `hir` i w `sema` z komentarzem, żeby się nie rozjechał. Nie ma jednego wspólnego traitu. Jest konwencja.

`alloc_in_binding` jest puste po `typecku`. Wypełnia je `hoist`, już na HIR, mutowalnie, w `frontend::check`. To jedyna adnotacja middle-endu.

Testy

`src/typeck/tests/mod.rs` jest duży i idzie przez `frontend::check`, nie przez sam `typeck`. Dzięki temu łapie też własność. `nullable.rs` trzyma `?:`, `!!` i porównania na `nullable`. `closures.rs` trzyma arność `trailing closure`. Gdy dodajesz operator, dopisujesz tu przypadek z dokładnym tekstem diagnostyki. Rozdział 21.

Podsumowanie

- HIR niesie typ i `UseKind`. Nie niesie identyfikatora regionu.
- `move` w AST staje się `UseKind::Move` na identyfikatorze.
- `Typeck` nie zgłasza błędów własności. `Sema` nie czyta `UseKind`.
- Oczekiwany typ steruje literałami, `None` i tym, czy `if` bez `else` jest błędem czy `unit`.
- Dwa spacery, `typeck` i `sema`, spotykają się dopiero w `region_walk`. Zgodność `peel_blocks` jest warunkiem tej zgody.

Rozdział 16. Hoist, escape i region_walk

Ten rozdział obejmuje

- Kiedy te trzy passy w ogóle będą
- Wzorzec hoista i jego granice
- Model głębokości w `escape::place`
- Po co jest jeden spacer regionów
- Flaga `codegen_push`

Tylko na czystym programie

W `frontend::check`, gdy wektor diagnostyk po semie i typecku jest pusty:

1. `region_walk::stamp_codegen_push(&hir, &mut report)` ustawia `ArenaNode.codegen_push`.
2. `hoist::annotate` dopisuje `alloc_in_binding`.
3. `escape::check_function` na każdej funkcji może dopisać diagnostyki i wtedy HIR wraca do `None`.

Przy błędzie typu hoist się nie wykona. Nie zobaczysz „drugiego” błędu escape obok błędu typu. To upraszcza passy (zakładają drzewo, które typeck uważa za spójne) i utrudnia diagnostykę łączoną.

Hoist

`src/hoist.rs`. Cel: inicjalizator, który i tak natychmiast ucieka do zewnętrznego `var`, budować od razu w jego arenie.

`try_hoist` patrzy na instrukcję `i` oraz `i+1`:

- `i` to `VarDecl` o nazwie `inner`, jeszcze bez `alloc_in_binding`,
- `i+1` to `Assign` na nazwę (nie na indeks),
- wartość deklaracji to literał napisowy albo wywołanie,
- prawa strona przypisania to identyfikator `inner` z `UseKind::Move`,
- nazwa celu jest w zbiorze `outer` (parametry i nazwy z zewnątrz) albo w nazwach już zadeklarowanych w tym bloku.

Wtedy `alloc_in_binding = Some(cel)`.

Zbiór outer przy zejściu w blok, for i while jest sumą dotychczasowych nazw. Hoist nie przeskakuje między gałęziami if. Nie patrzy na dominację. Dwie linie albo nic.

Codegen czyta pole przy emisji deklaracji i ustawia sink alokacji na dom celu. Jeśli pole jest puste, inicjalizator idzie do bieżącej płyty, a move przy przypisaniu kopiuje bajty (copy_into_arena).

Escape

src/escape.rs. Pytanie: na jakiej głębokości leżą bajty, i czy konsument jest co najmniej tak płytki, żeby je jeszcze widzieć.

Local { decl_depth, value_depth } na wiązaniu. value_depth potrafi być płytsze niż deklaracja, gdy hoist albo sink położył bajty wyżej. Sam pass nie zapisuje tego z powrotem do semy. Odrzuca albo przepuszcza.

place(expr, sink):

Wyrażenie	Głębokość bajtów
literal liczbowy, bool, napis, None	0
literal tablicy	maksimum z głębokości elementów i z głębokości bufora; bufor to sink, a gdy sink jest None albo 0, bieżąca głębokość
indeks typu z areną	głębokość wartości tablicy, nie bieżącego bloku
wycinek	głębokość odbiorcy
identyfikator Move/Promote przy sinku Some(d)	d
identyfikator Move/Promote przy sinku Some(0) (return)	value_depth źródła
concat	sink albo bieżąca głębokość
inne wołanie typu z areną	maksimum głębokości argumentów
if	maksimum gałęzi

return wartości z areną przy głębokości > 0 jest błędem. Potem check_return_string_form osobno zabrania concat i move/promote w wyniku, nawet gdy głębokość wyszła 0. Stąd return concat pada także w funkcji bez zagnieżdżenia.

Przypisanie liczy place ze sinkiem równym decl_depth celu. Jeśli value_depth celu zostałoby głębsze niż deklaracja, błąd „assigning a value whose bytes live in an inner region”.

Gałąź if otwarta jako region produkujący wartość (yields: true), bez sinku, z bajtami na głębokości tej gałęzi, daje błąd o String w gałęzi. Tekst jest na sztywno o String. Warunek jest uses_arena_storage.

Diagnostyki escape mają Phase::Ownership. Część z nich ma span: None (concat na return). CLI pomija wtedy numer linii.

Jeden spacer

`src/region_walk.rs` istnieje po to, żeby stempel, harmonogram codegenu i emisja LLVM nie miały trzech lekko różnych pętli po HIR. Komentarz na górze pliku: jeden spacer zsynchronizowany z dziećmi `ArenaNode`.

`RegionSite` nazywa miejsca, które otwierają dziecko raportu: blok, pętla `for`, `while`, gałęzie `if`, domknięcie. Etykiety muszą pasować do tych, które sema wpisała. Inaczej `take_child` nie znajdzie węzła.

`RegionVisitor` ma haki: wejście w funkcję, `enter_region`, `loop_latch`, `exit_region`, `skip_closure`, `after_expr`, oraz nadpisywalne `for_loop`, `while_loop`, `if_expr`. Domyślna ścieżka instrukcji nie obsługuje bloku i pętli — te idą przez region. Jeśli ktoś wywoła je jak zwykłą instrukcję, dostanie `unreachable!`. To jest `assert` na programiście kompilatora, nie diagnostyka użytkownika.

`short_circuit_logical_operands` pozwala emisji `&&` / `||` nie schodzić w prawy operand zwykłą ścieżką, bo prawa strona jest w osobnym bloku podstawowym. Stempel i tak musi wiedzieć, czy prawa strona alokuje. Stąd poprawka, która dla stempla ogląda prawą stronę, a dla emisji nie.

Kto dostaje `bork_arena_push`

`codegen_push_for_region`:

- etykieta domknięcia → nigdy,
- w przeciwnym razie `block_may_allocate_sink` na obranym ciecie.

Predykat jest prawdziwy, gdy w bloku jest literał napisu albo tablicy, `move/promote` typu z areną, `concat`, przypisanie lub `return` czegoś, co może alokować, zagnieżdżony blok albo gałąź, która może alokować. Fałszywy dla samych liczb, `break`, `continue`, `None`.

`stamp_codegen_push` ustawia korzeniowi funkcji `codegen_push` = `true` zawsze. Dziecku ustawia wynik predykatu. Domknięciu `trailing` ustawia `false` jawnie.

Skutek, opisany w `docs/memory-model.md` i zrealizowany w `src/codegen/regions.rs`: `dump` nadal ma węzeł na każdy region, ale LLVM woła `bork_arena_push` tylko gdy flaga jest prawdziwa. Blok, w którym są same `i32`, ma węzeł w dumpu i nie ma płyty w runtime. Funkcja zawsze pcha jedną płytę na wejściu, nawet gdy nic nie alokuje. To jest uproszczenie, nie konieczność modelu. Płyta funkcji i tak wraca na listę przy `pop`.

Pętla: jeden `enter` (jeśli flaga), na zatrasku `reset`, na końcu jeden `exit`. `TODO.md` prosi, żeby ten kontrakt nie rozjechał się między visitorami. Dziś emisja i stempel dzielą `region_walk`, więc rozjazd jest trudniejszy niż przy dwóch ręcznych pętlach. Nadal da się go zrobić, nadpisując hak i zapominając o `latch`.

Podsumowanie

- Hoist, stempel i escape biegną tylko po czystym typecku i semie.
- Hoist rozpoznaje dokładnie dwie sąsiednie instrukcje.

- Escape liczy głębokość bajtów i zabrania return świeżego napisu, nawet z głębokości 0, gdy formą jest concat albo move.
- region_walk jest wspólnym spacerem stempla i LLVM. HIR i raport muszą mieć tyle samo dzieci.
- codegen_push odcina puste regiony od bork_arena_push. Domknięcia nie pchają nigdy, i tak nie dochodzą do emisji.

Rozdział 17. Bramka, LLVM i runtime

Ten rozdział obejmuje

- Co gate puszcza
- Kształt modułu LLVM i nazwy symboli
- Emisję funkcji, wyrażeń, aren i tablic
- C ABI bork_runtime i linkowanie
- Dziury, które kończą się paniką albo `not supported yet`

Bramka

`src/codegen/gate.rs` schodzi po HIR i zbiera `Phase::Codegen`. Nie próbuje obniżyć „trochę”. Pierwsze trafienie zostaje diagnostyką, reszta konstrukcji i tak jest odwiedzana, więc jeden plik może dostać kilka `not supported`.

Puszczane bez własnego komunikatu bramki: literały, identyfikatory, tablice, indeks, wycinek, arytmetyka całkowita i porównania, `&& || .. !`, wywołanie bez trailing closure, `if`, pole `length` na typie `uses_arena_storage`.

Odrzucane teksty są w rozdziale 10. Bramka nie wie o panice `into_int_value`. Program z argumentem `String` **przechodzi bramkę** i wywraca się później, w emisji. To jest luka testu, o którą prosi `TOD0.md`: trzymać bramkę zsynchronizowaną z tym, co emisja umie, albo mieć test, że każda próbka `check-ok` albo przechodzi bramkę, albo jest na liście świadomych odmów. Tej listy dla „panic na `String`” nie ma.

Moduł

`emit_module (src/codegen/llvm/mod.rs):`

1. Wymaga funkcji o nazwie `main`.
2. `Codegen::new(context, "bork")` — moduł LLVM i builder.
3. `declare_function` dla każdej funkcji HIR, zanim którekolwiek ciało. Dzięki temuwołanie wstecz ma symbol.
4. `RegionEmitter` trzyma raport i `ArenaCalls`.
5. `emit_function` dla każdej funkcji.
6. `regions.finish()`, `module.verify()`.

Symbol `main` jest C `main`, typ LLVM `i32` (), bez parametrów. Wynik `unit` i tak zwraca stałe 0. Inna funkcja nazywa się `bork.{nazwa}` i ma `linkage internal`. Nie jest eksportowana.

`main` zwracające coś poza `i32` i `unit` odpada komunikatem ``main` returning `...` is not supported by codegen yet`. Sprawdzono dla `i64`, `f64` i `bool`.

Wartości

Prymityw całkowity jest wartością LLVM o odpowiadającej szerokości. `bool` jest `intem`. `Float`, tam gdzie emisja w ogóle dojdzie do instrukcji arytmetycznej sprzed walka, używa `build_float_*`. Ścieżka „after walk” dla dodawania `f32` zwraca błąd „float binary after walk”. Porównanie `f64`, które wpadło w `value_as_int`, panikuje. `i64` w funkcji pomocniczej i porównanie z progiem **działa**: listing z `sub(a: i64, b: i64)` i `if (sub(big, 50) > 40)` daje kod wyjścia 7. Ograniczenie floatów nie jest ograniczeniem szerokich intów.

`String` i tablica to { `ptr`, `i64` }. Literał napisu jest prywatnym globalem, a w wartości łąduje stały deskryptor. Długość pola w języku bywa `i32` (`length`); w deskryptorze jest `i64`.

Lokalny slot to `alloca` w bloku wejścia funkcji. Dla typu z areną slot pamięta `home_arena`: identyfikator regionu, w którym bajty powinny żyć. `alloc_sink` na `FnEmitter` jest ustawiany na czas emisji **wyniku** przypisania, `return` i `concat`, a czyszczony na czas argumentów. To jest realizacja reguły „tylko wynik, nie podwyrażenia” z rozdziału 9.

`move` i `promote` deskryptora wołają `copy_into_arena` albo `copy_array_into_arena`: alokacja w puli celu i `memcpy`. `Hoist` sprawia, że kopii czasem nie trzeba, bo bajty już tam powstały. `Elizji` `memcpy` jako osobnego passu nie ma (TODO w modelu pamięci: „`memcpy` elision”).

Areny w LLVM

`ArenaCalls` implementuje zlew regionów:

- `push` → `bork_arena_push`, uchwyt na stos,
- `reset` → `bork_arena_reset`,
- `pop` → `bork_arena_pop`,
- na martwym punkcie wstawienia (po `return`) `reset` i `pop` są pomijane.

`return` robi `unwind`: zdejmuję uchwyty, które funkcja jeszcze trzyma. Seria błędów naprawiana w historii (`skip region exit after return`, `sync region stack`, `clear arena handles`) dotyczyła podwójnego `pop` albo zostawienia uchwytu. Testy `codegen` liczą pary `push/pop`. Gdy ruszasz `return`, odpal te testy, nie tylko binarkę.

Alokacja użytkowa to `bork_arena_alloc(arena, size, align)`. Wyrównanie musi być potęgą dwójki. Runtime panikuje, gdy `end > 4096`.

Wyrażenia, które emisja zna

src/codegen/llvm/expr.rs i array/emit.rs. Skrót zachowań sprawdzonych binarką:

- arytmetyka i32 i i64, porównania intów, dzielenie ze strażnikiem abort,
- if jako PHI albo jako sterowanie ze skokiem,
- for i while z break / continue,
- && || ze zwieraniem,
- println / print liczb i literałów oraz lokalnych napisów,
- concat dwóch napisów, które da się załadować jako struktury (w main),
- literał tablicy, indeks ze strażnikiem, wycinek jako GEP plus nowy deskryptor, .length,
- przypisanie elementu, w tym String przez move w main.

concat alokuje jeden bufor w sinku i kopiuje dwa argumenty. Nie ma osobnej areny tymczasowej wołania.

Wywołanie funkcji użytkownika idzie albo przez emit_call (liczy argumenty emit_value), albo przez emit_call_with_values po spacerze. Ta druga ścieżka woła coerce_value_to_ty. Dla bool zwraca do inta. Dla floata zwraca błąd „float call argument after walk”. Dla reszty woła value_as_int, które robi into_int_value() **bez** sprawdzenia wariantu. Struktura String panikuje. To jest linia 901 w src/codegen/llvm/expr.rs w tej rewizji. Nie cytuję śladu Inkwella jako kontraktu. Cytuję go jako objaw.

Wywołanie pośrednie (callee nie jest identyfikatorem) daje diagnostykę indirect calls is not supported by codegen yet, o ile ścieżka w ogóle tam wejdzie. Bramka trailing closure zwykle odpada wcześniej.

Runtime

crates/bork_runtime/src/lib.rs. Pula to Mutex<ArenaPool> w OnceLock. acquire zdejmuję Box<Arena> z wektora albo alokuje nowy. release robi reset i odkłada. Zagnieżdżone żywe regiony mają różne płyty. Sąsiednie w czasie mogą dostać tę samą.

API extern "C":

Symbol	Rola
bork_arena_push	*mut c_void, płyta z puli
bork_arena_reset	offset = 0, płyta zostaje
bork_arena_pop	zwrot płyty do puli
bork_arena_alloc	bump, wynik *mut c_void
bork_print_i64 / bork_println_i64	stdout, flush
bork_print_str / bork_println_str	ptr + len

Nie ma symbolu bork_abort. Codegen deklaruje libc abort.

Testy runtime sprawdzają wyrównanie i przepełnienie. Są w tym samym pliku, pod cfg(test), i wchodzą w cargo test --workspace nawet bez feature codegen kompilatora. Sam archiwum do linkowania powstaje dopiero przy codegen, w build.rs.

Linkowanie

src/codegen/link.rs. Polecenie to w skrócie:

```
clang -o wyjście obiekt.o libbork_runtime.a -lgcc_s -lutil -lrt -lpthread -lm  
↪ -ldl
```

Ścieżka archiwum: zmienna środowiskowa BORK_RUNTIME_LIB albo wartość wkompiłowana `env!("BORK_RUNTIME_LIB")`. `build.rs` ustawia `rpath $ORIGIN` na binarce `bork`, żeby `libLLVM` mogło leżeć obok niej. `scripts/bundle-llvm.sh` kopiuje `SONAME libLLVM` używany przez `bork`. Maszyna, która tylko **uruchamia** skompilowany program `Bork`, nie potrzebuje LLVM. Maszyna, która uruchamia `bork build`, potrzebuje `clang` i, jeśli nie spakowano `libLLVM`, instalacji LLVM 23.

Weryfikacja, którą ta książka zrobiła

`bork` złożony z `--features codegen` (LLVM 23.1.2) zbudował i uruchomił między innymi: zwrot 42 z `add`, sumę pętli 10, `while/break` z wynikiem 3, `continue` z sumą 8, `println("hi")`, `Shared x`, `move` napisu `ab`, `concat` dający LR, `promote` dający temp, przypisanie `"b"`, `escape \n i \"`, dzielenie 8/2 z kodem 4, tablicę z wycinkiem i kodem 12, wycinek drukujący 20 i 2, indeks poza zakresem oraz 1/0 jako abort, brak `main` jako diagnostykę. Tych wyników nie trzeba brać na wiarę: `docs/book/przykłady/WYNIKI.md` je powtarza.

Podsumowanie

- Bramka odcina nullable, `?:`, `!!` i trailing closures komunikatem.
- Nie odcina `String` jako argumentu funkcji. Emisja wtedy panikuje.
- `main` to C `main` zwracające `i32`. Inne funkcje to `bork.nazwa`.
- Areny w LLVM to stos uchwytów do `bork_runtime`. Pętla resetuje, nie zdejmuje płyty co obrót.
- Napis i tablica to `{ ptr, i64 }`. Literał napisu leży w globalu.
- Linkuje `clang` z statycznym runtime. Kompilator sam ładuje `libLLVM-23`.

Rozdział 18. Diagnostyki, CLI i serwer LSP

Ten rozdział obejmuje

- Jedną strukturę Diagnostic dla CLI i LSP
- Zachowanie binarki bork
- Protokół, który bork-lsp naprawdę ogłasza
- Rozszerzenie Cursora / VS Code
- Czego hover nie pokazuje

Diagnostic

src/diag.rs. Cztery fazy, dwa poziomy (Error, Warning), wiadomość, opcjonalny Span. Nie ma kodów numerycznych, odpowiedzi help: ani notatek note:. Jedna linia, jedna myśl. Gdy kompilator chce powiedzieć dwie rzeczy, dostajesz dwie diagnostyki. Listing 2.4 (pętla i move) jest takim przypadkiem.

Mapowania:

Funkcja	Faza
from_parse	parse
from_sema	ownership
błędy typeck (budowane w Env::error)	type
escape	ownership
gate i codegen_error	codegen

LSP nie uruchamia bork build, więc fazy codegen w edytorze nie zobaczysz, dopóki ktoś nie wpieje bramki w frontend::check. Dziś bramka jest tylko w codegen::build. Program z None w edytorze jest „czysty”, a bork build go odrzuca. To jest ważne dla czytelnika, który ufa zielonej lampce.

CLI

src/main.rs. Dwie formy:

```
bork [--dump-arenas] <plik>
bork build [-o ścieżka] <plik>
```

--dump-arenas drukuje na stdout tylko gdy report jest Some. Przy błędzie parse jest cicho (poza stderr). Przy błędzie typu drzewo jest. --help kończy się kodem 0. Nieznany znacznik kończy się kodem 2.

build bez feature codegen nie próbuje LLVM. Mówi, że feature jest wymagany. Z feature: check, potem build, potem albo sukces (kod 0, nawet gdy programem jest println i nic nie zwraca), albo diagnostyki (1), albo toolchain (2).

Nakładanie ścieżek jest sprawdzane najpierw porównaniem ścieżek, potem canonicalize, gdy oba pliki istnieją. Nie jest to pełne rozwiązanie linków symbolicznych w każdą stronę. Wystarczy, żeby nie nadpisać źródła domyślnym wyjściem plik z plik.bork tylko wtedy, gdy to naprawdę ten sam plik. Domyślne wyjście ma obcięte rozszerzenie, więc a.bork → a. Konflikt trzeba wymusić ręcznie (-o a.bork przy wejściu a.bork).

Serwer

src/bin/bork_lsp.rs i src/lsp.rs. Feature lsp. Transport stdio, biblioteka tower-lsp.

Ogłaszane możliwości:

- synchronizacja pełnym tekstem dokumentu (FULL),
- hover,
- komenda bork.dumpArenas.

Nie ma uzupełniania, definicji, referencji, formatowania, symboli ani code action. didChange bierze ostatnią zmianę z powiadomienia i liczy analizę od zera. Nie ma inkrementalnego parsera.

Analiza to lsp::analyzer::source → frontend::check. Ten sam potok co CLI. Specyfikacja rozszerzenia z 21 września 2026 opisywała diagnostyki samego parsera. To jest nieaktualne. README w jednym nagłówku nadal mówi „Parse diagnostics”, a trzy zdania niżej opisuje pełny check. Wierz analyzer::source, nie nagłówkowi.

Diagnostyka LSP ma source: "bork" i treść "{faza}: {wiadomość}". Pozycje są UTF-16, zgodnie z protokołem. Pusty span (start == koniec) jest rozszerzany o jedną jednostkę, żeby podkreślenie było widoczne.

Epoka na URI odrzuca spóźnione publishDiagnostics, gdy użytkownik zdążył wpisać kolejną wersję. To jest jedyna współbieżność, o którą serwer dba. Nie ma puli procesów roboczych.

Hover i dump

Hover szuka słowa pod kursorem (ASCII alfanumeryczne i _) i trafienia w span wiązania w drzewie aren. Tekst markdown:

```
`nazwa` in arena `etykieta`
Ownership: ...
```

Nie ma typu HIR. Specyfikacja zostawiła typ na hoverze jako follow-up. Commit, który podpinał LSP do pełnego checku, też traktował typ jako opcjonalny. Nadal go nie ma.

Komenda `bork.dumpArenas` bierze opcjonalne URI albo pierwszy otwarty dokument. Wynik to JSON `{ "dump": "..."} plus window/logMessage`. Przy błędzie parse JSON ma `{ "error": "..."} Dump przy błędach semantycznych dopisuje linie # semantic errors`. To ten sam ASCII co `--dump-arenas`, plus ten dopisek.

Rozszerzenie

`tools/bork-lsp-extension`. Aktywacja na języku bork, rozszerzenie pliku `.bork`. Gramatyka TextMate w `syntaxes/bork.tmLanguage.json`. Konfiguracja nawiasów i komentarza `//` w `language-configuration.json`.

`server.js` uruchamia `target/debug/bork-lsp` z katalogu workspace, a gdy binarki nie ma, `cargo run --quiet --bin bork-lsp`. Bez otwartego folderu rozszerzenie nie startuje serwera. Komenda UI nazywa się `bork.dumpArenasView` i woła komendę serwera `bork.dumpArenas`. Test `test/server.test.js` pilnuje tej różnicy nazw oraz wyboru binarki.

Nie ma ustawień w `package.json`. Nie ma adaptera debuggera. Podświetlanie nie wie o błędach typów; od tego są diagnostyki LSP.

Podsumowanie

- Jedna struktura Diagnostic zasila CLI i LSP. Codegen do LSP nie dochodzi.
- Zielony plik w edytorze może nadal paść na `bork build` (bramka albo panic).
- Hover pokazuje arenę i własność, nie typ.
- Dump w edytorze i `--dump-arenas` dzielą `dump::dump_arenas`.
- Serwer liczy check od zera przy każdej zmianie i wyrzuca spóźnione odpowiedzi epoką URI.

Rozdział 19. Mapa repozytorium

Ten rozdział obejmuje

- Drzewo katalogów, które ma znaczenie
- Odpowiedzialność modułów jednym zdaniem
- Historię, która tłumaczy, dlaczego kod leży warstwami
- Kolejność czytania, gdy chcesz zrozumieć całość
- Pliki, które wyglądają na dokumentację, a są z innego etapu

Drzewo

Cargo.toml	workspace: bork + crates/bork_runtime
build.rs	lalrpop; przy codegen także staticlib runtime
src/lib.rs	parse(), reeksporte, próbki MVP i PROCESS_USER
src/main.rs	CLI check i build
src/bin/bork_lsp.rs	binarka serwera
src/parser.lalrpop	gramatyka
src/ast.rs	AST
src/layout.rs	nowe linie w nawiasach
src/span.rs	Span, SpannedName
src/diag.rs	Diagnostic
src/frontend.rs	check()
src/typeck/	AST → HIR
src/hir/	Hir* i Ty
src/sema/	ArenaReport, walk, policy
src/escape.rs	głębokość bajtów (prywatny moduł)
src/hoist.rs	alloc_in_binding
src/region_walk.rs	wspólny spacer (prywatny)
src/dump.rs	ASCII aren
src/builtins.rs	print, println, concat
src/arena.rs	model 4 KiB, nie faza potoku
src/lsp.rs	adapter protokołu
src/codegen/	bramka, regiony, link, llvm/
crates/bork_runtime/	C ABI płyt i druku
tests/parser.rs	parse
tests/arrays.rs	tablice przez check
tests/build.rs	bork build i uruchomienie binarki
tools/bork-lsp-extension/	klient VS Code / Cursor

docs/language.md	opis powierzchni
docs/memory-model.md	model pamięci
docs/superpowers/	specyfikacje i plany, część nieaktualna
TOD0.md	dług po tablicach i region_walk
scripts/bundle-llvm.sh	kopiowanie libLLVM obok bork

Moduły prywatne (escape, region_walk, layout, builtins) są widoczne w crate, nie w API bork::*. Z zewnątrz wchodzisz przez parse, frontend, sema, hir, typeck, dump, diag, lsp, codegen, arena, span, ast.

Jedno zdanie na moduł

Moduł	Zdanie
ast	Nietypowane drzewo po parserze
layout	Zamiana \n na \r w nawiasach, bez ruszania długości
span	Para bajtów
diag	Faza, treść, span
frontend	Skleja fazy w CheckResult
typeck	Produkuje HIR i błędy typu
hir	Typowane drzewo bez identyfikatorów regionów
sema	Drzewo aren i błędy nazw
escape	Odrzuca bajty, które przeżyłyby swoją płytę
hoist	Dwie sąsiednie linie → alokacja u celu
region_walk	Jeden spacer HIR z raportem
dump	Drzewo na tekst
builtins	Trzy intrinsics
arena	Bump 4096 i pula, referencyjny model
codegen	Bramka, obiekt LLVM, clang
codegen::llvm	Inkwell: funkcje, wyrażenia, tablice, uchwytty aren
lsp	Pozycje UTF-16, hover, dump
bork_runtime	Płyty i druk dla zlinkowanego programu

Jak repo rośło

Kolejność na main, skrócona do rzeczy, które widać w kodzie, nie do listy PR:

1. Parser LALRPOP i AST, próbka z trailing closure (MVP_SAMPLE).
2. LSP najpierw od samego parse. Dziś serwer woła pełny check.
3. Składnia nullable, napisów, Some/None.
4. Sema regionów, --dump-arenas, hover własności.
5. Wyrażeniowy move i val/var na parametrach.
6. Typowany HIR i frontend::check (Phase w diagnostyce).
7. LLVM, bork build, runtime, bramka.
8. Sink, hoist, promote, concat, escape.
9. Tablice [T; N], wycinki, while/break/continue, &&/||, wspólny region_walk.
10. Pin LLVM 23 i skrypty pakowania libLLVM.

Specyfikacje w docs/superpowers/specs/ opisują kroki 1, 4, 5, 6 i projekt LSP. Nie opisują tablic ani while. TODO.md jest świeższy niż te specyfikacje i mówi wprost, żeby plany w docs/superpowers/plans/ przenieść albo usunąć, gdy zostaną wchłonięte. Czytaj je jako historię decyzji, nie jako kontrakt. Rozjazdy są w dodatku C.

Kolejność czytania

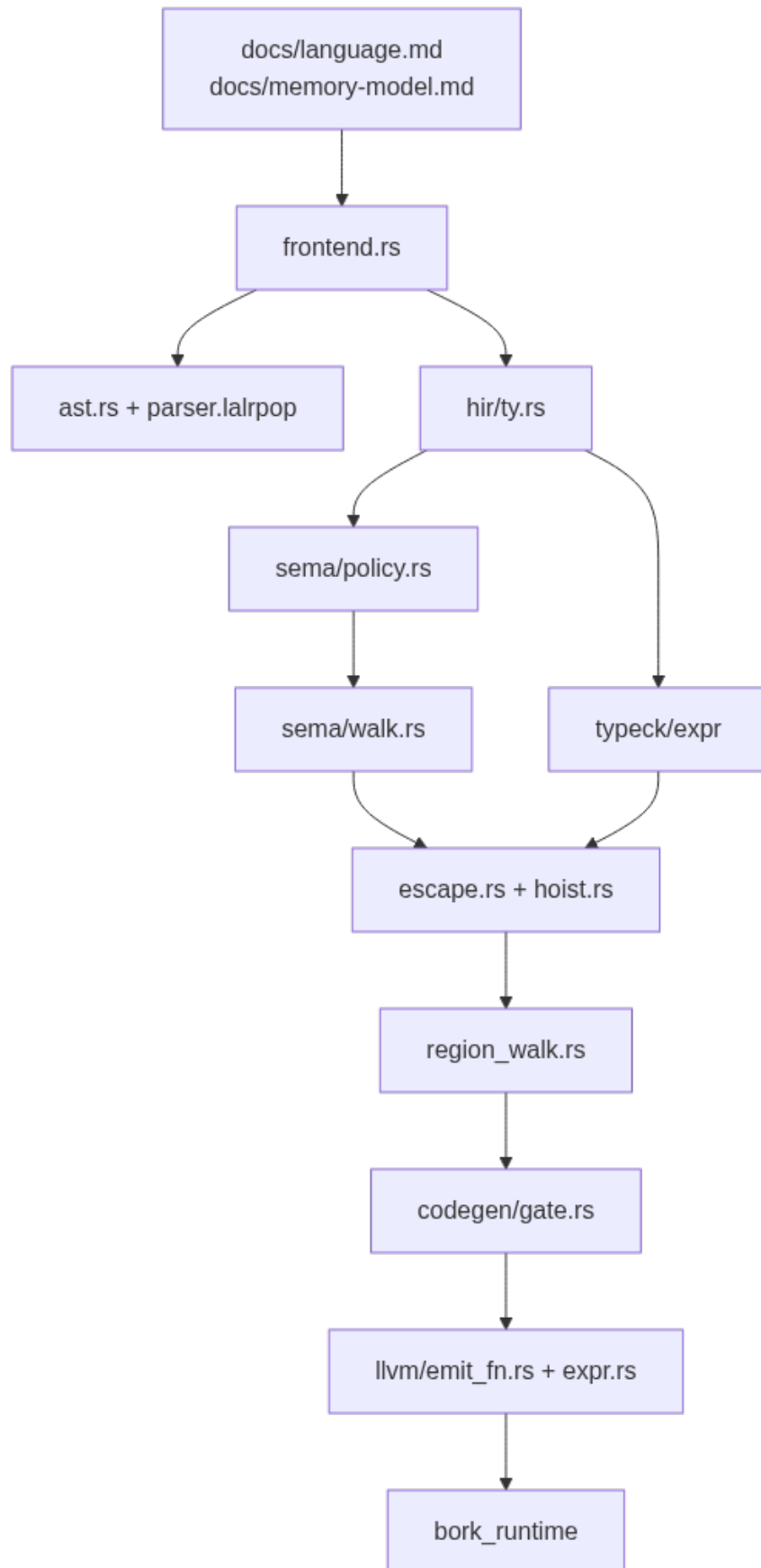
Gdy celem jest „rozumiem, co się dzieje z plikiem .bork”:

1. docs/language.md i docs/memory-model.md — powierzchnia i model. Potem ta książka tam, gdzie dokument się rozjeżdża (concat na var).
2. src/lib.rs — parse i dwie próbki. Próbką MVP to test „parser i sema dają radę”, nie test codegen.
3. src/frontend.rs — cały potok w 60 liniach.
4. src/ast.rs równolegle z początkiem parser.lalrpop (produkcje Program, Stmt, Expr). Nie czytaj 400 linii gramatyki ciągiem, dopóki nie znasz enumów.
5. src/hir/ty.rs — is_copy i uses_arena_storage. To dwa predykaty, do których wraca reszta.
6. src/sema/policy.rs — classify_use. Potem sema/walk.rs tylko wokół open_ordinary, Assign i If.
7. src/typeck/expr/control.rs i expr/call.rs — miejsca, gdzie język jest najbardziej „kotlinowy”.
8. src/escape.rs funkcja place i src/hoist.rs funkcja try_hoist. Są krótkie.
9. src/region_walk.rs od RegionVisitor i stamp_codegen_push, nie od makr kursora. Makra arena_cursor są mechaniką pożyczania. TODO.md sam proponuje je kiedyś zastąpić typem kursora.
10. src/codegen/gate.rs, potem codegen/mod.rs (build), potem llvm/mod.rs (emit_module), potem llvm/emit_fn.rs i llvm/expr.rs. array/emit.rs na końcu, gdy wiesz, jak wygląda deskryptor.
11. crates/bork_runtime/src/lib.rs — 150 linii, cały runtime.
12. tests/build.rs — co naprawdę zwraca proces.

Gdy celem jest tylko język, zatrzymaj się po punkcie 7 i czytaj testy sema oraz typeck zamiast LLVM.

Pułapki nawigacji

- src/arena.rs i ArenaNode to nie jest ta sama arena. Pierwsza ma offset i 4096 bajtów. Druga ma dzieci i wiązania.
- sema::analyze i frontend::check nie są wymienne. Pierwsza liczy typeck wewnętrznie. Druga podaje decl_tys.
- UseKind w HIR i Ownership w raporcie mają podobne nazwy (Copy, Shared, Move) i nie są tym samym typem.
- Feature codegen wyłącza pół plików z kompilacji. Ostrzeżenia dead_code na region_walk bez tego feature są oczekiwane: spacer jest wtedy używany tylko przez stempel, a część metod tylko przez LLVM.
- docs/superpowers/plans/ potrafi opisywać LLVM 18. Cargo.toml pinuje Inkwell z feature llvm23-1-force-dynamic. Wierz manifeście.



Rysunek 3: Diagram 3

Podsumowanie

- Potok mieści się w `frontend.rs`. Reszta katalogu `src/` to fazy.
- Czytaj `policy.rs` przed `walk.rs`, a `ty.rs` przed jednym i drugim.
- Specyfikacje w `docs/superpowers` są historią. `TODO.md` i kod są stanem.
- `arena.rs` nie stoi na ścieżce checku.
- Testy `tests/build.rs` są definicją tego, co binarka robi, nie komentarze w LLVM.

Rozdział 20. Jeden program przez cały kompilator

Ten rozdział obejmuje

- Program śledzony, uruchomiony od źródła do kodu wyjścia
- Co widzi parser i jaki jest AST w zarysie
- Co typeck dopisuje do HIR
- Jakie drzewo aren wypisuje sema
- Dlaczego hoist i escape milczą
- Które funkcje LLVM i runtime naprawdę pracują

Program jest krótki celowo. Ma funkcję, pętlę, wywołanie, napis `val` i blok, który ten napis tylko czyta. Wszystko to bork `build` obniża. Nie ma w nim `String` przekazywanego do funkcji użytkownika, bo ta ścieżka wywraca kompilator i ślad by się urwał paniką.

Program

Listing 20.1. Uruchomione. Stdout `sum\n`. Kod wyjścia 3, bo `0+1+2`.

```
fun add(a: i32, b: i32): i32 {  
    return a + b  
}  
  
fun main(): i32 {  
    var total = 0  
    for (i in 0..3) {  
        total = add(total, i)  
    }  
    val label = "sum"  
    {  
        println(label)  
    }  
    return total  
}
```

Zakres `0..3` jest prawostronnie otwarty. Trzy iteracje, nie cztery.

Krok 1. Layout i parse

W pliku nie ma nawiasu, który łamałby linię, więc `normalize_parenthesized_newlines` zwraca `None` i parser dostaje oryginał. `ProgramParser` buduje dwie `Function`.

Dla `add` AST ma dwa `Param` z `BindingKind::Val` (nie napisano `val`, a goła forma jest `val`), typ `i32`, ciało z jednym `Return` i `Binary Add`.

Dla `main`: `VarDecl total` bez adnotacji, `For` z `Binary RangeTo`, `VarDecl label`, `Stmt::Block` z `Expr` wywołania `println`, `Return`.

Gdyby tu był średnik, `parse` zwróciłby `UnrecognizedToken` i `check` skończyłby się jednym `Diagnostic` fazy `parse`, bez raportu. Nie ma.

Wejście: `bork::parse` w `src/lib.rs`. Gramatyka: produkcje `Function`, `Stmt`, `ExprRange`, `Atom`.

Krok 2. Typeck

`typeck::check` widzi dwie sygnatury: `add (i32, i32) -> i32`, `main -> i32`. `println` jest wbudowane, nie koliduje.

`total` bez adnotacji, inicjalizator `0` bez oczekiwanego typu poza tym, że literał całkowity domyślnie jest `i32`. Do `decl_tys` wpada `i32`. `label` to `String`. `i` w `for` dostaje `i32`, bo granice zakresu są `i32`. Warunek nie jest osobnym wyrażeniem `bool`; zakres jest osobnym typem `Range`.

Wywołanie `add(total, i)`: `callee` jest nazwą, arność 2, oba argumenty `i32`. `UseKind` odczytu `total` w ciele pętli wychodzi `Copy` (inny region, typ `Copy`). `println(label)`: builtin, argument `String`, w HIR `UseKind` odczytu `label` to `Shared`.

`return total` zgadza się z `i32`. Ścieżka funkcji zawsze wraca. HIR `add` to jeden `HirStmt::Return` z `Binary`.

Pliki: `src/typeck/mod.rs`, `stmt.rs`, `expr/call.rs`, `expr/binary.rs`.

Krok 3. Sema

`analyze_with_decl_tys` dostaje kolejkę typów deklaracji: `total`, potem `label`. `fun_sigs` pamięta, że oba parametry `add` są `Val`.

Korzeń `fun add` wiąże `a` i `b` jako `Local`. W ciele nie ma odczytu spoza regionu, który warto obserwować poza parametrami. `Dump` nie pokazuje osobnych obserwacji `Copy` dla parametrów użytych w tym samym regionie: `classify_use` zwraca `Local`, a `dump` lokalne obserwacje przy deklaracji pomija.

Korzeń `fun main`:

Arenas

```
├─ fun add
│   ├── a [Local]
│   └── b [Local]
└─ fun main
```

```

├─ total [Local]
├─ label [Local]
├─ ForLoop (i)
│   ├── i [Local]
│   └─ total [Copy]
└─ Block
    └─ label [Shared ← fun main]

```

To jest wyjście `--dump-arenas` z uruchomienia, nie rekonstrukcja. `ForLoop` powstał w `walk` na `Stmt::For`. `i` jest `RegionParam`. `total` w ciele przechodzi `classify_use`: inna arena, `is_copy` $\rightarrow$ `Copy`. Przypisanie `total = ...` jest `assign-up` (cel `Var`, `arena_id` mniejszy), więc lewa strona nie jest błędem „użyłeś `var` rodzica”. Prawa strona to `i32`, więc i tak `Copy`.

Blok po pętli: `open_ordinary` z etykietą `Block`. `label` jest `Val` i nie-`Copy`, inna arena $\rightarrow$ `Shared { from: "fun main" }`.

`loop_move_ban` na czas `for` zawiera `total`. Nikt nie woła `move`, więc zakaz milczy. `peel_blocks` nic nie skleja: żaden blok nie jest gołym opakowaniem.

Pliki: `src/sema/analize.rs`, `walk.rs`, `policy.rs` (`classify_use`), `region.rs` (`RegionFrame`), `dump.rs`.

Krok 4. Stempel, hoist, escape

Diagnostyk nie ma, więc `stamp_codegen_push` schodzi `region_walk`. Korzeń każdej funkcji dostaje `codegen_push = true`.

Ciało pętli: przypisanie `i32` i wywołanie `add` nie alokują sinku napisowego. Predykat `block_may_allocate_sink` dla ciała złożonego z samej arytmetyki jest fałszywy. Węzeł `ForLoop` zostaje w raporcie, ale **nie** prosi o `bork_arena_push`. To jest ten przypadek z modelu pamięci: „węzeł w dumpu, brak płyty”.

Blok z `println(label)` nie tworzy nowego napisu. Czyta `Shared`. Predykat alokacji sinku na samym wywołaniu `println` nie jest `concat` ani literałem. Literał `"sum"` stoi przy deklaracji `label` w ciele funkcji, nie w bloku. Blok wewnętrzny też nie musi pchać własnej płyty.

Hoist: nigdzie nie ma pary `var inner / outer = move inner`. `alloc_in_binding` zostaje puste.

Escape: `return total` nie jest typem z areną. `label` nie wraca z wewnętrznego regionu. `place` nie zgłasza nic.

HIR zostaje w `CheckResult`.

Pliki: `src/region_walk.rs` (`stamp_codegen_push`, `block_may_allocate_sink`), `src/hoist.rs`, `src/escape.rs`.

Krok 5. Bramka i moduł

`gate` nie znajduje trailing closure, `None`, `Some`, `?:`, `!!` ani obcego pola. Przechodzi.

emit_module deklaruje main jako i32 () oraz bork.add jako funkcję wewnętrzną. Potem emituje ciało.

emit_function dla main otwiera blok wejścia, alloca na total i na deskryptor label. Region funkcji pcha płytę (codegen_push korzenia). Literał "sum" jest globalem; deskryptor { ptr, len: 3 } łąduje w slocie. Płyta funkcji w tym programie może zostać nietknięta przez alloc, bo znaki są w globalu. Push i tak jest.

Pętla: indeks w rejestrze albo w alloca, porównanie z 3, ciało woła bork.add, dodawanie jest build_int_add po stronie add i przypisanie w main. Zatrask nie musi resetować płyty, jeśli enter pętli w ogóle nie wypchnął uchwytu. Spacer i tak woła loop_latch; ArenaCalls na braku uchwytu regionu nie resetuje cudzej płyty.

println obniża się do bork_printf_str. Po bloku, jeśli nie było pusha, nie ma popa. return total łąduje i32 i zwraca. unwind zdejmuję uchwyt funkcji.

add to load dwóch parametrów, build_int_add, ret. Bez napisów.

Pliki: src/codegen/gate.rs, src/codegen/llvm/mod.rs, src/codegen/llvm/emit_fn.rs, src/codegen/llvm/expr.rs, src/codegen/llvm/arena.rs, src/codegen/regions.rs.

Krok 6. Obiekt, link, proces

write_object bierze natywny target (na maszynie, na której książka była sprawdzana: Linux x86-64), ustawia triple i data layout, pisze plik obiektowy. link_executable woła clang z libbork_runtime.a.

Proces main zwraca 3. println pisze trzy bajty sum i nową linię, potem flush. Pula aren w runtime żyje w statycznym Mutex. Po pop płyta funkcji wraca na listę. Proces się kończy, więc pula znika z pamięcią procesu.

Czego ten ślad nie pokrywa

Nie było hoista, promote, move napisu, wycinka, while, zwierania ani błędu escape. Każde z nich ma listing w części I i test w tests/build.rs albo w src/escape.rs. Ślad miał pokazać **szczęśliwą ścieżkę przez wszystkie funkcje graniczne**, nie każdy hak.

Gdy będziesz powtarzał ślad na programie z var piece = concat(...) i out = move piece, zatrzymaj się dłużej w kroku 4: hoist::annotate ustawi alloc_in_binding, a w kroku 5 emisja deklaracji ustawi alloc_sink na dom out zanim wyemituje concat. Reszta kroków jest ta sama.

Podsumowanie

- Śledzony program zwraca 3 i drukuje sum. Checker i bork build są zgodne.
- Parser widzi dwie funkcje i zakres ... Typeck widzi i32 i String.
- Sema oznacza total jako Copy w pętli i label jako Shared w bloku.
- Pętla bez alokacji nie dostaje własnej płyty, choć ma węzeł w dumpu.
- Hoist i escape nie mają tu nic do roboty. To też jest wynik, nie pominięcie.

- LLVM woła `bork.add` i `bork_println_str`. Runtime dostaje deskryptor literału z globala.

Rozdział 21. Testy, debugowanie i nowa funkcja języka

Ten rozdział obejmuje

- Którą komendą odpalić który zestaw
- Gdzie szukać, gdy binarka kłamie, a checker milczy
- Ścieżkę dodania konstrukcji, na przykładzie `while`
- Dług z `TOD0.md`, którego nie trzeba odkrywać drugi raz
- Czego nie robić przy pierwszym patchu

Testy

Z README i z `.github/workflows/ci.yml`:

```
cargo test --workspace
cargo test --workspace --features codegen
```

Pierwsza komenda **nie** składa LLVM. Łapie parser, semę, typeck, escape przez `frontend::check`, LSP, runtime (`bork_runtime` ma własne testy jednostkowe puli) i bramkę o tyle, o ile test bramki jest za `cfg(feature = "codegen")`. Job CI `codegen` jest osobny: instaluje LLVM 23 i clang, puszcza workspace z feature i jeszcze smoke spakowanego `libLLVM`.

Test	Co utwierdza
<code>tests/parser.rs</code>	tokens, NL, move, pierwszeństwo, spany błędów
<code>src/sema/tests/*</code>	własność nazw
<code>src/typeck/tests/*</code>	typy, często przez pełny check
<code>tests/arrays.rs</code>	tablice bez uruchamiania binarki
<code>tests/build.rs</code>	<code>bork build</code> i kod wyjścia albo <code>stdout</code>
<code>src/codegen/gate.rs</code>	teksty <code>not supported</code>
<code>src/lsp.rs</code>	pozycje, hover, dump
<code>crates/bork_runtime</code>	<code>bump</code> , <code>align</code> , <code>overflow</code>
<code>tools/bork-lsp-</code>	ścieżka binarki kontra <code>cargo run</code>
<code>extension/test/server.test.js</code>	

tests/build.rs jest za `#![cfg(feature = "codegen")]`. Bez feature plik znika z kompilacji. Nie dziw się, że cargo test „nie widzi” `builds_for_loop_sum`.

Testy build piszą źródło do `CARGO_TARGET_TMPDIR`, wołają `CARGO_BIN_EXE_bork` z `build -o`, potem uruchamiają binarkę. Gdy dodajesz zachowanie runtime, dopisuj tu oczekiwany kod albo stdout, nie tylko test jednostkowy emisji. Książka oparła listingi na tym samym kontrakcie.

Debugowanie

Checker. `cargo run --bin bork -- --dump-arenas plik.bork`. Gdy drzewo nie ma węzła, którego oczekujesz, błąd jest w semie (`walk.rs`), nie w LLVM. Gdy drzewo jest, a bork build panikuje, błąd jest w emisji albo w `coerce_value_to_ty`.

Faza. Prefiks `parse / ownership / type / codegen` mówi, którego katalogu nie czytać. `ownership` bez słowa `after move` i bez `not Copy` często jest z `escape.rs`, nie z `policy.rs`. Słowa `inner region`, `concat`, `moved` or `promoted` są z `escape`.

Rozjazd spaceru. Komunikat `internal arena schedule mismatch` znaczy: `region_walk` nie dostał dziecka `ArenaNode` albo dostał je w złym miejscu. Porównaj etykietę w semie z `RegionSite`. Najczęstsza przyczyna przy nowej konstrukcji: sema otwiera region, a visitor LLVM o nim nie wie, albo odwrotnie.

Panic into_int_value. Wartość LLVM nie jest intem. Patrz, jaki ty ma wyrażenie. `String` i `float` na ścieżce `emit_call_with_values` są znanymi ofiarami. Naprawa należy do `coerce_value_to_ty`: dopasowanie `BasicValueEnum`, a dla struktur przekazanie deskryptora bez kastu na i64. Nie maskuj tego `catch_unwind`.

Runtime abort bez diagnostyki. Indeks albo dzielenie. Odpal binarkę pod debuggerem i zobacz, czy stanąłeś w abort. Przepelnienie areny daje panic Rusta z tekstem `arena overflow`, bo runtime jest pisany w Ruście i panic nie jest łapany.

LLVM verify. `module.verify()` po emisji wywraca build, gdy IR jest zepsuty (zły typ PHI, brak ret). To lepsze niż cichy zły kod. Gdy verify pada, IR jest już zły; nie szukaj błędu w clang.

TIP

TIP. bork bez feature `codegen` jest szybki do iteracji po komunikatach. Przepinaj feature dopiero, gdy check jest czysty i chcesz zobaczyć bramkę albo proces.

Ścieżka nowej konstrukcji

Poniżej jest ścieżka, którą widać po fakcie na `while / break / continue`. Nie jest to przepis z planu, tylko kolejność warstw, które musiały ruszyć, bo każda z nich w kodzie o `while` wie.

1. **Gramatyka.** Terminal i produkcja `Stmt`. Dla `while`: warunek w nawiasach, ciało blokiem. `break` i `continue` jako instrukcje ze spanem. Test w `tests/parser.rs`, że NL działa i że słowo nie jest identyfikatorem w złym miejscu.

2. **AST.** Wariant Stmt::While, Break, Continue. Bez typów.
3. **Typeck.** Warunek oczekuje bool. loop_depth rośnie tak samo jak przy for. break poza pętlą to błąd typu. Nowy HirStmt. Test tekstu diagnostyki.
4. **Sema.** Region WhileLoop przez open_ordinary. Ten sam loop_move_ban co for. Jeśli zapomnisz zakazu, test move_outer_binding_inside_loop_errors nie pokryje while, dopóki go nie skopiujesz. Warto skopiować.
5. **HIR a peel.** Jeśli ciało jest blokiem, nie dokładaj drugiego regionu w typecku. Sema i HIR muszą mieć po jednym dziecku.
6. **Escape / hoist.** Nowa instrukcja w spacerze escape i w hoist, inaczej deklaracja w while nie będzie widziana. Dla while wystarczy zejść w ciało jak w for.
7. **region_walk.** RegionSite i hak while_loop. Stempel codegen_push zacznie widzieć alokacje w ciele. Bez tego LLVM i raport się rozjadą w chwili, gdy ciało alokuje napis.
8. **Bramka.** while nie potrzebuje odmowy, bo emisja go umie. Nowa konstrukcja, której emisja nie umie, **musi** dostać reject w gate.rs, żeby użytkownik dostał diagnostykę zamiast paniki. To jest lekcja z String jako argumentu: bramka go nie zna, emisja panikuje.
9. **Emisja.** Bloki podstawowe: nagłówek, ciało, zatrask, reset jeśli region wypchnął płytę, break jako skok do bloku wyjścia. Potem test w tests/build.rs z konkretnym kodem wyjścia. Dla while+break jest to 3.
10. **Dokument.** docs/language.md i, jeśli ruszasz pamięć, docs/memory-model.md. TODO.md prosi, żeby dokument nadażał za bramką. Ta książka jest trzecim opisem. Przy zmianie semantyki aktualizuj co najmniej language.md. Książka w docs/book opisuje jedną rewizję; nie zaktualizuje się sama.

Konstrukcja, która jest tylko cukrem składniowym, może skończyć się w parserze desugaringiem do istniejącego AST. W tym repozytorium prawie nic się tak nie dzieje. move wyrażeniowy i move regionalny są osobnymi węzłami. Nowy cukier lepiej zostawić osobnym węzłem, jeśli ma własną diagnostykę.

Dług, który już jest zapisany

Z TODO.md, skrót bez udawania, że to robimy w tej książce:

- rozciąć visitor codegenu, żeby FnEmitter nie był jednocześnie RegionVisitor (cykl codegen_walk ↔ expr),
- kursor aren zamiast makra, gdy makro dalej urośnie,
- test, że harmonogram pętli to jeden enter, wiele resetów, jeden exit,
- bramka zsynchronizowana z frontendem,
- błędy spaceru zawsze jako WalkError::from_diagnostic, bez gubienia tekstu użytkownika w opakowaniu schedule_error,
- więcej testów push/pop na zagnieżdżonym if i sinkach napisów,
- nie commitować luźnych smoke.bork w korzeniu repo.

internal arena schedule mismatch wokół floatów jest przykładem punktu drugiego i czwartego: tekst użytkownika („float binary...”) utonął w opakowaniu, a bramka w ogóle nie powiedziała „float nie jest obniżany”.

Pierwszy patch

Nie zaczynaj od LLVM. Dopisz test parsera albo typeck, który na czerwono nazywa zachowanie, i doprowadź frontend::check do tego tekstu. Bramkę dodaj w tym samym patchu, jeśli emisji nie będzie. Pusta bramka przy nowej składni jest tym, co zamienia przyszłą panikę w diagnostykę.

Nie zmieniaj peel_blocks tylko w jednym z dwóch miejsc.

Nie „naprawiaj” paniki into_int_value przez odrzucenie wszystkich String w bramce, jeśli w main napisy działają. To obciąłoby listingi, które są legalne. Wąskie miejsce to kasta argumentu wołania.

Podsumowanie

- cargo test --workspace nie uruchamia LLVM. Feature codegen uruchamia tests/build.rs.
- Dump aren debuguje semę. Panic into_int_value debuguje emisję. schedule mismatch debuguje region_walk.
- Nowa konstrukcja idzie warstwami: gramatyka, AST, typeck, sema, walk, bramka, emisja, test binarki.
- Jeśli emisji nie ma, bramka jest obowiązkowa.
- TODO.md jest listą znanych cięć. Sprawdź ją, zanim opiszesz rozjazd jako odkrycie.

Rozdział 22. Ćwiczenia

Ten rozdział obejmuje

- Ćwiczenia na checker, bez LLVM
- Ćwiczenia na bork build i kod wyjścia
- Ćwiczenia czytania kodu kompilatora
- Odpowiedzi, które da się zestawić z komunikatem albo z plikiem

Odpowiedzi są w tym samym rozdziale, pod treścią. Spróbuj najpierw odpalić kompilator. Komunikaty z części I wolno uznać za ściągę, nie za porażkę.

Część A. Checker

A1. Napisz funkcję `main`, która zwraca `i32` i nie ma ani jednego `return`. Jaki jest dokładny początek komunikatu (faza i pierwsze słowa)?

A2. Zadeklaruj `var s = "hi"` i `var t = s`. Popraw program najmniejszą zmianą tak, żeby check przeszedł, a `s` nie dało się użyć niżej.

A3. Wytlumacz, czemu ten program nie parsuje się, i podaj poprawną wersję o tym samym wyniku:

```
fun main(): i32 {  
    val n = 0 - 3  
    return n  
}
```

Ta wersja jest poprawna. Ćwiczenie: zamień `0 - 3` na `-3` i nazwij fazę błędu.

A4. `val a: [i32; 2] = [1, 2, 3]`. Ile diagnostyk dostaniesz i jakich faz? Nie poprawiaj programu, tylko je nazwij.

A5. Napisz `for` po zakresie `i64`. Jaki typ granic checker wymaga?

A6. Program z dokumentacji:

```
var left = "L"  
var right = "R"  
{  
    var piece = concat(left, right)  
    out = move piece  
}
```

dopisz brakującą deklarację `out` i spraw, żeby `check` przeszedł na dwa sposoby: przez `val` oraz przez `move`.

A7. `return concat("a", "b")` w funkcji zwracającej `String`. Czy komunikat ma numer linii?

Część B. Binarka

Uruchom bork `build` i proces.

B1. Ile wynosi kod wyjścia sumy `for (i in 0..5) total = total + i`?

B2. `while` podbija `i` od 0 i robi `break` przy `i == 3`, zanim zwiększy. Co zwraca `main`?

B3. `println("hi")` w `main` bez typu wyniku. Co jest na `stdout` i jaki jest kod wyjścia?

B4. `return 1 / 0`. Czy bork `build` kończy się błędem? Co robi proces?

B5. `val a = [10, 20, 30]` i `a[9]`. Czy `check` przechodzi?

B6. Weź listing 20.1. Zmień zakres na `0..4`. Jaki kod wyjścia przewidujesz i dlaczego, zanim uruchomisz?

B7. `fun main(): i64 { return 1 }`. Jaka faza i czy pada `checker`, czy dopiero `build`?

Część C. Kod kompilatora

C1. Wskaż plik i funkcję, które ustawiają kolejność „najpierw diagnostyki semy, potem typy”.

C2. Gdzie jest pojemność 4096? Podaj dwa pliki. Który wykonuje się w procesie użytkownika?

C3. Czemu `--dump-arenas` przy `fun oops` (nie drukuje `Arenas`?

C4. `MoveBlock.captures` bywa `None` albo `Some([])`. Które wnioskuje?

C5. Dlaczego `HIR` nie ma pola `region_id`? Gdzie `codegen` bierze `region`?

C6. Która funkcja w `sema/policy.rs` zwraca `Shared`?

C7. Co trzeba zmienić, żeby przekazanie `String` do funkcji użytkownika dało diagnostykę zamiast paniki? Nie musisz pisać patcha. Nazwij funkcję, która panikuje, i powiedz, czemu bramka tego nie łapie.

C8. Nowy operator binarny, którego LLVM jeszcze nie ma. Wymień warstwy z rozdziału 21 w kolejności i zaznacz, która jest obowiązkowa, żeby nie było paniki.

Odpowiedzi A

A1. `type: function must return a value of type i32 on all paths`. `Span` bywa pusty, więc linia może zniknąć z prefiksu.

A2. `var t = move s`. Komunikat przy gołym `s` to `use move s to transfer ownership`.

A3. `return -3` jest fazą `parse`, `unexpected token`. `0 - 3` jest odejmowaniem i dla `main()`: `i32` daje kod wyjścia, który owija się w `i32` (wartość ujemna jako kod procesu). Checker jest zadowolony.

A4. Dwie diagnostyki fazy `type`: literał ma 3 elementy, oczekiwano 2; inicjalizator ma typ `[i32; 3]`, oczekiwano `[i32; 2]`.

A5. `range bounds must have type i32`.

A6. `var out = "prefix"` na zewnątrz. Albo `val left` i `val right`, albo `concat(move left, move right)`. Wynikowy napis to `LR`, nie `prefixLR`.

A7. Nie. `Escape` stawia `span`: `None`. Zostaje plik: `error: ownership: returning the result of concat....`

Odpowiedzi B

B1. 10.

B2. 3.

B3. `Stdout` hi plus nowa linia. Kod 0, bo `main` bez typu zwraca `unit`, a `codegen` zamienia to na `i32 0`.

B4. Build się udaje. Proces kończy się `SIGABRT (134)`.

B5. Check przechodzi. Proces aboruje. Indeks nie jest częścią typu.

B6. $0+1+2+3 = 6$.

B7. Checker przechodzi. Build: faza `codegen`, ``main` returning `i64` is not supported by codegen yet`.

Odpowiedzi C

C1. `frontend::check` w `src/frontend.rs`. `Typeck` zwraca diagnostyki, `sema` zwraca błędy, wektor buduje się z `semy` i dopiero `append` typów.

C2. `crates/bork_runtime/src/lib.rs` (stała `ARENA_CAPACITY`, ten plik jest w procesie użytkownika) oraz `src/arena.rs` (model w kompilatorze, nie jest wołany przez `semę`).

C3. `CheckResult.report` jest `None`, gdy `parse` pada. CLI drukuje `dump` tylko przy `Some`.

C4. `None` wnioskuje. `Some([])` to jawne `move ()`.

C5. Komentarz w `src/hir/mod.rs`: regiony żyją w `ArenaReport`. `Codegen` bierze je w `region_walk`, zsynchronizowanym z `HIR`.

C6. `classify_use`, gałąź `BindingKind::Val` po sprawdzeniu `Copy` i tej samej areny.

C7. Panikuje `value_as_int` przez `into_int_value`, wołane z `coerce_value_to_ty` przy `emit_call_with_values`. Bramka ogląda kształt `HIR` (czy jest `Some`, trailing closure, operator), a nie „czy ten argument jest strukturą `LLVM`”. Wywołanie z argumentem `String` wygląda jak zwykły `Call`, więc bramka milczy.

C8. Gramatyka, AST, typeck, sema (jeśli operator rusza nazwy), escape (jeśli rusza bajty), region_walk (jeśli otwiera region; operator zwykle nie), **bramka zanim emisja istnieje**, potem emisja i tests/build.rs. Obowiązkowa przeciw panice jest bramka, dopóki expr.rs nie ma gałęzi dla operatora. Inaczej match na BinOp wpadnie w not_yet_supported albo, gorzej, w ścieżkę int/float i zły into_*_value.

Podsumowanie

- Ćwiczenia A kończą się na tekście diagnostyki. Ćwiczenia B na kodzie procesu.
- Ćwiczenia C wskazują pliki, nie ogólniki.
- 0..n jest otwarty z prawej. Suma 0..5 to 10, suma 0..4 to 6.
- Bramka nie zastępuje coerce_value_to_ty. Stąd osobna odpowiedź C7.

Dodatek A. Ściągawka składni

Stan: frontend tej rewizji. Gwiazdka * oznacza: checker przyjmuje, bork build nie obniża albo panikuje. Szczegóły w dodatku C.

Program

```
fun nazwa(parametry) { instrukcje }  
fun nazwa(parametry): Typ { instrukcje }
```

Parametr: nazwa: Typ, val nazwa: Typ, var nazwa: Typ. Goły = val. Instrukcje rozdziela NL. Komentarz: //.

Typy

```
i8 i16 i32 i64 u8 u16 u32 u64  
Int Long Byte Float Double      // aliasy i32 i64 u8 f32 f64  
f32 f64 bool unit String  
[T; N]  T?  (A, B) -> R  ((A) -> R)?
```

Copy: nienulłowy prymityw. Reszta nie.

Instrukcje

```
val nazwa = wyrażenie  
val nazwa: Typ = wyrażenie  
var nazwa = wyrażenie  
var nazwa: Typ = wyrażenie  
nazwa = wyrażenie  
nazwa[indeks] = wyrażenie  
{ instrukcje }  
for (nazwa in lo..hi) { instrukcje }  
while (warunek) { instrukcje }  
break  
continue  
return  
return wyrażenie  
move { instrukcje }
```



```

move () { instrukcje }
move (a, b) { instrukcje }
wyrażenie

for: lo..hi po i32, bez hi. while: warunek bool.

```

Wyrażenia

Od najciaśniejszego: wywołanie i przyrostki, !, * /, + -, porównania, .., &&, ||, ?:.

```

nazwa
liczba
liczba.liczba
"napis"           // \n \r \t \\ \"
true false
[e, e, e]
nazwa[i]
nazwa[lo..hi]     // lo i hi literały i32
nazwa.pole
nazwa?.pole
wyrażenie!!
wyrażenie ?: wyrażenie
Some(wyrażenie) None
move nazwa
promote nazwa
f(a, b)
f(a) { x -> instrukcje }          *
f(a) move (s) { x -> instrukcje } *
- nie istnieje jako minus jednoargumentowy

```

if (warunek) { } else { } jest wyrażeniem. Bez else, gdy nikt nie oczekuje typu, wynikiem jest unit.

Własność w jednej tabeli

Sytuacja	Zapis
Copy (i32, bool, ...)	goła nazwa
rodzic val nie-Copy	goła nazwa (Shared)
rodzic var nie-Copy	move nazwa
val nie-Copy do parametru var	move nazwa
literał do parametru var	bez move (codegen String dziś panikuje) *
zapis do zewnętrznego var	outer = ... albo outer = move inner albo outer = promote held
return napisu	return nazwa na głębokości funkcji albo return "literał"

Wbudowane

```
print(i32 | i64 | String)
println(i32 | i64 | String)
concat(String, String) -> String
```

Polecenia

```
bork plik.bork
bork --dump-arenas plik.bork
bork build plik.bork
bork build -o wyjście plik.bork
cargo test --workspace
cargo test --workspace --features codegen
```

Kody: 0 sukces, 1 zły program, 2 złe wywołanie lub toolchain, 101 panic procesu bork, 134 SIGABRT procesu użytkownika (abort).

Fazy diagnostyki

parse, ownership, type, codegen.

Dodatek B. Słowniczek

Arena. W runtime: płyta 4096 bajtów i offset. W semie: węzeł ArenaNode w drzewie regionów. To nie jest ten sam obiekt.

Arena pool. Lista wolnych płyt w `bork_runtime`. `pop` oddaje płytę, `push` ją bierze. `reset` nie oddaje.

Assign-up. Przypisanie do `var` z regionu przodka. Nie jest odczytem lewej strony. Prawa strona może alokować w arenie celu.

AST. Drzewo z `src/ast.rs` po parserze, bez typów wywnioskowanych.

Bramka (gate). Przejście po HIR, które odrzuca konstrukcje spoza podzbioru codegen, zanim powstanie moduł LLVM.

Bump. Alokacja przez przesunięcie offsetu. Brak zwalniania pojedynczego obiektu.

Check. `frontend::check`: `parse`, `typeck`, `sema`, a przy braku błędów także `stempel`, `hoist` i `escape`.

Codegen. Feature Cargo i faza diagnostyki. Obniżenie HIR do obiektu LLVM i linkowanie.

Copy. Przejście granicy regionu dla nienulowalnego prymitywu. Źródło zostaje.

Deskryptor. Wartość `{ ptr, len }` dla `String` i `[T; N]`. Bajty są gdzie indziej.

Escape. Pass `escape::place`. Pilnuje, żeby bajty nie były użyte po śmierci płyty. Diagnostyka ma fazę `ownership`.

HIR. Typowane drzewo. Nie niesie identyfikatora regionu.

Hoist. Rozpoznanie dwóch sąsiednich instrukcji i ustawienie `alloc_in_binding`, żeby bajty powstały od razu w arenie celu.

Local. Nazwa zadeklarowana w tym regionie.

Move. Przeniesienie własności. Wiązanie źródłowe jest martwe. Bajty w starej płycie zostają śmieciem aż do `reset`.

Ownership. Faza diagnostyki `semy` i `escape`. Nie jest osobnym `passem` o tej nazwie w kodzie.

Promote. `promote` nazwa przy `assign-up`. Kopiuje bajty do areny zewnętrznego `var` i unieważnia źródło.

Region. Blok, ciało funkcji, gałąź if, ciało pętli albo domknięcie, o ile `peel_blocks` go nie skleił z otoczeniem.

Shared. Odczyt `val` nie-Copy z regionu rodzica. Dziecko nie kopiuje bajtów i nie zabija nazwy.

Sink. Miejsce, które konsumuje wartość: cel przypisania, `return`, albo wynik `concat`. Steruje, gdzie `bump` położy bajty wyniku.

Span. Para offsetów bajtowych w pliku źródłowym.

UseKind. Adnotacja HIR na identyfikatorze: `Local`, `Shared`, `Copy`, `Move`, `Promote`. Nie zastępuje analizy semy.

codegen_push. Flaga na `ArenaNode`. LLVM woła `bork_arena_push` tylko gdy jest ustawiona (korzeń funkcji zawsze).

decl_tys. Wektor typów deklaracji w kolejności źródła, przekazany z typecku do semy.

loop_move_ban. Zbiór nazw, których nie wolno przenieść w ciele pętli, bo były widoczne na wejściu.

peel_blocks. Zdjęcie zagnieżdżonych bloków, które zawierają tylko jeden blok. Licznik łąduje w `compacted_braces`.

unit. Typ funkcji bez zadeklarowanego wyniku. Nie jest nazwą `Unit`.

Dodatek C. Niedokończone, rozjazdy i paniki kompilatora

Ten dodatek jest listą miejsc, w których kod jest węższy niż język frontendu, dokumentacja jest węższa albo szersza niż kod, albo kompilator zachowuje się w sposób, którego nie da się nazwać diagnostyką. Nic z tej listy nie jest propozycją naprawy. Książka nie zmienia kompilatora.

Świadomie niezaimplementowane

Z `escape.rs`, `docs/language.md` i `docs/memory-model.md`:

- arena wyniku po stronie wołającego, więc `return move`, `return concat` i `return` bajtów z wewnętrznego regionu są odrzucane,
- `promote` na `return`,
- `hoist` poza wzorcem dwóch sąsiednich linii,
- elizja `memcpy`, gdy bajty już leżą w puli celu,
- osobna arena tymczasowa na czas zwykłego wołania,
- `codegen` trailing closures, `Some`, `None`, `!!`, `?:`,
- `codegen` pól innych niż `.length`,
- `main` z parametrami oraz `main` zwracające typ inny niż `i32` i `unit`,
- wywołanie pośrednie,
- moduły, struktury, generyki, wyjątki, `borrow checker`, `GC`.

Z `TODO.md` (refactor, nie semantyka języka): rozcięcie visitora `codegenu`, kursor `aren` zamiast makra, test zgodności bramki z frontendem, więcej złotych testów `push/pop`, uporządkowanie `docs/superpowers/plans/`.

Rozjazdy dokumentu z kodem

Dokument	Mówi	Kod robi
<code>docs/language.md</code> , przykład <code>concat(left, right)</code> przy <code>var</code>	program jest wzorcem	<code>checker: `left` is not Copy; move it into `Block`</code>
<code>docs/language.md</code> , pierwszeństwo	pomija <code>&& \ \ !</code>	gramatyka ma te piętra

Dokument	Mówi	Kod robi
docs/superpowers/specs/2026-09-23-typed-hir-llvm-design.md	LLVM 18, kolejność „własność potem typy”	LLVM 23, typeck potem sema
specyfikacja LSP	diagnostyki parsera	frontend::check
specyfikacja MVP aren	brak pełnego typecku	typeck jest
README, nagłówek „Parse diagnostics”	parse	akapit niżej i kod: pełny check
specyfikacja parsera	wynik Unit, brak spanów	prymityw unit, spany są
language.md o MIN / -1	runtime abort	strażnik w guard_int_div jest; literału ujemnego nie da się zapisać

Paniki i ostre krawędzie, sprawdzone

Uruchomione bork build (LLVM 23.1.2, rustc 1.98.1):

1. **Argument String do funkcji użytkownika** (f("hello"), f(move s), show(s) dla val s) panikuje w src/codegen/llvm/expr.rs w value_as_int (into_int_value). Frontend jest czysty. Bramka milczy. Kod procesu kompilatora 101.
2. **Porównanie f64** (if (x > 1.0) przy x: f64) panikuje w tym samym miejscu, bo wartość jest FloatValue. **Dodawanie f32** daje diagnostykę, nie panikę: internal arena schedule mismatch: float binary after walk is not supported by codegen yet. Dwa operatory float, dwa różne złe zakończenia.
3. **Indeks poza zakresem i dzielenie przez zero** kompilują się. Proces użytkownika kończy się SIGABRT, kod 134, bez tekstu Bork.
4. **Przepelnienie areny** nie było odpalane dużym literałem w książce (trudno przekroczyć 4096 krótkim przykładem bez pętli alokującej, a pętla resetuje płytę). Kod w bork_runtime panikuje tekstem arena overflow: need {end} bytes, capacity 4096. Test jednostkowy runtime to zamyka. To nie jest diagnostyka kompilacji.
5. **return concat i część błędów escape** nie mają spanu. CLI pomija linię i kolumnę.
6. **ParseError::User** (zły cel przypisania, za duża liczba) wskazuje koniec pliku.
7. **Sema parametrów trailing closure** używa Ty::Unknown. Dump pokazuje Shared dla parametrów Int, które typeck uważa za Copy. Check przechodzi. To mylące przy czytaniu --dump-arenas, nie jest błędem użytkownika.
8. **Komunikat escape o gałęzi if** mówi String, a warunek obejmuje każdy uses_arena_storage (także tablicę).
9. **u32 / alias Byte w main(): i32** nie jest ograniczeniem codegenu szerokich intów. i64 w funkcji pomocniczej i porównanie działają (kod wyjścia 7 na sub). Ograniczenie main dotyczy typu **wyniku** main, nie lokalnego i64.
10. **src/arena.rs kontra bork_runtime**. Dwie stałe 4096. Sema nie woła żadnej. Łatwo „naprawić” zły plik.

11. **Opakowanie `internal arena schedule mismatch`.** Potrafi ukryć zwykłe „not supported yet” w tekście, który brzmi jak błąd wewnętrzny. Czasem jest błędem wewnętrznym (rozjazd dzieci areny), czasem jest tylko prefiksem.
12. **Specyfikacje w `docs/superpowers`** opisują świat przed tablicami, przed `while` i przed LLVM 23. `TOD0.md` każe je przenieść, gdy wchłoną się w ten plik albo w zamknięte PR. Nie są mapą kodu.

Co jest stabilne

Żeby lista dziur nie przesłoniła reszty: parser, typeck liczb i tablic, sema Copy/Shared/Move, dump aren, for/while/break/continue, println liczb i napisów w main, concat i promote w main, wycinki o literałowych granicach oraz linkowanie z runtime są pokryte testami `tests/build.rs` albo `frontend::check` i zostały powtórzone przy pisaniu tej książki. Dziury są na brzegach: nullable w codegenie, floaty, String jako argument wołania, powrót świeżego napisu.

Jak czytać niejasność

Gdy komentarz, specyfikacja i kod się różnią, kolejność zaufania przyjęta w książce jest taka: test wykonawczy (`tests/build.rs`), potem `frontend::check` na przykładzie, potem kod fazy, potem `docs/language.md`, na końcu `docs/superpowers`. Hasło README o borrow checkerze i GC jest zgodne z modelem. Przykład concat na dwóch var nie jest.